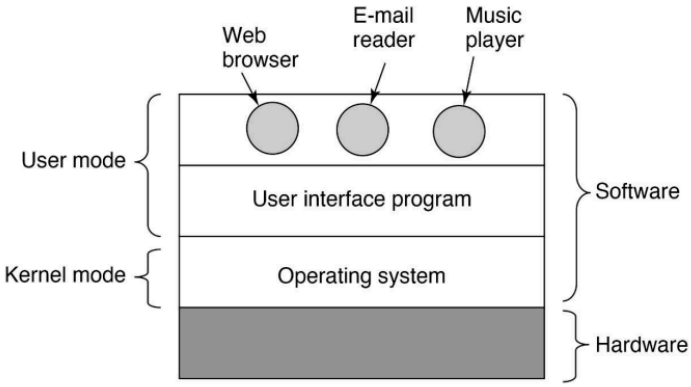
SISTEMI OPERATIVI E RETI DI CALCOLATORI (clicca per indice)

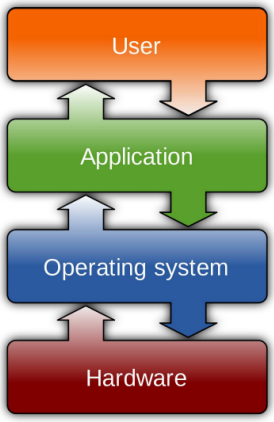
Sistemi operativi

I **sistemi operativi** esistono poiché rappresentano una soluzione ragionevole al problema di realizzare un sistema elaborativo che si possa impiegare facilmente, per eseguire i programmi e agevolare la soluzione dei problemi degli utenti.
Il sistema operativo è il solo programma che funziona sempre nel calcolatore, generalmente chiamato **kernel** (nucleo).



Oltre al kernel vi sono due tipi di programmi: i **programmi di sistema**, associati al sistema operativo, ma che non fanno necessariamente parte del kernel, e i **programmi applicativi**, che includono tutti i programmi non correlati al funzionamento del sistema.
I sistemi operativi mobili sono costituiti esclusivamente da un kernel, ma anche da un **middleware**, ovvero da una collezione di ambienti software che fornisce servizi aggiuntivi per chi sviluppa applicazioni.

Le funzioni principali del S.O. sono:
- la gestione dell'unità centrale di elaborazione (CPU);
- la gestione della memoria voltatile (RAM);
- la gestione della memoria di massa (Hard Disk);
- la gestione dell'I/O (Input/Output);
- la gestione dei processi;
- la gestione dei file (file system).



Inoltre, fornisce anche:
- l'interfaccia per l'utente (user, admin o developer);
- l'interprete dei comandi o shell;
- la gestione della sicurezza;
- il controllo accesso alle risorse.

Definizione 5**W** - 1**H** del Sistema Operativo

(**W**ho) **SO**: l'unico programma che funziona sempre nel calcolatore.
(**W**hy) **Obiettivo principale**: virtualizzazione dell'elaborazione (rendere il sistema più possibile "user-friendly" per gli utenti.
(**H**ow) **Compito principale** : controllo delle applicazioni attive (il funzionamento del S.O. necessita della creazione di questa nuova fattispecie: l'applicazione).
(**W**hat) **Operazione**: computer resource management (le risorse del sistema).
(**W**here) **Generalmente adottato**: computer general purpose (elaboratori).
(**W**hen) **Compleanno ufficiale**: 01/01/1970

Tipologie di Sistemi di Elaborazione

Tradizionalmente il computer è visto come una macchina sequenziale: un processore esegue le istruzioni una alla volta, in sequenza, e ogni istruzione è a sua volta una sequenza di operazioni.

Nel tempo si sono affermate diverse tipologie di sistemi di elaborazione:
- sistemi monoprocessore;
- mainframe classico;
- personal computer;
- sistemi multiprocessore;
- sistemi distribuiti;
- cluster;
- cloud computing;
- mainframe moderno;
- computer palmari e smartphone;
- sistemi multimediali;
- sistemi di elaborazione in tempo reale;
- sistemi dedicati (embedded system, IoT, OT).

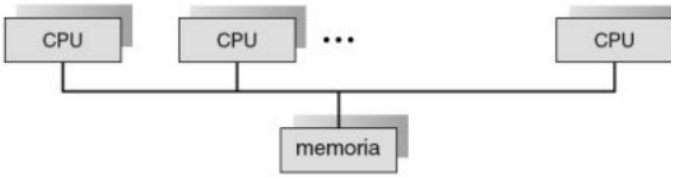
Sistemi Monoprocessore

Un **sistema monoprocessore** dispone di un'unica CPU centrale, che esegue istruzioni di natura generale. Accanto a questa lavorano alcuni processori secondari, ciascuno con un compito particolare, come il controllore del disco, quello della tastiera o il BIOS. Questi processori:
- svolgono compiti specifici;
- eseguono solo un insieme ristretto di istruzioni;
- non eseguono processi utente, che restano di competenza della CPU centrale.

I **sistemi desktop** di oggi permettono un'interazione evoluta con i sistemi centrali, pur svolgendo localmente anche piccole attività di elaborazione. Dispongono di grafica e di dispositivi per l'interazione avanzata e sono spesso sistemi multiprocessore integrati.
Per la gestione delle risorse il sistema operativo usa tre tecniche collegate:
- **Multiprogrammazione**: la memoria viene ripartita tra più processi, che restano caricati contemporaneamente;
- **Multitasking**: la CPU viene condivisa tra questi processi;
- **Time sharing**: è il modo in cui avviene la condivisione. Il tempo della CPU è diviso in piccole fette, assegnate a turno ai processi, così che l'utente abbia l'impressione che vadano avanti tutti insieme.

Sistemi Multiprocessore

Un **sistema multiprocessore** è formato da più processori in stretta comunicazione tra loro. Nei sistemi a processori strettamente connessi (*tightly coupled*), i processori condividono la **memoria**, i **bus** e l'**orologio** (*clock*), e la comunicazione avviene di solito attraverso la memoria condivisa.



Vantaggi dei sistemi paralleli:
- **Maggiore quantità di elaborazione**: con n unità la velocità teorica è n volte quella di una sola (in pratica un po' meno, perché i processori devono coordinarsi).
- **Economia di scala sulle periferiche**: le periferiche sono condivise tra i processori invece di essere duplicate.
- **Maggiore affidabilità**, che si realizza in due modi:

- **Graceful degradation**: il sistema degrada in modo progressivo, cioè le prestazioni calano in proporzione al numero di guasti, senza fermarsi del tutto;
- **Fault tolerance**: il sistema tollera i guasti, ma richiede il riconoscimento del guasto, la diagnosi ed eventualmente la riparazione.

Sistema multiprocessore asimmetrico:
- Ogni processore ha un lavoro specifico assegnato.
- Un processore principale (*master*) organizza e gestisce il lavoro dei processori *slave*.
- L'organizzazione è gerarchica.
- È più comune nei sistemi molto grandi.

Sistema multiprocessore simmetrico (**SMP**):
- Ogni processore può eseguire tutte le operazioni.
- L'organizzazione è non gerarchica: i processori sono alla pari.
- Molti processi possono girare contemporaneamente senza deterioramento delle prestazioni.
- Serve un bilanciamento del carico tra i processori.
- La maggior parte dei sistemi moderni supporta SMP.

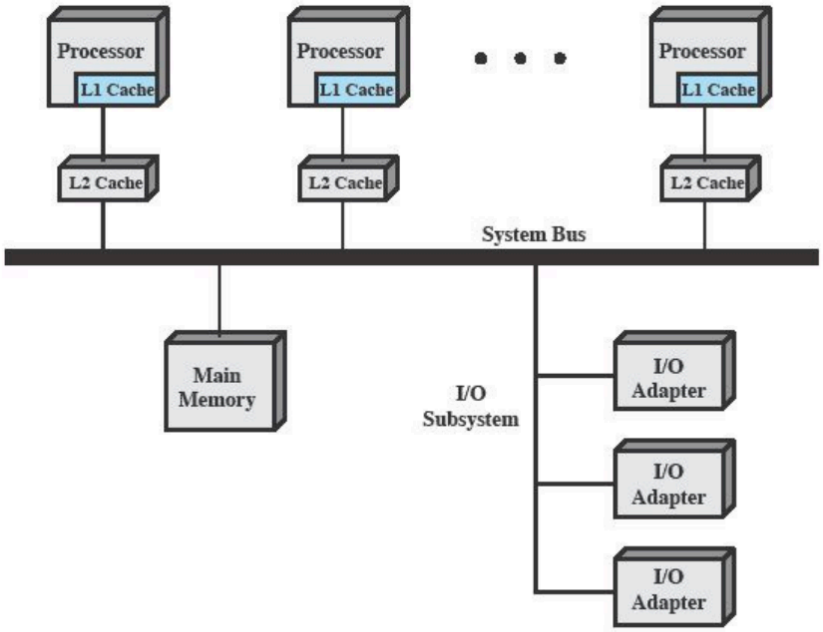
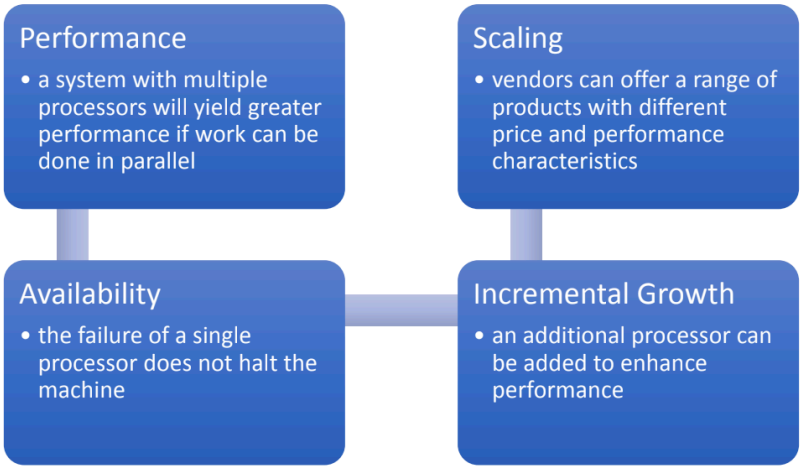


Figure 4.9 Symmetric Multiprocessor Organization

Caratteristiche di un sistema SMP

Un sistema SMP è un sistema di calcolo autonomo (stand-alone) con le seguenti caratteristiche:
- due o più **processori simili** e di capacità comparabile;
- i processori condividono la **stessa memoria principale** e sono interconnessi tramite un bus o un altro schema di connessione interno;
- i processori condividono l'accesso ai **dispositivi di I/O**;
- tutti i processori possono svolgere **le stesse funzioni**;
- il sistema è controllato da un **sistema operativo integrato**, che gestisce l'interazione tra i processori e i loro programmi a livello di job, task, file ed elementi di dati.

I principali problemi di progettazione sono:
- Processi o thread concorrenti simultanei;
- Scheduling;
- Sincronizzazione;
- Gestione della memoria;
- Affidabilità e tolleranza ai guasti.

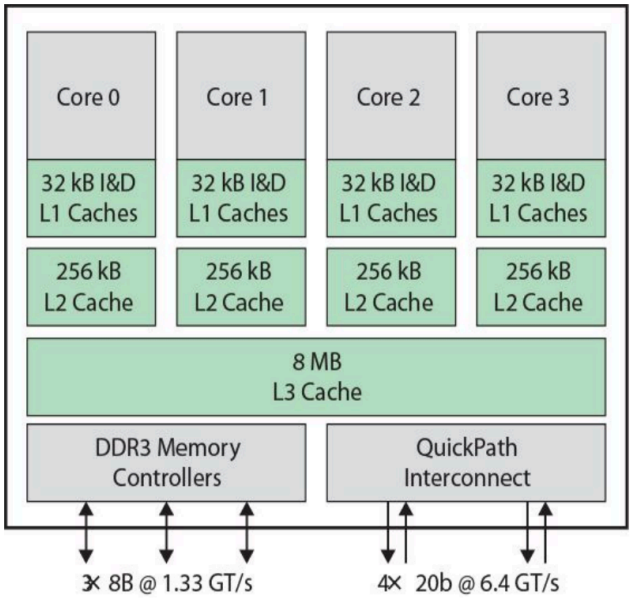


Computer multicore

Un computer multicore è detto anche **chip multiprocessor**. Combina due o più processori (chiamati **core**) su un unico pezzo di silicio (**die**):
- Ogni core contiene tutti i componenti di un processore indipendente.
- Inoltre, i chip multicore includono anche la **cache L2** e, in alcuni casi, la **cache L3**.

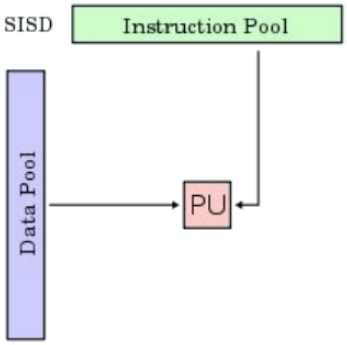
Es. Intel Core i7

Il chip supporta due forme di comunicazione esterna verso altri chip:
- **Controller di memoria DDR3**:
  - Porta sul chip il controller di memoria per la memoria principale DDR (*double data rate*).
  - Con il controller di memoria sul chip, il **Front Side Bus** viene eliminato.
- **QuickPath Interconnect** (**QPI**):
  - Permette comunicazioni ad alta velocità tra i chip di processori collegati.

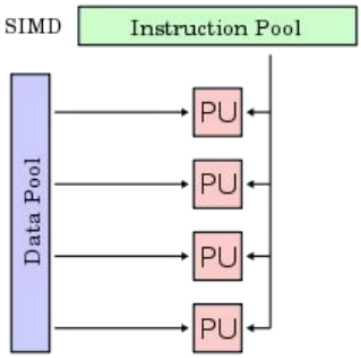


Categorie di sistemi di calcolo (tassonomia di Flynn)

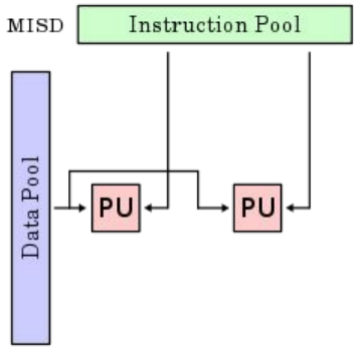
1. **Single Instruction Single Data** (**SISD**):
   - Un singolo processore esegue un singolo flusso di istruzioni per operare su dati memorizzati in un'unica memoria.



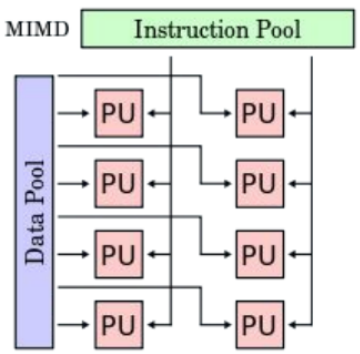
2. **Single Instruction Multiple Data** (**SIMD**):
   - Ogni istruzione viene eseguita su un diverso insieme di dati dai diversi processori.



3. **Multiple Instruction Single Data** (**MISD**):
   - Una sequenza di dati viene trasmessa a un insieme di processori, ciascuno dei quali esegue una diversa sequenza di istruzioni.



4. **Multiple Instruction Multiple Data** (**MIMD**):
   - Un insieme di processori esegue simultaneamente diverse sequenze di istruzioni su diversi insiemi di dati.



Sistemi Distribuiti

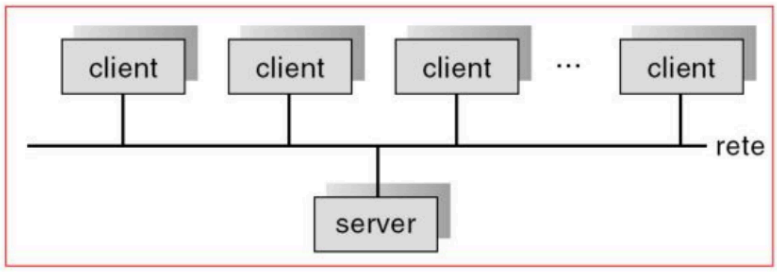
Nei sistemi distribuiti il calcolo viene distribuito tra diversi elaboratori **fisicamente distinti**, che possono essere **eterogenei**.

Questi sistemi sono **sistemi lascamente connessi**:
- Ogni processore possiede una propria memoria locale;
- I processori comunicano tra loro mediante linee di comunicazione, come bus ad alta velocità o linee telefoniche.

i Vantaggi dei sistemi distribuiti:
- **Condivisione delle risorse**:
  - dati;
  - servizi.
- **Rapidità di calcolo**: distribuzione del carico.
- **Affidabilità**.

I sistemi distribuiti necessitano di un'infrastruttura di rete, che può essere:
- una rete locale (*Local Area Network*, **LAN**);
- una rete geografica (*Wide Area Network*, **WAN**).

Inoltre, possono essere organizzati come sistemi **client-server** o punto-a-punto (**peer-to-peer**), oppure possono sfruttare infrastrutture di **cloud computing**.



Cluster

Un **cluster** è un'architetture formata da più computer fortemente connessi tra loro, realizzata usando computer disponibili sul mercato.
Rispetto ai sistemi SMP ha una capacità di elaborazione superiore e permette di eseguire contemporaneamente un'applicazione su più PC. Per questo richiede la **programmazione parallela**, cioè programmi con componenti eseguibili in parallelo.
Le caratteristiche principali sono:
- **economie di scala sulle periferiche**;
- possono essere **simmetrici** o **asimmetrici**;
- **affidabilità del sistema in caso di guasti**: ogni PC è controllato da almeno un altro PC, che recupera il lavoro in caso di guasto (*Business Continuity*).

Cloud Computing

Il **cloud computing** è un insieme di tecnologie informatiche che permettono di utilizzare risorse **hardware** (ad esempio storage, CPU) o **software**, distribuite e virtualizzate in rete.
Il termine the cloud significa "nuvola" di risorse, le cui caratteristiche non sono note all'utilizzatore.
Si usa con il modello **pay-as-you-go**, cioè si paga in base a quanto si utilizza.
Il cloud computing comprende diversi modelli di servizio:
- **SaaS** (*Software as a Service*);
- **PaaS** (*Platform as a Service*);
- **CaaS** (*Container as a Service*);
- **FaaS** (*Function as a Service*);
- **IaaS** (*Infrastructure as a Service*).

Mainframe classico

Il **mainframe classico** è un'architettura orientata all'elaborazione di **lavori non interattivi** (*job*). E' composto da un processore, una memoria centrale (milioni di gigabyte), numerosi nastri e dischi (circa 1000) e stampanti.
L'elaborazione avviene a **lotti** (*batch*): i tempi di processo si riducono raggruppando i job (processi) in batch con necessità simili. Il sistema esegue numerosi lavori di routine alla volta, con prodigiose quantità di I/O e senza la presenza di alcun utente che interagisca con la macchina.
Esistono due tipi di sistemi:
- **sistemi monoprogrammati**: la CPU è sottoutilizzata;
- **sistemi multiprogrammati**:
  - la memoria centrale è ripartita tra i job (**multiprogrammazione**);
  - la CPU è condivisa (**multitasking**).

Mainframe moderno (HPC)

Il **mainframe moderno** è costituito da **grandi server**, spesso raffreddati a liquido. Supporta molti utenti che operano contemporaneamente ed è alla base dei giganteschi server web centralizzati.
E' composto da CPU, memoria centrale, terminali, nastri e dischi, stampanti. Elabora contemporaneamente più flussi di attività (processi) e si occupa dell'elaborazione di transizioni e della condivisione del tempo macchina.
Sono **sistemi multiutente**, che usano tre tecniche:
- **multiprogrammazione**: ripartizione della memoria tra i processi;
- **multitasking**: condivisione della CPU;
- **time sharing**: gestione della CPU in condivisione di tempo.

Sistema dedicato (embedded)

I **sistemi dedicati** sono sistemi di elaborazione pensati per supportare **una sola applicazione**. Ne sono esempi gli elettrodomestici, i sistemi hi-fi, il motore dell'automobile (CANBus), i sistemi biomedicali, le protesi, le carte di credito e cosi via.
Le loro caratteristiche sono:
- tutto il software è su **ROM**;
- hanno ridotte caratteristiche di prestazioni computazionali, memoria e periferiche;
- hanno spesso caratteristiche di **real-time** e **multitasking**.

Sistemi di elaborazione in tempo reale (real-time)

I **sistemi real-time rispondono agli eventi in tempo reale**, fornendo la risposta nel rispetto di rigorosi vincoli temporali. Si distinguono in:
- sistemi in tempo reale stretto (**hard real-time**);
- sistemi in tempo reale lasco (**soft real-time**);

Hanno un'architettura in grado di scambiare segnali con il mondo esterno attraverso **sensori** e **attuatori**, usano:
- schede di acquisizione segnali (input digitali/analogici);
- schede di attuazione controlli (output digitali/analogici).

Sistemi multimediali

I **sistemi multimediali** dispongono di supporti avanzati per l'**interazione multimediale**. Ne sono esempi i sistemi di controllo delle console per giochi (Nintendo Wii, Microsoft Xbox, Sony PlayStation, ecc.), ma anche PuzzleLinux.
In questi sistemi la trasmissione dei dati deve attenersi a **specifiche frequenze**.
Sono **sistemi interattivi multiprocesso**, che usano tre tecniche:
- **multiprogrammazione**. ripartizione della memoria tra i processi;
- **multitasking**: condivisione della CPU;
- **time sharing**: gestione della CPU in condivisione di tempo.

PDA/Smartphone

I **PDA** (*Personal Digital Assistant*) sono sistemi di elaborazione **portatili e di dimensioni estremamente ridotte**, orientati al supporto di attività personali. Ne fanno parte:
- i sistemi palmari;
- i telefono cellulari.

Sono **sistemi interattivi multiprocesso** con:
- ridotto consumo di potenza;
- basso numero di processi.

Gli **smartphone** ne sono un'evoluzione (sistemi operatici iOS e Android) e hanno:
- processori multi-core;
- interfacce multi-touch;
- prestazioni elevate.

Microkernel

Un **microkernel** è un **piccolo nucleo del sistema operativo** che fornisce la base per estensioni modulari.

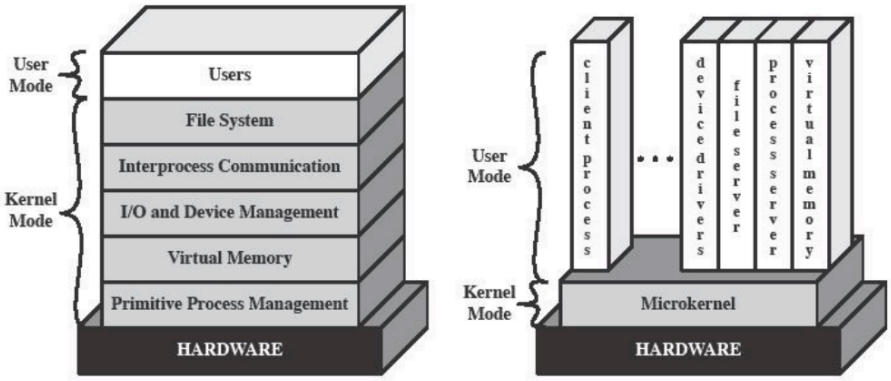


Figure 4.10 Kernel Architecture

Progettazione del microkernel

Gestione della memoria (Memory Management)
- **Gestione della memoria a basso livello**: consiste nel mappare ogni pagina virtuale su un frame di pagina fisica (*physical page frame*).
- La maggior parte dei compiti di gestione della memoria avviene nello **spazio utente** (*user space*).

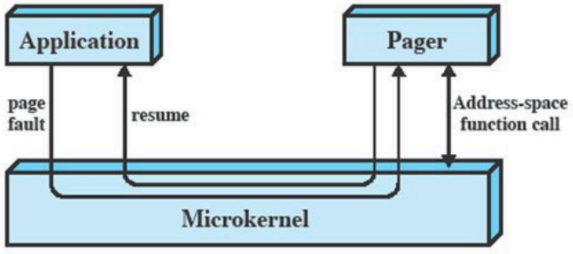


Figure 4.11 Page Fault Processing

Comunicazione tra processi (Interprocess Communication)
In un sistema operativo basato su microkernel, la comunicazione tra processi o thread avviene tramite **messaggi**.

Un messaggio comprende:
- un **header** (intestazione), che identifica il processo mittente e quello destinatario;
- un **body** (corpo), che contiene dati diretti, un puntatore a un blocco di dati oppure alcune informazioni di controllo sul processo.

Gestione di I/O e interrupt (I/O and interrupt Management)
All'interno di un microkernel è possibile gestire gli **interrupt hardware come messaggi** e includere **le porte di I/O negli spazi di indirizzamento**.
- Un particolare processo a livello utente (*user-level*) viene assegnato all'interrupt, e il kernel mantiene la corrispondenza (*mapping*) tra i due.

Vantaggi di un'organizzazione a microkernel

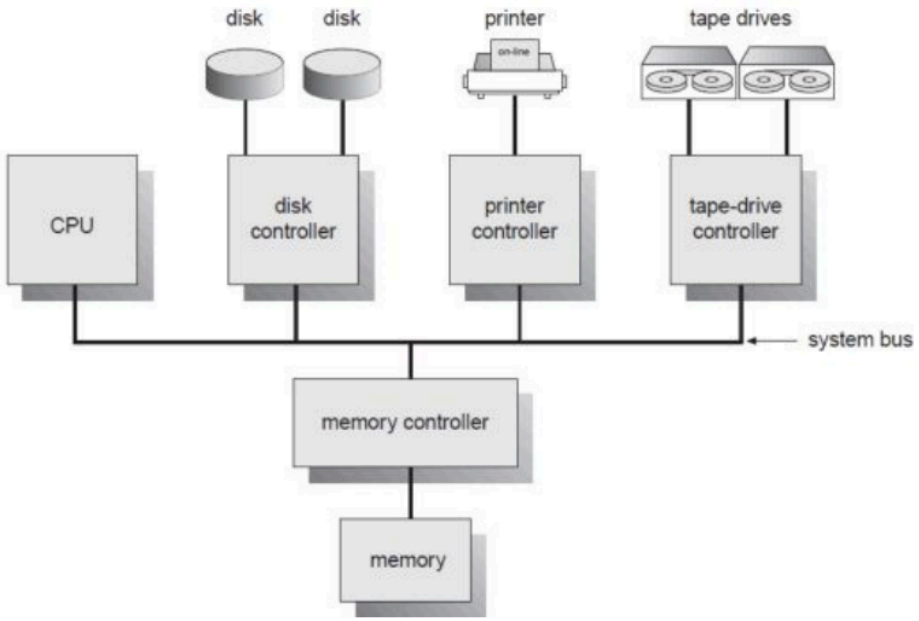
- **Interfacce uniformi** per le richieste fatte da un processo;
- **Estensibilità**;
- **Flessibilità**;
- **Portabilità**;
- **Affidabilità**;
- **Supporto ai sistemi distribuiti**;
- **Sistemi operativi orientati agli oggetti**.

S.O. : Obiettivi

Input/Output

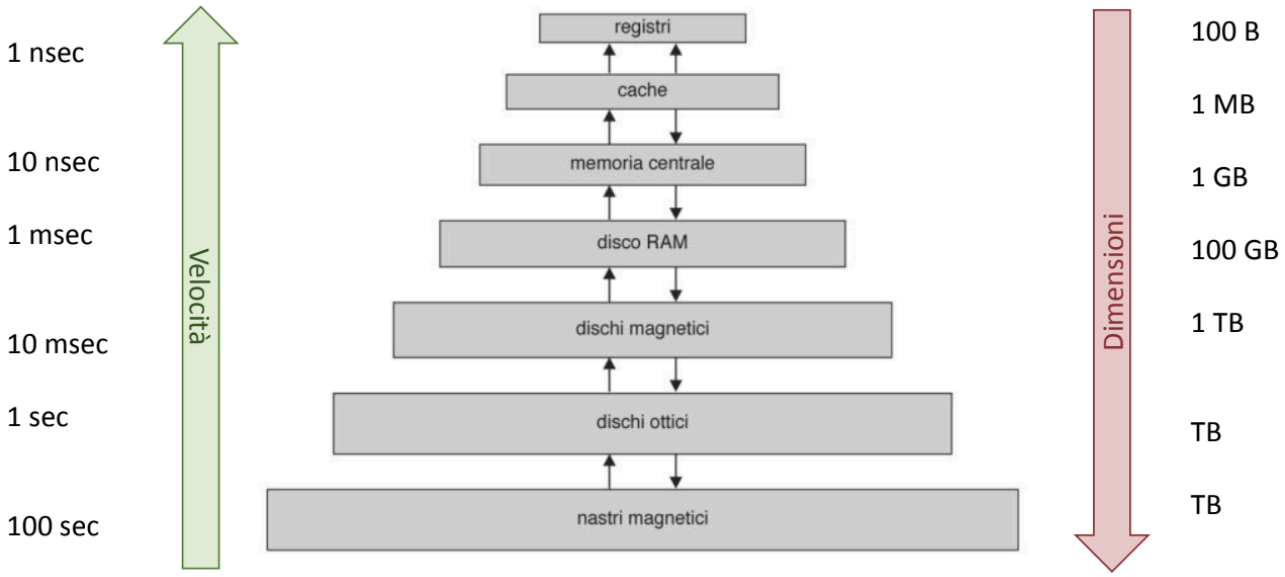
Un calcolatore è composto da una CPU e diversi **controllori di dispositivi**. Sono tre le risorse condivise:
- **Bus**: mezzo condiviso di comunicazione;
- **RAM**: luogo condiviso di allocazione dei dati da elaborare;
- **CPU**: strumenti condiviso di elaborazione.

Lo spostamento dei dati da/verso la CPU prende il nome di **I/O**:
- L'I/O avviene tra il dispositivo e il **buffer locale del controller**;
- La CPU guida poi lo spostamento dei dati (che viaggiano sul **bus**) dai buffer locale dei controller alla memoria, e viceversa.



L'obiettivo è rendere il sistema **quanto meno impattato possibile dalla lentezza dei dispositivi periferici**.

Cache
- La cache di **primo e secondo livello** è specifica per ogni core;
- La cache di **terzo livello** è condivisa tra i core. Questa condivisione è all'origine di vulnerabilità come **Meltdown** e **Spectre**.



Tecniche di I/O
1. I/O programmato;
2. I/O gestito con interrupt;
3. I/O con DMA;
4. I/O con canali (processore separato).

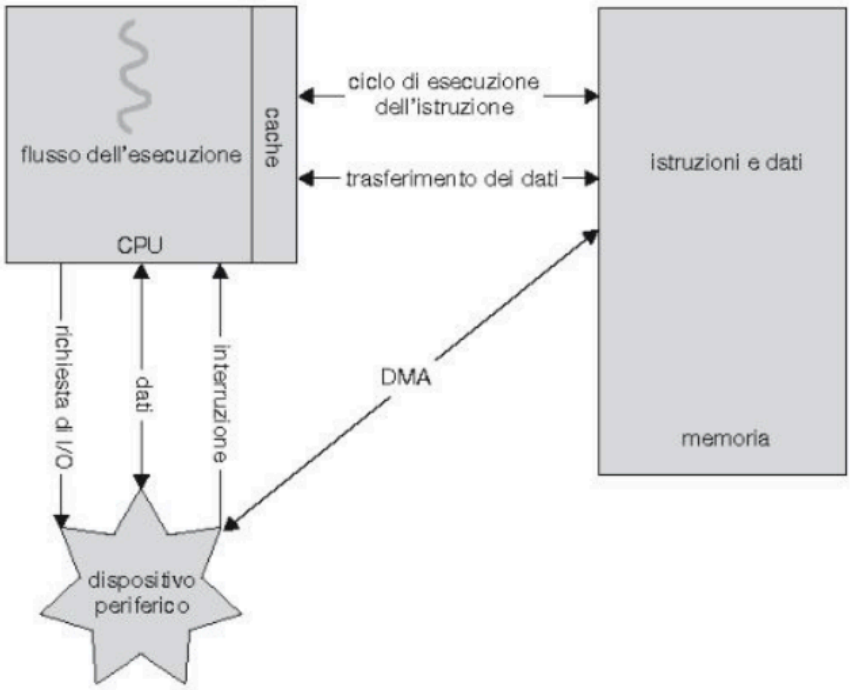
I/O programmato
L'I/O programmato è tipicamente utilizzato da architettura di **fascia bassa** o nel **controllo industriale** con basse necessità di I/O multiplo:
- Per ogni scrittura il processore esegue una **istruzione di I/O**. Per scrivere un blocco di n caratteri, la CPU esegue n istruzioni di output carattere.
- Per ricevere un dato, la CPU legge in un **ciclo stretto** la porta di I/O in attesa del carattere. L'informazione di input disponibile viene tipicamente affidata a un **bit di controllo**.

Svantaggio: non è ottimale nell'ipotesi in cui l'I/O di quantità di dati relativamente grosse occupi una parte rilevante del tempo macchina. Il microprocessore spenderà la maggior parte del suo tempo in operazioni di I/O.

Interrupt
Ogni controllore di dispositivo I/O dispone di una **memoria interna** (**buffer**). Per avviare un'operazione di lettura (*read*) di un byte, il controllore:
- copia i dati sul buffer;
- informa il **driver del dispositivo** tramite **interrupt**;
- il driver cede il controllo al sistema operativo, restituendo un **puntatore ai dati nel buffer**.

Questo metodo è **oneroso per grosse dimensioni** (molto byte).

DMA (**Direct Memory Access**)
Una volta impostato il buffer, il controllore del dispositivo I/O **trasferisce direttamente grandi porzioni di dati** dal dispositivo alla memoria. E' necessario **un solo interrupt**, quindi una sola interazione con la CPU per ogni blocco di byte.

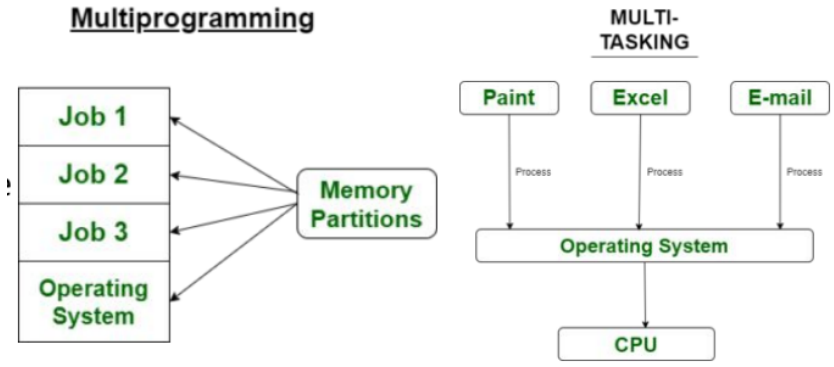


Virtualizzazione

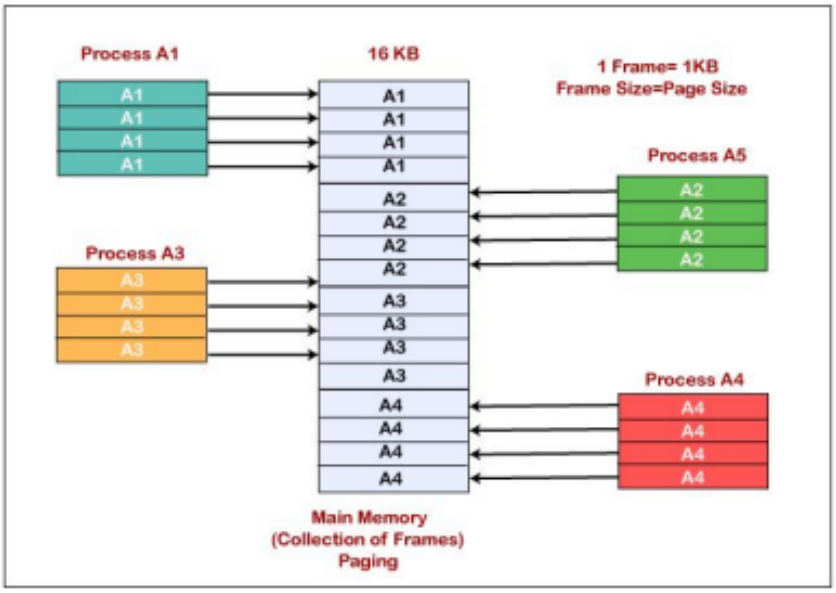
Un **processo** è un programma in esecuzione:
- indipendente dagli altri programmi;
- che non deve interferire con gli altri programmi;
- che trae giovamento dalla gestione ottimizzata delle risorse;
- che deve sottostare all'allocazione e alla condivisione ordinata delle risorse rispetto a spazio e tempo.

L'obiettivo è rendere il sistema quanto più possibile "**dedicato**" all'esecuzione di ogni processo.

Condivisione temporale della CPU
- **Elaborazione batch**: un processo alla volta, dall'inizio alla fine dell'elaborazione, senza comunicazioni intermedie.
- **Multiprogrammazione**: scheduling dei job basato su un corretto *spooling*. Ogni processo rimane in esecuzione fintantoché non genera un interrupt (eccezione). L'I/O viene attivato contemporaneamente al *context switch*.
- **Time-sharing** (**multitasking**): l'alternanza dei processi nella CPU è assegnata a una condivisione equa del tempo di CPU tra i processi. Si usa un **timer**, cioè un registro scalante impostato dal sistema operativo che genera un interrupt.



Condivisione spaziale della RAM
Ogni processo ha accesso solo alla porzione di memoria a lui riservata dal sistema operativo.



Astrazione

Semplificazione dell'uso dell'elaboratore (facilità di programmazione)
- **System call**: esplicita richiesta da parte del programma i accedere a determinati servizi del sistema operativo. E' **l'interfaccia tra processo e sistemo operativo**.
- **Trap**: routine eseguite dal sistema operativo a seguito di un **interrupt software**, dovuto a:
  - un errore nella computazione;
  - il tentativo di eseguire un comando privilegiato essendo in modalità utente.

L'obiettivo è alzare il più possibile il **livello di astrazione** dei componenti del sistema di elaborazione.

S.O. : Funzioni

Error detection

Il sistema deve essere in grado di **rilevare errori** nella CPU o nei dispositivi, come:
- la divisione per 0;
- la fine della carta;
- il guasto di una connessione di rete.

Per ogni errore identificato (**interrupt**) è necessario eseguire un'appropriata azione di ricoveri (**trap**).

L'obiettivo è **impedire che il sistema vada in blocco**.

Dual-mode

Una volta avviato il sistema operativo, la CPU può operare in **due modalità**:
1. **User mode**: la CPU sta eseguendo codice di un utente;
2. **Kernel mode** (*supervisor mode*, *system mode*, *monitor mode*): la CPU sta eseguendo codice del sistema operativo.

La CPU ha un **mode bit** nel registro di stato (*Process Status Word*, o **PSW**) che indica in quale modo si trova: **kernel** (**0**) o **user** (**1**).

Questa modalità serve a:
- **proteggere i registri** da accessi erronei o internazionali;
- consentire l'**accesso completo all'hardware solo in modo kernel**;
- permettere un **passaggio controllato da modo utente a modo kernel**, tramite:
  - interruzioni;
  - chiamate di sistema (*system call*), trap/fault.

L'obiettivo è rendere il sistema quanto più possibile **servo da esecuzioni pericolose**.



Process Management/Memory Management

L'obiettivo è **creare l'ambiente di esecuzione per i programmi**.

Gestione dei processi in relazione alla CPU (**esecuzione**)
- creazione e terminazione dei processi;
- sospensione e riattivazione dei processi;
- schedulazione dei processi;
- sincronizzazione tra processi;
- gestione di situazioni di stallo (*deadlock*);
- comunicazioni tra processi (memoria condivisa o scambio di messaggi).

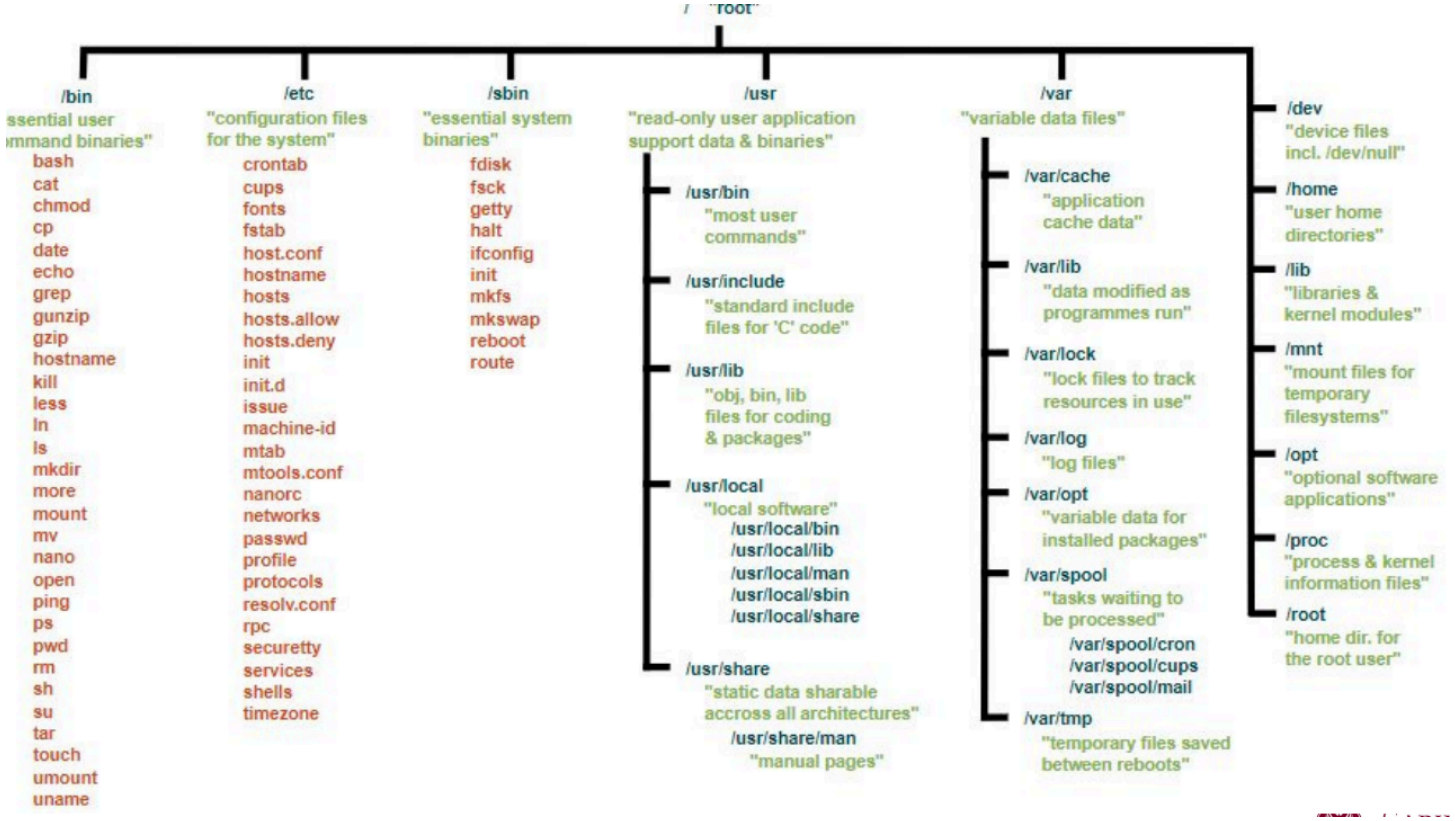
Gestione dei processi in relazione alla memoria (**allocazione**)
- multiprogrammazione;
- allocazione e deallocazione della memoria ai processi;
- caricamento e scaricamento di processi e di loro porzioni in memoria centrale;
- protezione della memoria centrale.

File system

Un **file** è un contenitore di informazioni e dati in formato digitale. Il file system **nasconde all'utente l'organizzazione della memoria** e gestisce:
- file e directory;
- creazione e cancellazione;
- lettura e scrittura;
- copiature;
- ricerca;
- salvataggio e ripristino;
- protezione e sicurezza (diritti di accesso).

In Unix "**Everything is a file**": il file è una pregevole astrazione del sistema operativo.

1. **-** : regular file
2. **d** : directory
3. **c** : character device file
4. **b** : block device file
5. **s** : local socket file
6. **p** : named pipe
7. **l** : symbolic link



Shell/GUI/Transaction

L'obiettivo è rendere il sistema "**user-friendly**" per gli utenti, secondo necessità.

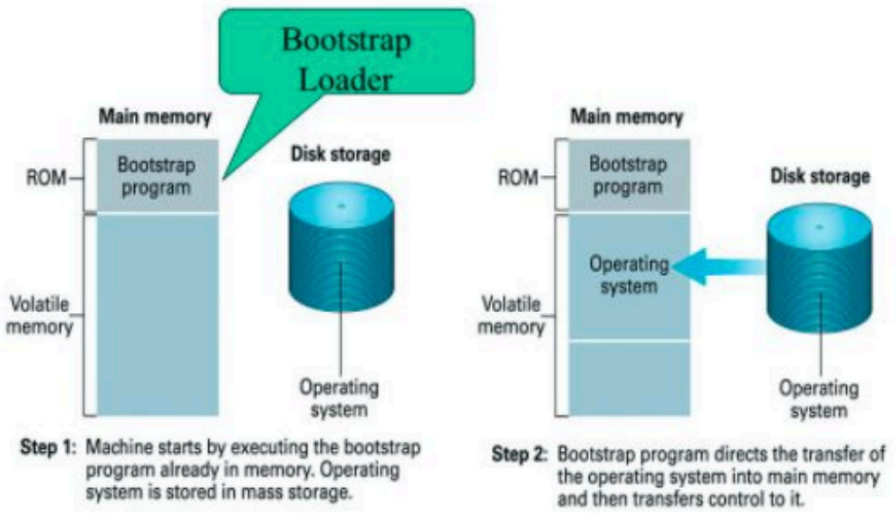
S.O. : Servizi

Boot

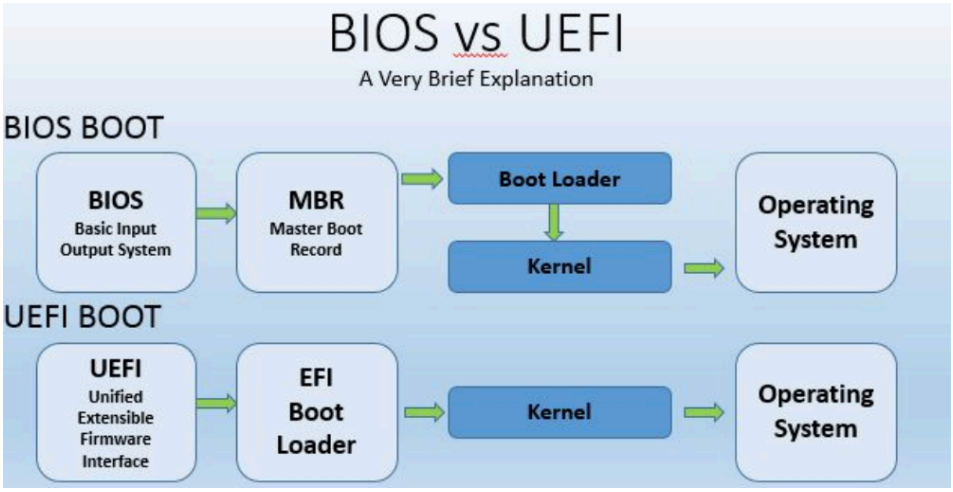
L'obiettivo è **permettere al sistema di ripartire in ogni momento**.

I pasi sono:
1. **Accensione**.
2. **Esecuzione del BIOS** (in ROM o EEPROM), *Basic I/O System*. Inizializza tutti i registri della CPU, i controllori delle periferiche e la memoria centrale.
3. **Esecuzione del bootstrap program** (o *bootstrap leader*). Carica il kernel del sistema operativo da una unità di memoria e lo esegue.
4. **Controllo del sistema** da parte del sistema operativo.

Posizionamento del sistema operativo
In generale, il sistema operativo si trova nel **disco**, in una partizione detta **attiva**, mentre il **bootstrap program** si trova in memoria ROM o EEPROM:
- Il BIOS cerca le periferiche avviatili e da il controllo al bootstrap program.
- Il bootstrap program si avvale di un frammento di codice, che sa in quale parte del disco si trova il sistema operativo. Questo codice risiede in un blocco del disco a una posizione fissa (tipicamente 0), detto **boot block** o **master boot record** (**MBR**).
- La partizione del disco che contiene il boot block è detta **partizione di avvio** (*boot partition*).



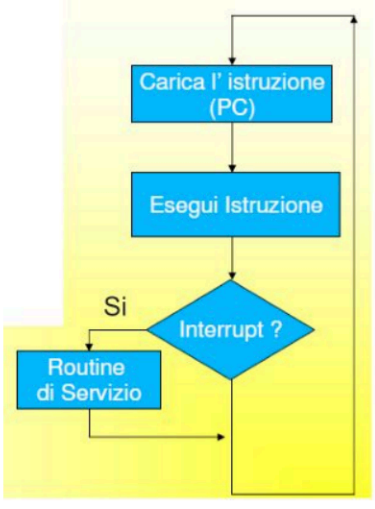
- I moderno sistemi **UEFI** hanno una partizione apposita dedicata alle immagini di sistema operativo.



Interrupt handling

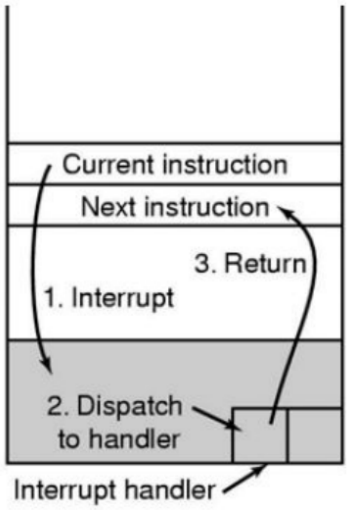
Il sistema esegue sequenzialmente le istruzioni. All'arrivo di un interrupt (**I/O guidato da interrupt**), la CPU invoca un "**gestore**" (*routine di servizio*) opportuno attraverso l'interrupt vector:
- il **vettore di interrupt** contiene gli indirizzi in memoria di tutte le routine di servizio;
- l'hardware deve **salvare l'indirizzo dell'istruzione interrotta**, in modo tale che, una volta ultimata la gestione dell'interrupt, possa essere ripristinato.

Nota: gli interrupt in arrivo sono **disabilitati mentre un altro interrupt viene gestito**, per evitare che vadano persi.



Passi necessari alla gestione di una interruzione:

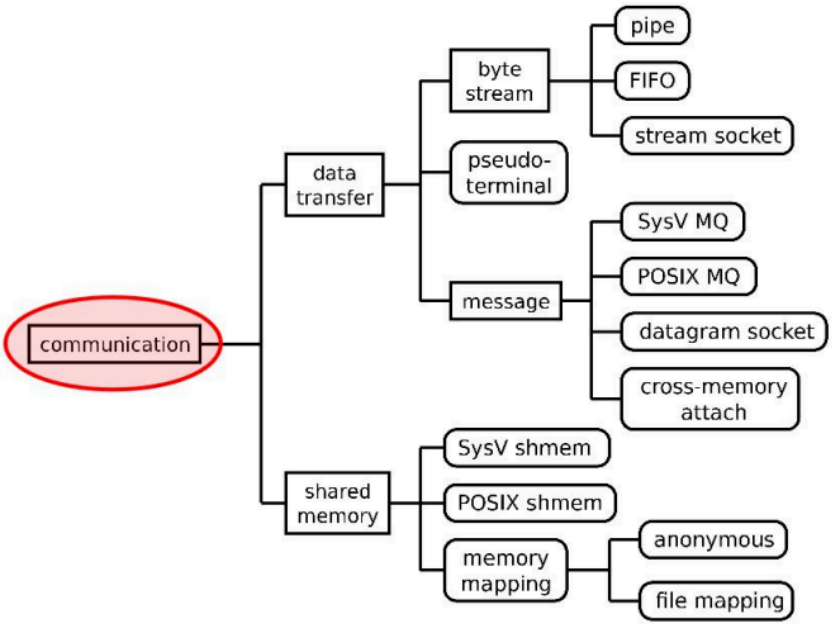
1. La CPU rileva l'interruzione e legge l'indirizzo del dispositivo (device) sul bus:
   - salvataggio dei registro PC e SW (stato della CPU) sullo stack e passaggio in modo kernel.
2. L'indirizzo del dispositivo viene usato come indice nella tabella dei gestori delle interruzioni (interrupt vector);
3. Il gestore selezionato prende il controllo e svolge le operazioni necessarie.
4. Quando il gestore termina:
   - ripristino di PC e PSW, ritorno in modo utente;
   - si esegue l'istruzione successiva a quella interrotta.



IPC (Inter-Process Communication)

L'obiettivo è permettere ai processi di comunicare fra loro:

- **Segnale** (*signals*): per eventi asincroni.
- **Pipe**: per reindirizzare il risultato della elaborazione.
- **Socket**: scambio di datagram.
- **Code di messaggi** (*Message Queue*): per comunicazioni in multicast.
- **Memoria condivisa** (*Shared Memory*): per condividere dati fra processi con trust reciproco.
- **Semaforo** (*Semaphore*): per coordinare processi che laborano su una stessa risorsa (—> *critical region*).

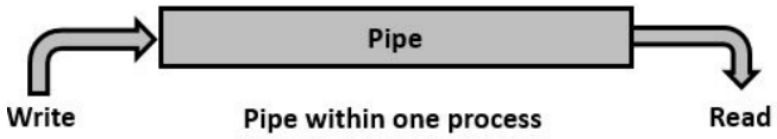


Signal in Unix

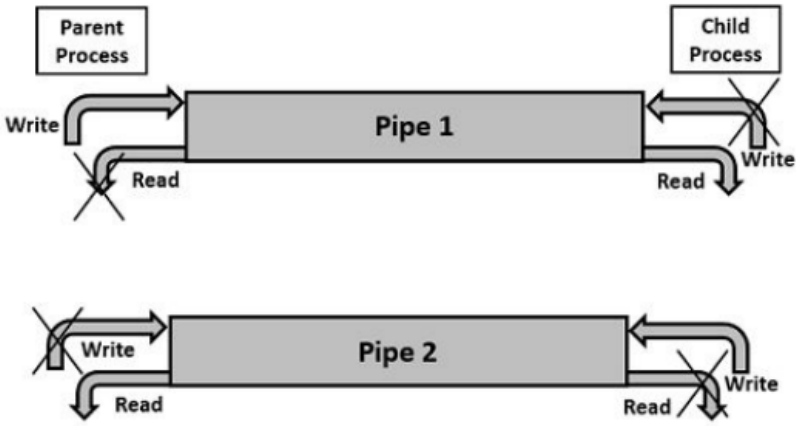
Signal	Value	Action	Comment
SIGHUP	1	Term	Hangup detected on controlling terminal or death of controlling process
SIGINT	2	Term	Interrupt from keyboard
SIGQUIT	3	Core	Quit from keyboard
SIGILL	4	Core	Illegal Instruction
SIGABRT	6	Core	Abort signal from abort(3)
SIGFPE	8	Core	Floating point exception
SIGKILL	9	Term	Kill signal
SIGSEGV	11	Core	Invalid memory reference
SIGPIPE	13	Term	Broken pipe: write to pipe with no readers
SIGALRM	14	Term	Timer signal from alarm(2)
SIGTERM	15	Term	Termination signal
SIGUSR1	30,10,16	Term	User-defined signal 1
SIGUSR2	31,12,17	Term	User-defined signal 2
SIGCHLD	20,17,18	Ign	Child stopped or terminated
SIGCONT	19,18,25	Cont	Continue if stopped
SIGSTOP	17,19,23	Stop	Stop process
SIGTSTP	18,20,24	Stop	Stop typed at terminal
SIGTTIN	21,21,26	Stop	Terminal input for background process
SIGTTOU	22,22,27	Stop	Terminal output for background process

The signals **SIGKILL** and **SIGSTOP** cannot be caught, blocked, or ignored.

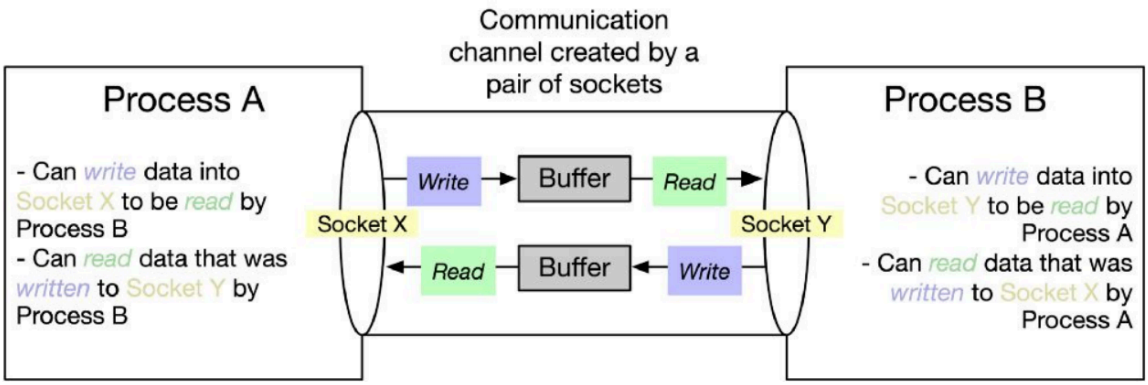
Pipe (datagram FIFO) in Unix
L'obiettivo è **permettere a 2 processi di scambiarsi informazioni fra loro**.
La **pipe** è una comunicazione **one-way**: un flusso (*stream*) **unidirezionale**. Si scrive (write) a un'estremità e si legge (*read*) dall'altra.



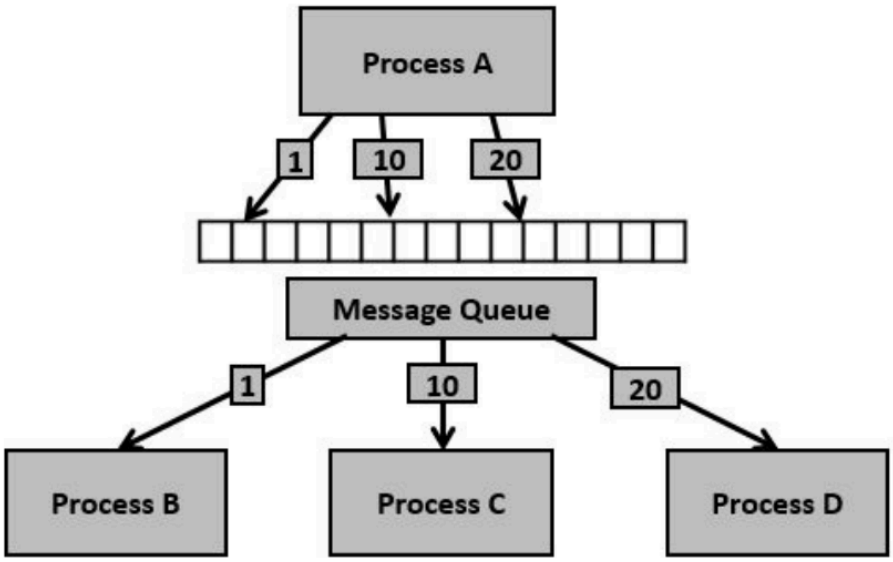
Combinando **2 pipe** si ottiene una comunicazione in entrambe le direzioni tra il processo padre (*parent process*) e il processo figlio (*child process*).



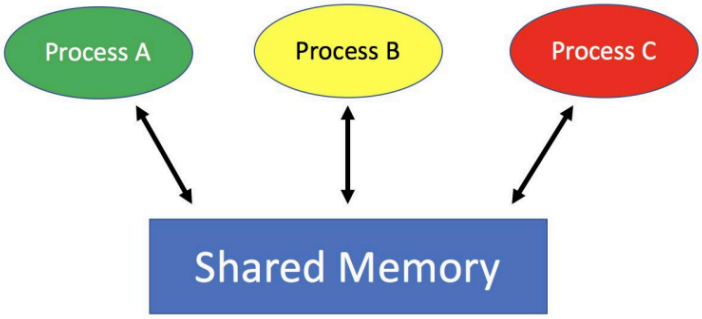
Socket (stream/datagram) in Unix
L'obiettivo è **permettere a 2 processi di scambiarsi informazioni fra loro**.
La comunicazione è one-way: un **flusso** (*stream*) o **blocchi di dati** (*datagram*) **unidirezionale**.



Multicast Queue in Unix
L'obiettivo è **permettere a molteplici processi di scambiarsi informazioni non riservate**.

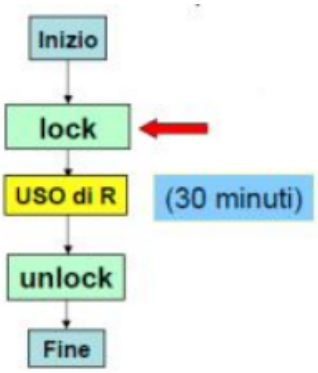


Shared Memory
La **shared memory** è la memoria condivisa fra processi. L'obiettivo è **allocare porzioni di memoria condivisa fra processi**.
I parametri sono:
- **SHMMAX**: massima dimensione (in byte) di un singolo segmento di memoria condivisa;
- **SHMMNI**: numero massimo di segmenti di memoria condivisa presenti sul sistema;
- **SHMALL**: numero totale di pagine di memoria condivisa.

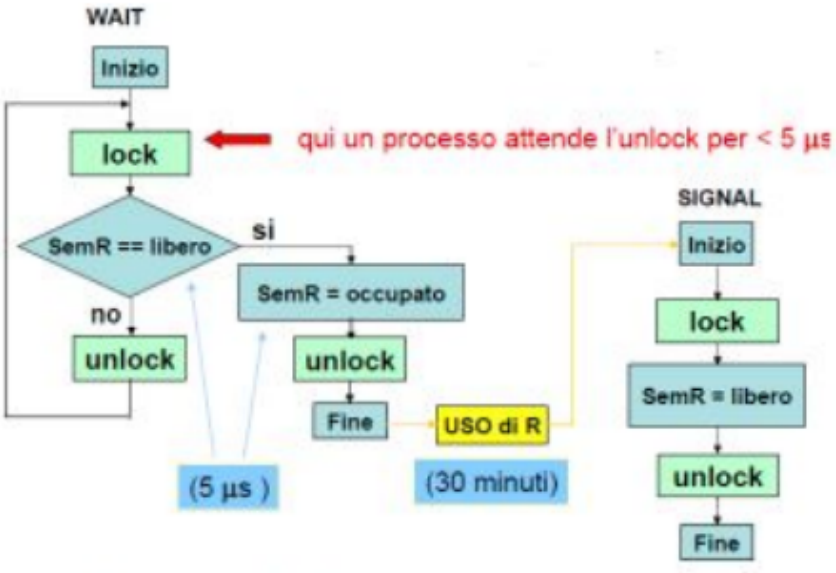


Semaphore in Unix
L'obiettivo è **proteggere una risorsa da accessi concorrenti** e risolvere il problema in modo efficiente in un ambiente multiprogrammato,

- **Sezione critica**: operazioni (istruzioni eseguite da un processo) su variabili di stato di una risorsa comune.
- **Busy waiting**: il processo esegue NOP in attesa che la risorsa bloccata (lock()) venga liberata dal processo che la usa (unlock()).



- **Semaforo**: meccanismo di gestione di una sezione critica, che fa uso di wait() (al posto di lock()) e signal() (al posto di unlock()):
  - la risorsa è guidata da un semaforo che inizialmente vale 1, per indicare lo stato di risorsa libera;
  - il valore del **semaforo binario** è un indicatore che indica se la risorsa è **libera** (1) o **occupata** (2).



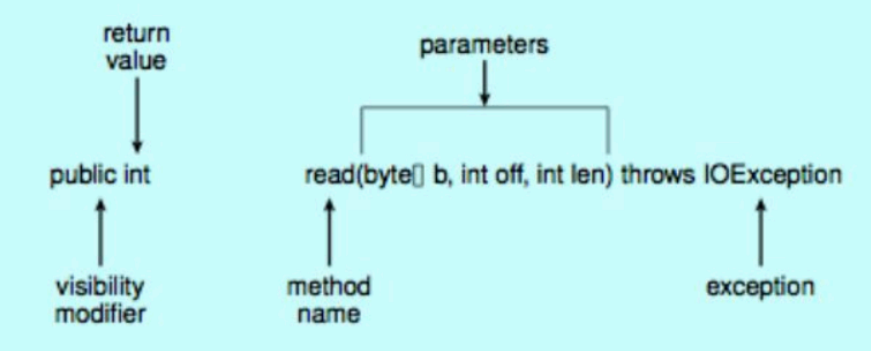
System Calls
L'obiettivo è **offrire ai processi le funzionalità offerte dal sistema tramite una interfaccia semplice**.
Una chiamata di sistema è l'interazione elementare tra un programma utente e il sistema operativo. Con le chiamate di sistema i programmi utente accedono ai **servizi del sistema operativo**, disponibili come istruzioni in:

- linguaggio assembler;
- linguaggi di programmazione di più alto livello come C e C++, che permettono alla chiamate di sistema di avvenire direttamente.

Java non lo permette:

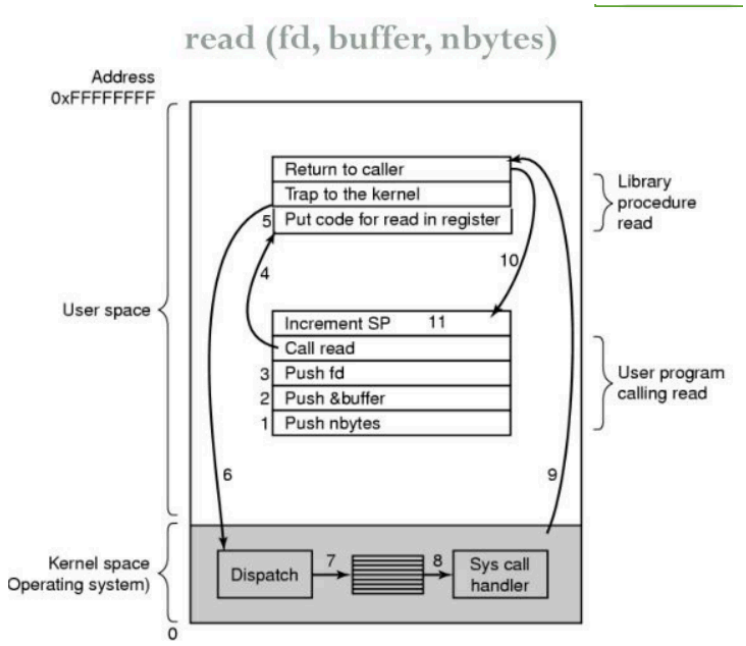
- poiché una chiamata di sistema è specificata dal sistema operativo e da come risulta un codice specifico della piattaforma;
- tuttavia, se richiesto, è possibile attraverso **metodi nativi**; java può richiamate metodi scritti in un altro linguaggio (C o C++) che eseguono la chiamata.

Il **programmatore** tipicamente **non** invoca le **chiamate di sistema**, ma delle funzioni messe a disposizione da un **API**. Ad esempio, il metodo Java read().

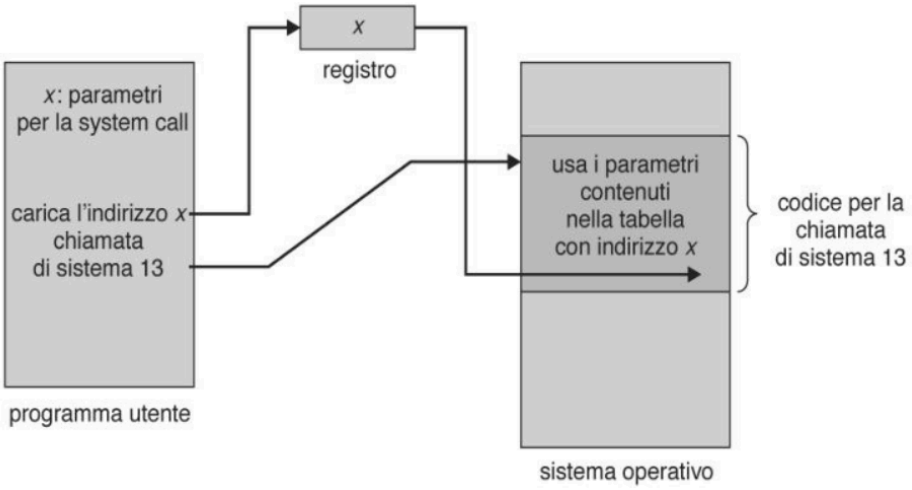


Le chiamate di sistema possono richiedere lo **scambio di informazioni (parametri)** tra un programma e il sistema operativo. Lo scambio dei parametri avviene in generale in **3 modi:**
- **Attraverso registri**: attuabile solo se il numero dei parametri non supera quello dei registri.
- **Attraverso una memoria provvisoria detta pila (stack)**: i parametri sono posti dal programma e prelevati dal sistema operativo. Non limita il numero o la lunghezza dei parametri.
- **Attraverso una tabella in memoria**: l'indirizzo X della tabella viene passato come parametro in un registro.

Dettaglio dell'esecuzione di read (fd, buffer, nbytes)
1. **1-3**: salvataggio dei parametri sullo stack.
2. **4**: chiamata della funzione di libreria read.
3. **5**: caricamento del codice della system call in un registro fissato **Rx**.
4. **6**: esecuzione della **TRAP**: passaggio in *kernel mode* e salto al codice del *dispatcher*.
5. **7-8**: selezione della system call secondo il codice in Rx e invocazione del gestore appropriato.
6. **9**: ritorno alla funzione di libreria: ripristino della *user mode* e caricamento del program counter (**PC**).
7. **10-11**: ritorno al codice utente (nel modo usuale) e incremento dello stack pointer (**SP**) per rimuovere i parametri della chiamata a read.



Attraverso una tabella in memoria, e l'indirizzo x della tabella viene passato come parametro in un registro.



Identification / Authentication / Authorization / Accounting
L'obiettivo è **consentire l'accesso al sistema e alle sue risorse solo agli utenti che ne hanno diritto**.
I passi sono quattro:
1. **Identification**.
2. **Authentication**: si basa su qualcosa che l'utente sa (something you know), possiede (something you own) o è (something you are).
3. **Authorization**: permette l'accesso alla risorsa (resource).
4. **Accountability**.

User Authentication / Authorization
Il file /etc/passwd ha tipicamente permessi di file system che gli permettono di essere **leggibile da tutti gli utenti del sistema** (world-readable), anche se può essere modificato solo dal superutente o utilizzando alcuni comandi privilegiati a scopo speciale.
Ogni riga ha 7 campi (esempio: nish:x:501:501:Nishant Rastogi:/home/nish:/bin/bash):
1. **Username** (login name);
2. **Password** (la «x» indica che è nel file /etc/shadow);
3. **UID** (User Identifier);
4. **GID** (Group Identifier);
5. **General Electric Comprehensive Operating Supervisor** (nome completo, edificio, stanza, telefono d'ufficio, telefono di casa, indirizzo email, ecc.);
6. **Home directory;**
7. **Shell** di avvio a ogni nuova connessione;

L'obiettivo è consentire l'accesso al sistema solo agli utenti che ne hanno diritto.

Accounting
L'obiettivo è **registrare gli accessi al sistema e poterli rendicontare** (se necessario).
- **Windows**: Security Log.
- **Linux**: /var/log/auth.log.
- **File di rendicontazione**: /var/log/ utmp, btmp, wtmp.

File Authorization
L'obiettivo è consentire l'accesso al sistema e alle sue risorse solo agli utenti che ne hanno diritto.
In Unix i permessi (ad es. drwxr-xr-x) si leggono così:
- **tipo di file**:
  - d: directory;
  - -: regular file;
  - l: symbolic link.
- **classi di permessi**: user, group, others.

- **flag di permesso**:
  - r: readable;
  - w: writable;
  - x: executable;
  - -: disabled.

Anche in Windows i permessi si impostano per gruppo o utente dalla scheda Security delle proprietà (Allow / Deny).

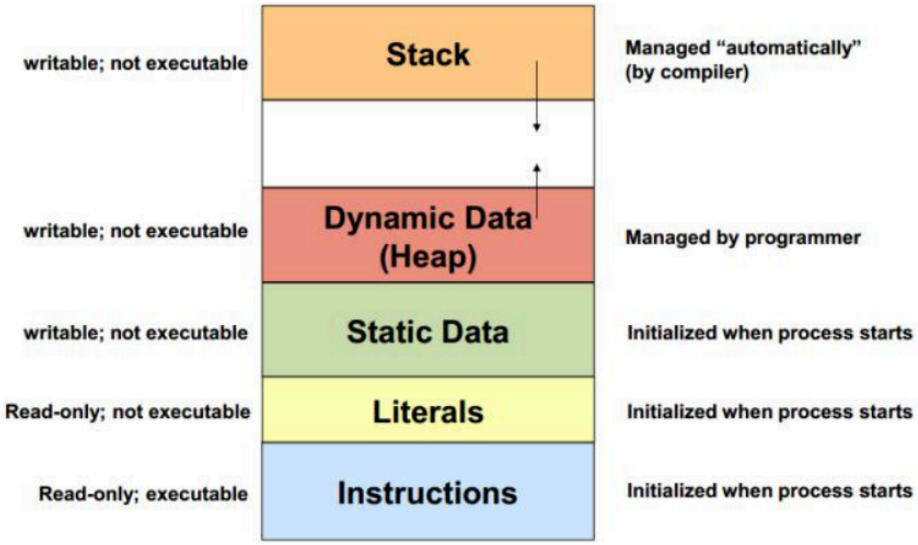
I Processi

Un **processo** è un programma in esecuzione:
- indipendente dagli altri programmi;
- che non deve interferire con gli altri programmi;
- che trae giovamento dalla gestione ottimizzata delle risorse;
- che deve sottostare all'allocazione e alla condivisione ordinata delle risorse rispetto a spazio e tempo.

Differenze tra processo e programma:
- **Processo** = esecuzione delle istruzioni —> entità attive. Comprende:
  - contatore di programma;
  - valori delle variabili;
  - risorse in uso.
- **Programma** = lista di istruzioni —> entità passiva.

Due processi associati allo stesso programma sono due differenti **istanze** di esecuzione dello stesso codice.

La **memoria occupata** dal processo è composta da:
- **Instructions**: codice del programma (puntato dal *Program Counter*, **PC**);
- **Literals**: parametri del programma;
- **Static data**: variabili globali in memoria centrale;
- **Heap** (mucchio): dati dinamici;
- **Stack** (pila): funzioni (metodi), variabili locali, indirizzi di rientro.

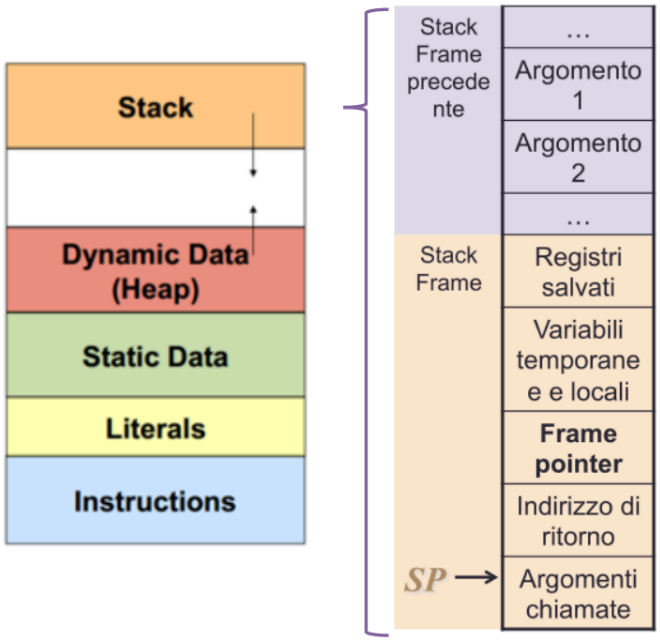


Lo **stack** è una pila di dati che tipicamente cresce verso il basso:
- I record di attivazione delle funzioni (stack frame) vengono inseriti (pusher) e rimossi (popped) dallo stack.
- Lo task frame contiene dati relativi ad una funzione:
  - parametri;

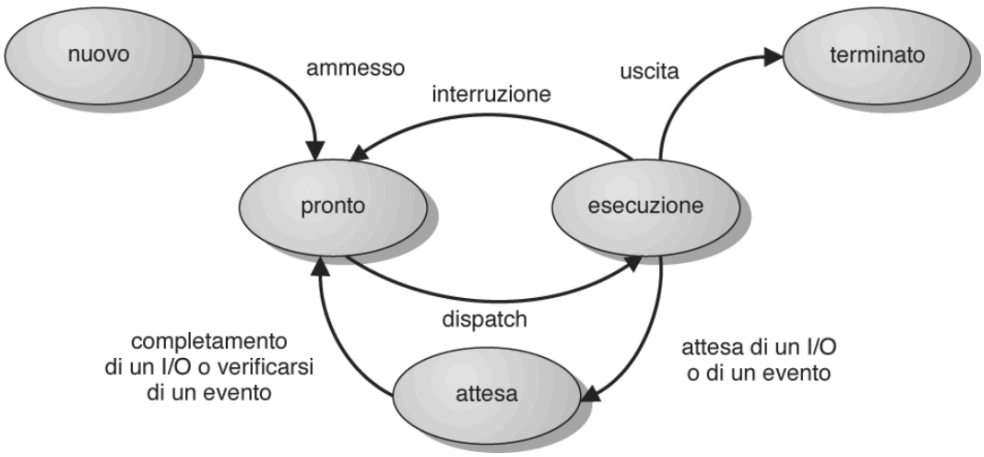
  - variabili locali;
  - dati necessari per ripristinare il frame precedente.
- Viene puntato dal **Frame Pointer** (**FP**).

Lo **heap** è una pila di dati che tipicamente cresce verso l'alto:
- il programmatore si occupa di come utilizzare l'heap (con il SO).
- Si deve occupare di allocare (malloc C) e deallocare l memoria.

Se stack e heap **collidono** si verificano **errori**: esistono chiamate apposta per ridimensionarli.



I possibili **stati di uso del processore** da parte del processo sono:
- **Nuovo** (*new*): il processo è stato creato;
- **In esecuzione** (*running*): le istruzioni vengono eseguite;
- **In attesa** (*waiting*): il processo sta aspettando il verificarsi di qualche evento;
- **Pronto all'esecuzione** (*ready*): il processo è in attesa di essere assegnato a un processore;
- **Terminato** (*terminated*): il processo ha terminato l'esecuzione.



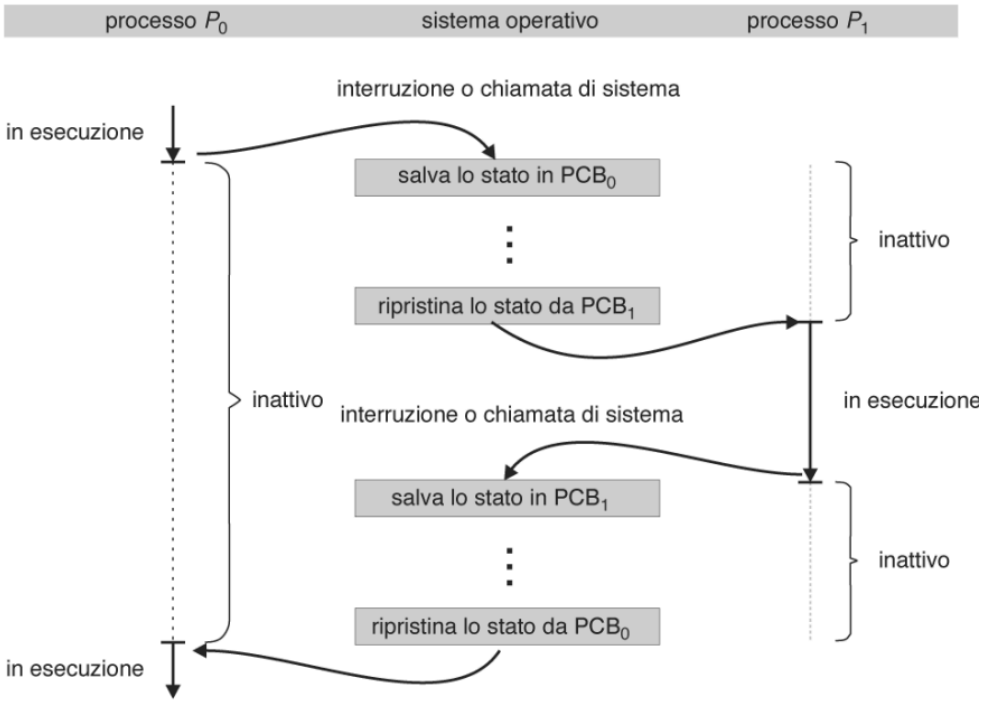
PCB

Process Control Block (**PCB**): Struttura dati del **kernel** che mantiene le informazioni sul processo

Context Switch

Il **context switch** è il passaggio della CPU da un processo a un altro. Consiste in due operazioni:
- la **sospensione** del processo in esecuzione;
- il **caricamento** del nuovo processo.

Inoltre:
- il tempo. per il cambio di contesto è puro **tempo di gestione del sistema**: non vengono compiute operazioni utili per la computazione di processi;
- i tempi (10ms) per i cambi di contesto dipendono dal **supporto hardware** (as esempio registri disponibili e/o registri dedicati).



Unix SysCall: fork(), wait(), exit()

Un **processo in esecuzione** può creare numerosi sottoprocessi usando un'apposita chiamata di sistema (create_process):
- **Processo generante** = **processo padre**;
- **Processo generato** = **processo figlio**.

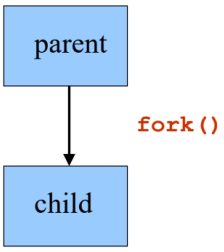
Ogni processo ha un identificatore univoco **PID** (intero).

In unix si usa la chiamata **fork()**, che crea un nuovo processo:
- si forna un albero di processi;
- inizialmente il figlio è la copia del padre;
- la forck() ritorna un valore:
  - 0: figlio;
  - != 0: padre.
- **exec()** sovrascrive lo spazio degli indirizzi del chiamante caricando un nuovo programma.

Dopo la creazione del figlio, l'esecuzione del processo padre:
- continua in modo concorrente, oppure;
- attende la terminazione del figlio —> **wait()**;
- la **chiamata fork()** crea un nuovo processo. Il valore di ritorno vale:
  - 0 per il figlio;
  - diverso da 0 per il padre.

La **fork** crea una copia dello spazio degli indirizzi del genitore. Entrambi, padre e figlio, continuano l'esecuzione della chiamata successiva alla fork.
La chiamata **wait(NULL)** blocca il chiamante fino alla terminazione di un figlio (il primo che termina).
waitpid(…, int pid, …) permette di attendere la terminazione di uno specifico figlio.

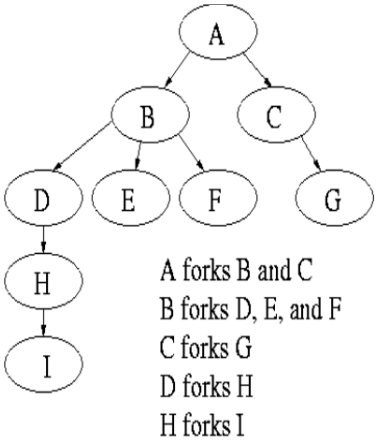
Creazione di nuovi processi



Il meccanismo alla base della creazione di nuovi processi è il seguente:
- un processo esegue **fork** per creare un processo figlio;
- padre e figli vengono eseguiti in modo concorrente;
- il processo figlio è un **duplicato** del processo padre.

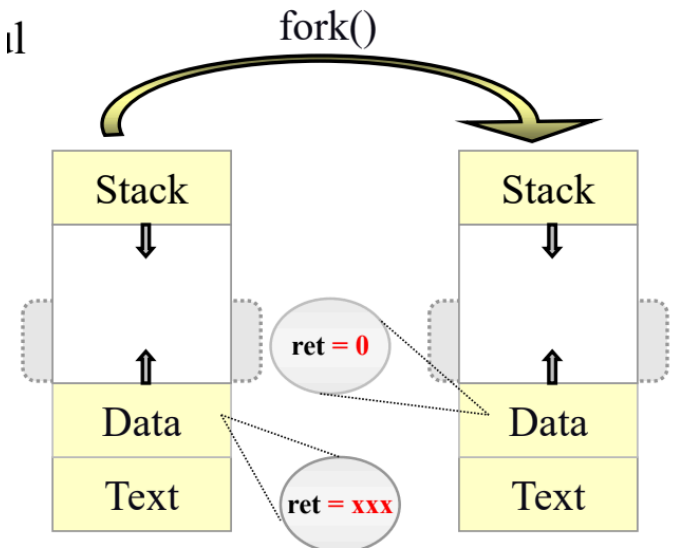
Creazione di processi

Dopo una fork, sia il padre sia il figlio continuano a essere seguiti, e ciascuno può a sua volta creare altri processi con fork:
- Ne risulta un **albero di processi**: la radice dell'albero è un processo speciale creato dal sistema operativo durante l'avio.
- Un processo può scegliere di attendere la terminazione dei figli. Ad esempio, se C esegue la chiamata di sistema wait(), resta bloccato finche G non termina.



Fork()

Il processo corrente si divide in 2 processi: **padre** e **figlio**. La fork():
- ritorna -1 se on ha successo;
- ritorna 0 nel figlio;
- ritorna l'identificatore del figlio nel padre.



Il processo figlio eredita dal padre:
- una copia identica della memoria;
- i registri della CPU;
- tutti i file che sono stati aperti dal padre.

L'esecuzione prosegue in modo **concorrente** con l'istruzione successiva alla chiamata di sistema fork.
Il **contesto di esecuzione** (**PCB**) del processo figlio è una **copia del contesto del padre** al momento della chiamata.

Terminazione del processo

La **terminazione del processo** si divide in:
- normale;
- anomala.

La **terminazione normale** avviene dopo l'ultima istruzione, tramite la chiamata **exit**:
- il figlio può restituire un valore di stato (di solito un intero) al padre, ad esempio tramite la chiamata **wait()**;
- le risorse del processo sono deallocate dal sistema operativo:
  - file aperti;
  - memoria fisica e virtuale;
  - aree di memoria per I/O.

Nella **terminazione in caso di anomalia**, il padre può terminare l'esecuzione di uno dei suoi figli per varie ragioni:
- eccessivo uso di una risorsa;
- compito non più necessario.

Terminazione a cascata:
- Se il padre sta terminando, alcuni sistemi operativi non permettono a un processo figlio odi proseguire.
- In unix la terminazione a cascata non esiste: i processi figli vengono adottato dal processo **init**.

La mancata terminazione tramite la chiamata **wait()** porta a un processo "**zombie**": un processo la cui elaborazione è terminata, ma a cui il padre non ha dato il consenso per la terminazione.

System calls: exit(), terminazione ordinata

Per terminare l'esecuzione, un figlio può chiamare **exit(status)**. Questa chiamata di sistema:
- salva il risultato, cioè l'argomento di **exit**;
- esegue tutte le funzioni specificate con **atexit(fun)** e **on_exit(fun)**;
- scarica gli stream con **fflush()**;
- chiude tutti i file e le connessioni aperte (non quelli condivisi con altri processi);
- chiama **_exit(status)**.

System calls: _exit(), fine dell'esecuzione

Per finire l'esecuzione, un figlio può chiamare **_exit(status)**. Questa chiamata di sistema:
- salva il risultato, cioè l'argomento di **exit**;
- dealloca la memoria;
- se il processo ha figli in esecuzione, li assegna a **init**;
- controlla se il padre è vivo:
  - se il padre è vivo, conserva il valore del risultato finché il padre non lo richiede (con **wait**). In questo caso il processo figlio non muore davvero, ma entra in uno stato **zombie**/**defunct**;
  - se il padre non è vivo, il figlio termina (muore).

System calls: wait(), attesa della fine di un figlio

Il padre può voler attendere la fine dei figli (ad esempio una shell che aspetta il completamento delle operazioni):

- **Attesa di un figlio qualsiasi**: **wait()**
  - si blocca finché un figlio termina;
  - ritorna il processo ID del processo figlio;
  - oppure ritorna -1 se non esistono figli (cioè sono già terminati).
- **Attesa di un figlio specifico**: **watpid()**
  - si blocca finché termina il figlio con un particolare process ID.

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

Thread

Un processo ha due caratteristiche:

- **Scheduling/esecuzione**: segue un percorso di esecuzione che può essere alternato (*interleaved*) con altri processi. L'unità di *dispatching* è detta **thread** o *lightweight process* (processo leggero).
- **Resource ownership** (possesso delle risorse): include uno spazio di indirizzamento virtuale che contiene l'immagine del processo. L'unità del possesso delle risorse è detta **processo** o **task**.

Queste due caratteristiche sono trattate in modo **indipendente** dal sistema operativo.

Ogni processo tradizionale (mono-thread) esegue **un'attività alla volta**, quindi molte attività necessitano di **molti processi**: **fork()**.
Ogni fork() implica risorse aggiuntive:

- **Risorse di memoria duplicate**: area istruzioni, variabili globali, ecc;
- **Scheduling addizionale**: creazione del nuovo processo e delle strutture (PCB);
- **Context switch addizionale**: aumento del consumo di CPU senza elaborazioni fatturali.

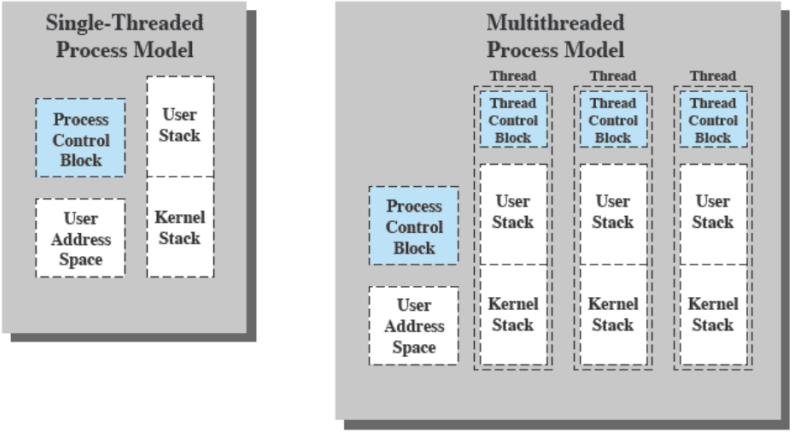
Per evitare questi costi si punta alla condivisione delle risorse (**spazio**: memoria e **tempo**: elaborazione) fra **attività analoghe**.

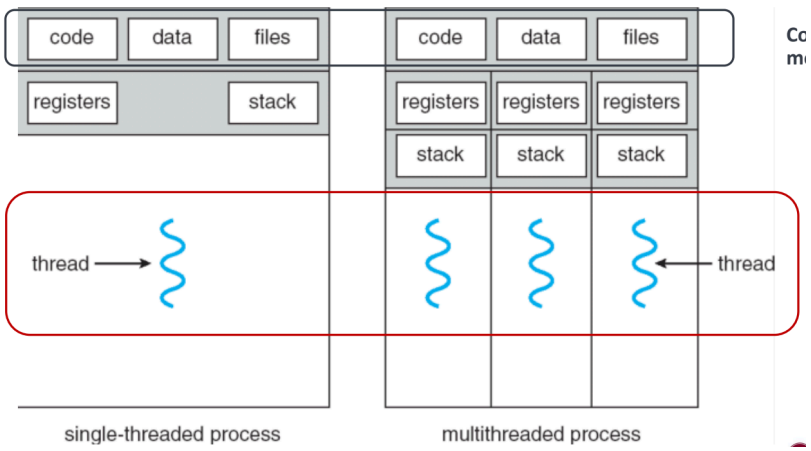
Un **thread** è un flusso di esecuzione (filo logico) indipendente all'interno dello stesso processo. E' anche detto lightweight process (processo leggero), cioè con un contesto alleggerito rispetto a un processo:

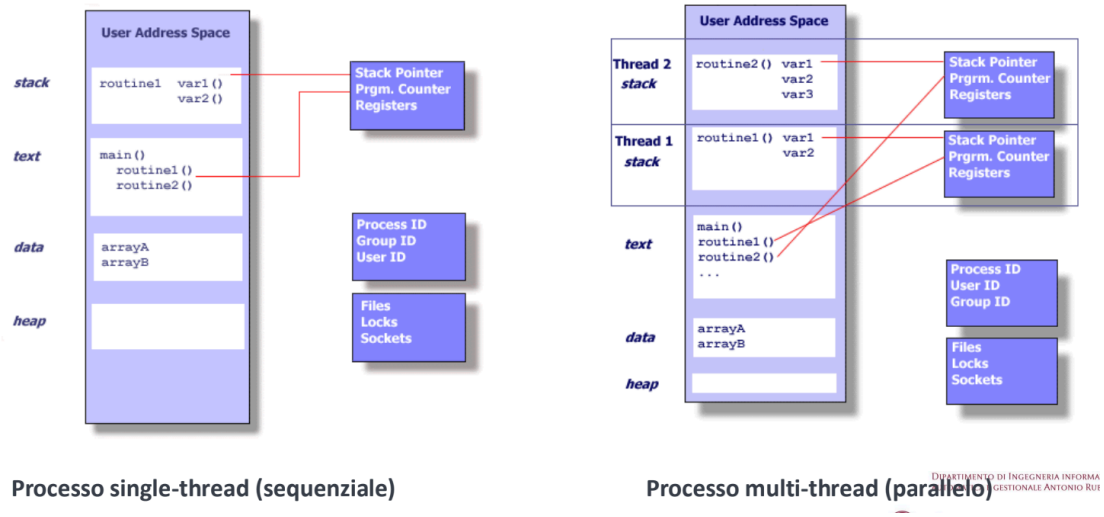
- **Spazio di indirizzamento**: condiviso con gli altri thread del processo;
- **Scheduling**: tramite la struttura di rappresentazione Thread Control Block (TCB), che punta al PCB del processo contenitore.

L'obiettivo è la condivisione/riuso delle risorse.

In altre parole, il **multithreading** è la capacità di un sistema operativo di supportare più percorsi di esecuzione concorrenti all'interno di un singolo processo.







Processo single-thread (sequenziale)

Processo multi-thread (parallelo)

I **thread** hanno stati di esecuzione e possono sincronizzarsi tra loro, in modo simile ai processi. Di queste due caratteristiche della funzionalità dei thread si parla separatamente:
- **stati**;
- **sincronizzazione**.

Gli **stati** associati ai thread sono:
- **running** (in esecuzione);
- **ready** (pronto);
- **blocked** (bloccato);

Per cambiare lo stato di un thread si hanno le seguenti operazioni:
- **Spawn** (di un altro thread): creazione di un nuovo thread;
- **Block**: blocco del thread;
- **Unblock**: sblocco dei thread;
- **Finish** (del thread): al termine si deallocano il conteso dei registri e gli stack.

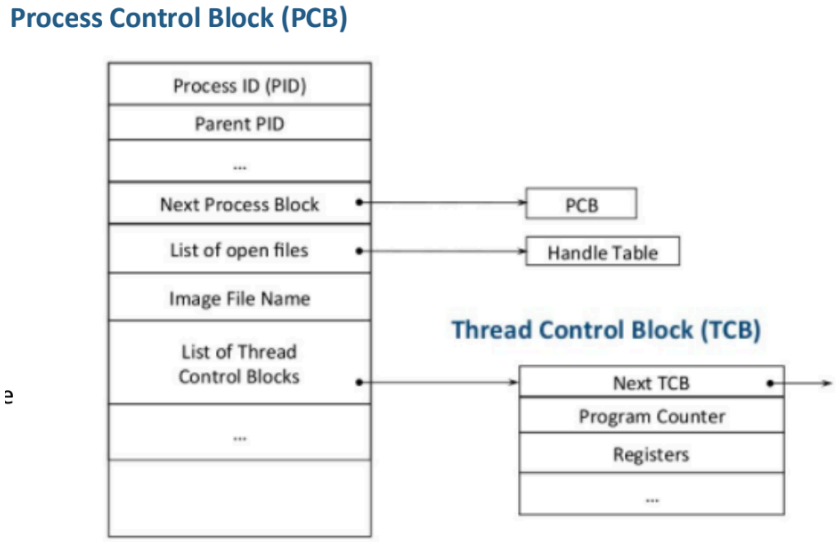
<u>PCB e TCB</u>

Il **Process Control Block** (**PCB**) contiene le informazioni del processo, tra cui:
- Process ID (PID);
- Parent PID;
- Next Process Block (puntatore al PCB successivo);
- List of open files (puntatore alla Handle Table);
- Image File Name;
- List of Thread Control Blocks (puntatore ai TCB dei thread del processo).

Al livello del processo si trovano quindi:
- lo **spazio di memoria**;
- le **risorse private** (con le corrispondenti tabelle dei descrittori).

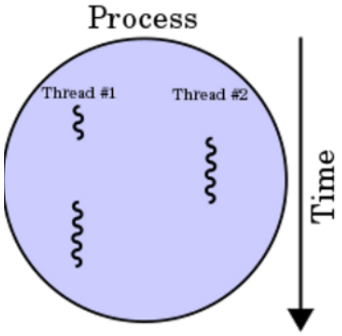
Il **Thread Control Block** (**TCB**) è una struttura dati del kernel che mantiene le informazioni sul thread. Esso contiene:
- Next TCB (puntatore al TCB successivo);
- Program Counter;
- Registers.

Al livello del thread si trovano invece:
- lo **stato della computazione** (registri, stack, PC, …);
- gli **attributi** (schedulazione, priorità);
- il **descrittore di thread** (tid, priorità, segnali pendenti, …);
- la **memoria privata**: Thread Specific Data (TSD).



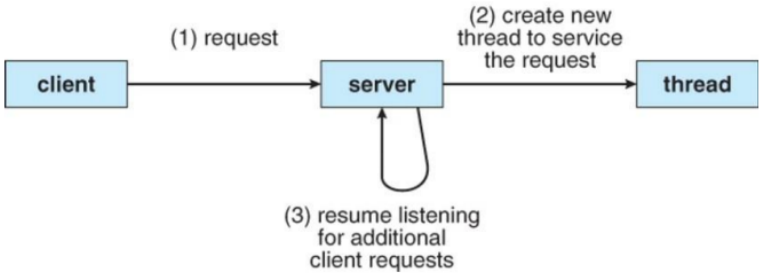
Thread concorrenti e paralleli

Concurrent Thread (thread concorrenti)
Un esempio è **MS Word**, cha ha più thread che eseguono diversi compiti in parallelo, tramite thread:
- prendere l'input da tastiera;
- controllo ortografico;
- conteggio dell'ortografia;
- …



Parallel Thread (thread paralleli)
Un esempio è il **web server,** un sistema multithread in cui ogni richiesta è gestita da un thread separato. Il funzionamento è il seguente:
1. il client invia una richiesta (request);
2. il server crea un nuovo thread per servire la richiesta;

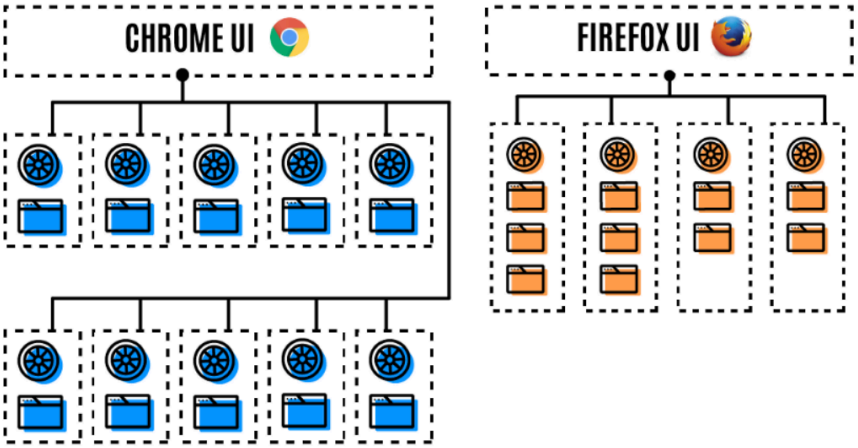
3. il serve riprende l'ascolto di ulteriori richieste dei client.



Thread in processi utente

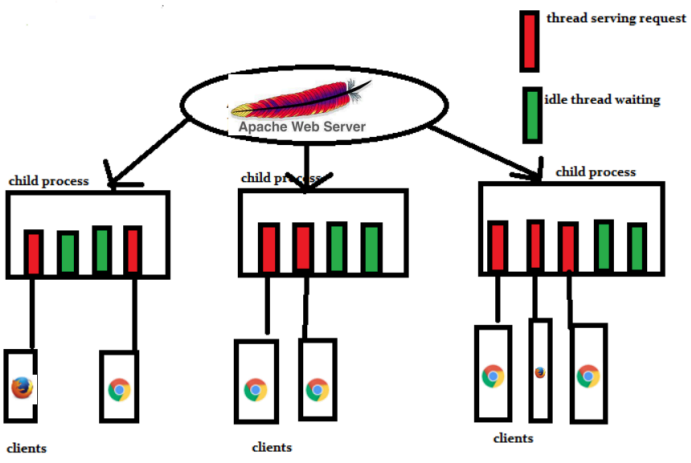
Chrome:
- ad ogni tab corrisponde un processo separato;
- all'interno di ogni processo sono arrivati thread concorrenti (trasporto dati, rendering, etc.);
- si ottengono alte performance, ma con un consumo di memoria e batterie.

Firefox:
- i primi 4 tab sono implementati in processi diversi;
- i tab successivi sono implementati tramite thread paralleli;
- le prestazioni sono minori, ma si ha un risparmio di risorse.



Thread in processi server

Il web server di Apache (httpd: HTTP Daemon) crea un certo numero di processi figli, ed ognuno di essi viene creato un certo numero di thread paralleli.

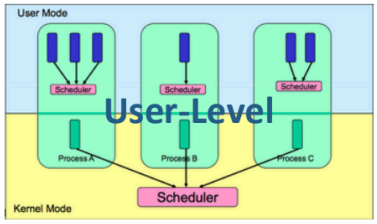
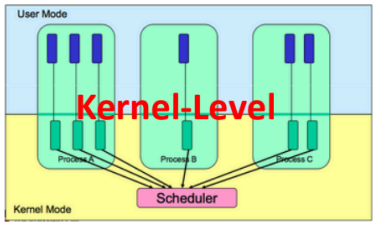


Modello di thread: User vs Kernel Level

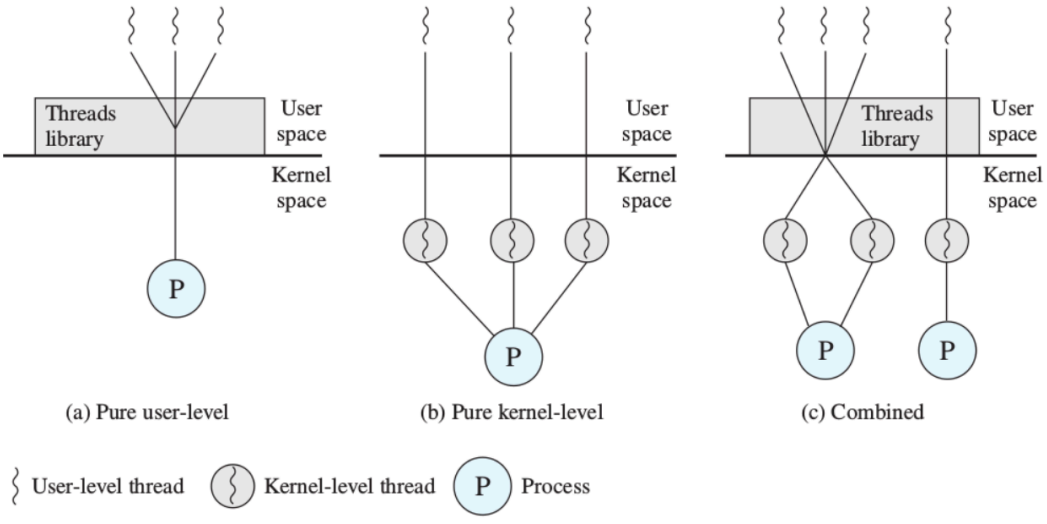
User-Level Thread (ULT)
Sono thread all'interno del **processo utente**, gestiti da una libreria specifica (da un supporto run-time):
- sono gestiti senza l'aiuto del kernel;
- lo switch non richiede chiamate al kernel.

Kernel-Level Thread (KLT)
La gestione dei thread è affidata al **kernel** tramite chiamate di sistema:
- gestione integrata di processi e thread da parte del kernel;
- lo switch è provocato da chiamate al kernel.

Sono chiamati anche:
- kernel-supported threads;
- lightweight processes.

Ambito	Vantaggi	Svantaggi
	• Lo switch non coinvolge il kernel, e quindi non ci sono cambiamenti della modalità di esecuzione • Maggiore libertà nella scelta dell'algoritmo di scheduling che può anche essere personalizzato • Poiché le chiamate possono essere raccolte in una libreria, c'è maggiore portabilità tra SO	• Una chiamata al kernel può bloccare tutti i thread di un processo, indipendentemente dal fatto che in realtà solo uno dei suoi thread ha causato la chiamata bloccante • In sistemi a multiprocessore simmetrico (SMP) due processori non possono essere associati a due thread del medesimo processo
	• Il kernel può eseguire più thread dello stesso processo anche su più processori • Il kernel stesso può essere scritto multithread	• Lo switch coinvolge chiamate al kernel e questo comporta un costo • L'algoritmo di scheduling è meno facilmente personalizzabile • Meno portabile

DIPARTIMENTO DI INGEGNERIA INFORMATICA AUTOMATICA E GESTIONALE ANTONIO RUBERTI



Modello multi-thread

I thread a livello utente vengono messi in relazione, per permettere l'accesso alle risorse, con i thread a livello kernel.
Esistono diversi tipi di relazione tra thread a livello utente e thread a livello kernel:
- Molti-a-uno;
- Uno-a-uno;

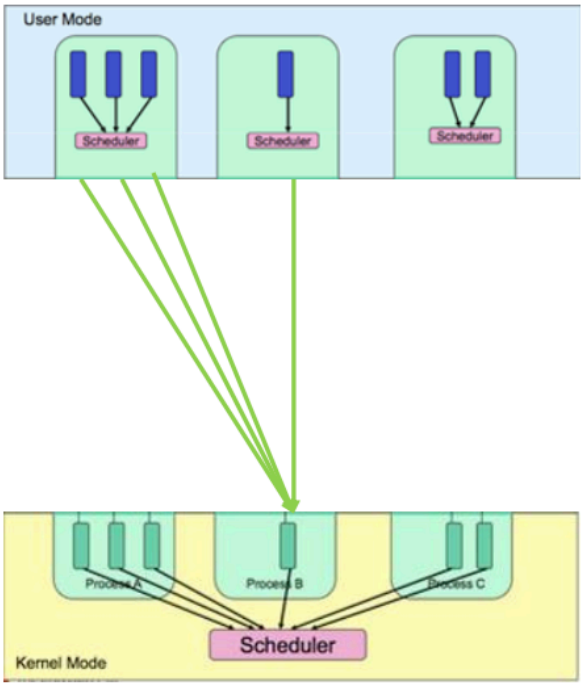
- Molti-a-molti;
- A due livelli.

Molti-a-uno
Riunisce molti thread di livello utente in un unico kernel thread.

Vantaggi:
- Gestione dei thread efficace.

Svantaggi:
- Intero processo bloccato se un thread invoca una chiamata di sistema bloccante;
- Un solo thread può accedere al kernel (impossibile eseguire un thread in parallelo).

Esempi:
- Solars Green Threads;
- GNU Portable Threads.

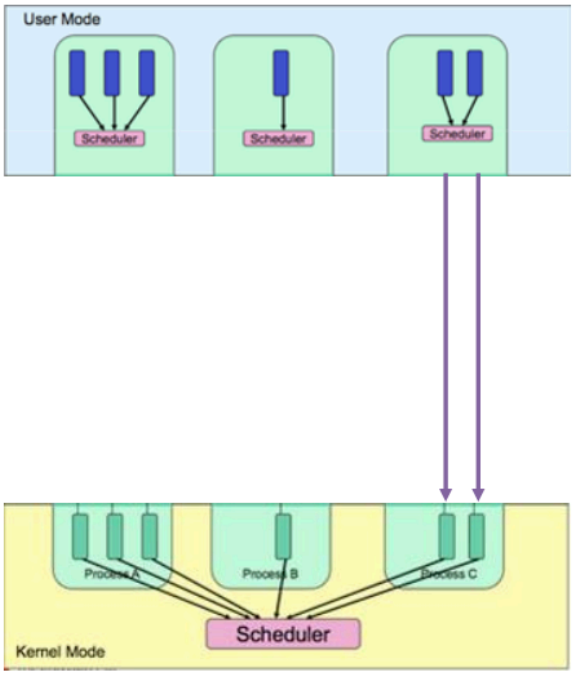


Uno-a-uno
Mappa ciascun thread utente in un kernel thread.

Vantaggi:
- Maggiore concorrenza (possibili thread in parallelo e le chiamate bloccanti non bloccano tutti i thread).

Svantaggi:
- La creazione di molti thread a livello kernel compromette le prestazioni dell'applicazione (es. limite del numero di thread a livello kernel).

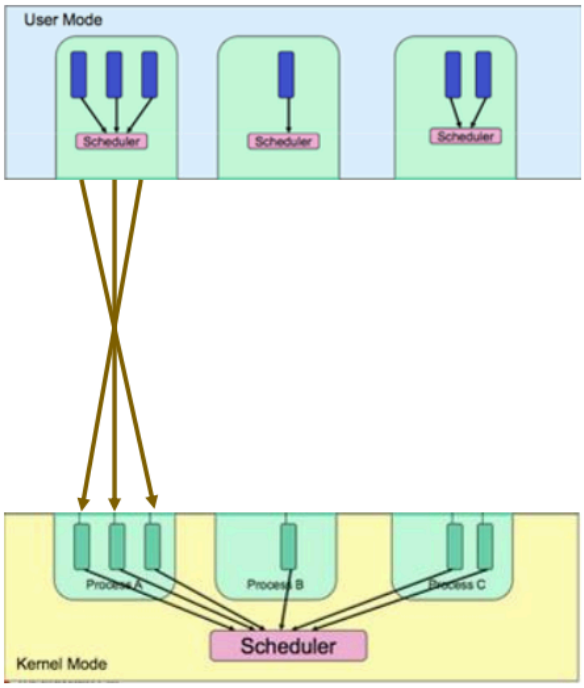
Esempi:
- Windows NT/XP/2000;
- Linux;
- Solaris 9.



Molti-a-molti
Permette di aggregare molti thread a livello utente verso un numero più piccolo o equivalente di kernel thread.

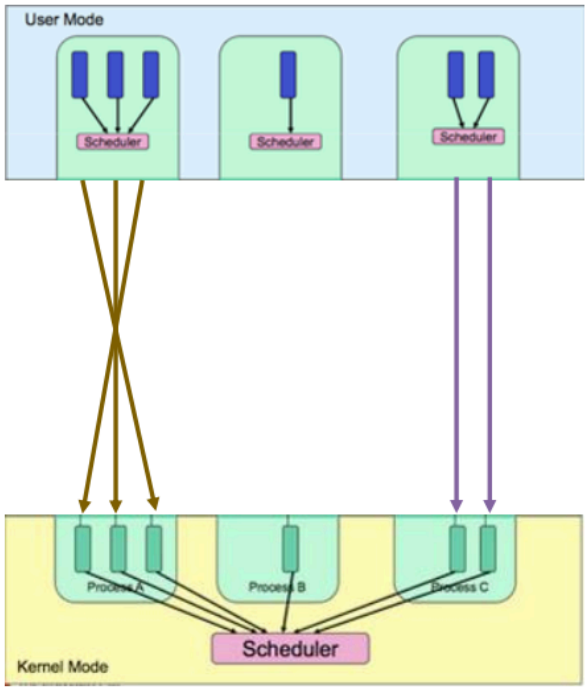
Vantaggi:
- Risolve le limitazioni dei modelli precedenti (il numero massimo di processi a livello kernel può essere personalizzato in base all'architettura; n_max maggiore per multicore).

Esempi:
- Solaris, versioni precedenti alla 9;
- Windows NT/2000 (pacchetto ThreadFiber).



A due livelli
Simile al modello molti-a-molti, eccetto che permette anche di associare un thread di livello utente ad un kernel thread.

Esempi:
- IRIX;
- HP-UX;
- Tru64 UNIX.



Librerie per i thread

La gestione dei thread è effettuata attraverso specifiche librerie.

Libreria collocata a livello utente (POSIX Pthreads)
- codice e strutture dati per la libreria risiedono nello spazio utente;
- invocare una funzione della libreria non significa invocare una chiamata di sistema.

Libreria collocata a livello kernel (POSIX Pthreads, Win32 thread)
- codice e strutture dati per la libreria risiedono nello spazio del kernel;
- invocare una funzione della libreria significa invocare una chiamata di sistema.

Pthread - User Level Thread

Pthread è lo standard **POSIX** (IEEE 1003.1c), un'API per la creazione e la sincronizzazione dei thread:
- fornisce una specifica per il comportamento della libreria dei thread;
- i progettisti di sistemi operativi possono implementare la specifica nel modo che desiderano;
- è frequente nei sistemi operativi UNIX (Solaris, Linux, Mac OS X).

I Pthreads sono tipi del linguaggio di programmazione C:

```
#include <pthread.h>
```

`pthread_t`: specifica l'identificatore di un thread
`pthread_attr`: specifica gli attributi di un thread (e.g. proprietà per lo scheduling)
`pthread_attr_init(&pthread_attr)`: inizzializza gli attributi di un thread ai valori di default
`pthread_create(pthread_t *tid, pthread_attr *attr, void*(), void*argv)`: crea un user thread al quale associare gli attributi inizializzati, la funzione da eseguire e gli argomenti
`pthread_join (pthread_t *tid, <value>)`: mette in pausa il thread corrente fino alla terminazione del thread tid. Se il secondo parametro non è nullo viene usato per restituire il valore di ritorno del thread

System call fork() ed exec()

fork()
Solitamente vi sono varie versioni della funzione fork(), a seconda che si desideri:
- duplicare solo il thread che lo invoca;
- duplicare tutti i thread del processo ospitante

clone() in Linux
In Linux esiste la funzione clone(): una nuova chiamata di sistema versatile che può essere utilizzata per creare un nuovo thread di esecuzione. A seconda delle operazioni passate, il nuovo thread di esecuzione può aderire alla semantica di:
- un processo UNIX;
- un thread POSIX;
- una via di mezzo;
- qualcosa di completamente diverso (come un contenitore diverso).

E' possibile specificare tutti i tipi di opzioni che determinano se la memoria, i descrittori di file, i vari spazi dei nomi, i gestori dei segnali e cosi via vengono **condivisi** o **copiati**.

La funzione fork() è attualmente implementata come rapper che richiama clone().

exec()
La funzione exec(), viceversa, funziona analogamente a quanto visto prima dell'introduzione dei thread: rimpiazza l'intero processo, quindi anche tutti i thread.

Thread in Linux

Linux li definisce come **task** piuttosto che thread.
La creazione di un thread avviene attraverso la chiamata di sistema clone(), che permette di stabilire il "grado di condivisione" tramite **flag** passati come parametri:

Flag	Significato
CLONE_FS	Condivisione delle informazioni sul file system
CLONE_VM	Condivisione dello stesso spazio di memoria
CLONE_SIGHAND	Condivisione dei gestori dei segnali
CLONE_FILES	Condivisione dei file aperti

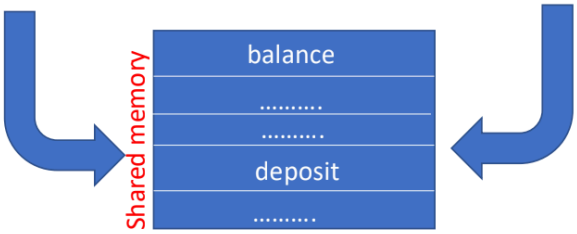
Se nessun flag è impostato, non c'è alcuna condivisione e l'effetto di **clone()** è simile a quello della **fork()** (funzione "wrapper").

Thread safety

Un codice è **thread-safe** quando più thread possono eseguirlo simultaneamente senza interazioni non volute, cioè:
- senza *clobbering* (sovrascrittura) dei dati condivisi;
- senza creare *race condition*.

Esempio:
Un'applicazione crea diversi thread, ognuno dei quali chiama la stessa routine di libreria:
- La routine di libreria accede/modifica una struttura globale o una locazione in memoria.
- Dato che ogni thread chiama questa routine, è possibile che provino a modificare questa struttura/locazione nello stesso momento.
- Se la routine non impiega un qualche **meccanismo di sincronizzazione** per prevenire la corruzione dei dati, allora non è thread-safe.

Thread 1	Thread 2	Balance
Read balance: $1000		$1000
	Read balance: $1000	$1000
	Deposit $200	$1000
Deposit $200		$1000
Update balance $1000+$200		$1200
	Update balance $1000+$200	$1200



Concorrenza: mutua esclusione e sincronizzazione

In un sistema multiprogrammato, multitasking, multithread si definiscono i seguenti concetti:
- **Race Condition** (corsa critica): situazione in cui il risultato dell'elaborazione su una risorsa condivisa da più processi, di cui almeno uno vi scrive, dipende da come essi si alternano (run-time: non predicibile, in funzione della schedulazione, degli eventi esterni, degli interrupt, etc.).
- **Mutual Exclusion** (mutua esclusione): proibire che più di un processo acceda contemporaneamente alla risorsa condivisa.
- **Critical Region** (sezione critica): parte del codice di un processo in cui avviene l'accesso alla risorsa condivisa.

Per impedire l'ingenerasi di race condition devono essere rispettati i seguenti requisiti:
1. **Mutual Exclusion**: un solo processo può essere in una regione critica;
2. **Velocità relative**: le assunzioni su velocità e numero id processi sono ininfluenti;
3. **Progresso**: nessun processo la cui elaborazione è al di fuori della sezione critica può fermare altri processi;
4. **Bounded Waiting**: nessun processo deve aspettare indefinitamente per entrare nella sua regione critica.

Termini chiave pt.1

Race condition: situazione in cui più thread o processi leggono e scrivono un dato condiviso e il risultato finale dipende dalla temporizzazione relativa della loro esecuzione. Il "perdente" della corsa è il processo che aggiorna per ultimo e che determinerà il valore finale della variabile.

Mutual Exclusion (mutua esclusione): requisito per cui, quando un processo si trova in una sezione critica che accede a risorse condivise, nessun altro processo può trovarsi in una sezione ricca che accede a una qualsiasi di quelle risorse condivise.

Critical Section (sezione critica): sezione di codice all'interno di un processo che richiede l'accesso a risorse condivise e che non deve essere eseguita mentre un altro processo si trova in una sezione di codice corrispondente.

L'output di un processo deve essere indipendente dalla velocità di esecuzione degli altri processi concorrenti.

Atomic operation (operazione atomica): funzione o azione implementata come sequenza di una o più istruzioni che appare indivisibile, cioè nessun altro processo può vedere uno stato intermedio o interrompere l'operazione. La sequenza di istruzioni è garantita eseguirsi come gruppo, oppure

non eseguirsi affatto, senza alcun effetto visibile sullo stato del sistema. L'atomicità garantisce l'isolamento dai processi concorrenti.

Critical Region

In un sistema multiprogrammato, multitasking, multithread si definiscono i seguenti concetti:
- **Deadlock** (stallo): situazione in cui due o più processi non possono procedere con la prossima istruzione, perché ciascuno attende l'altro.
- **Livelock** (stallo attivo): situazione in cui due o più processi cambiano continuamente il proprio stato, l'uno in risposta all'altro senza fare alcunché di utile.
- **Starvation** (inedia): un processo, pur essendo ready, non viene mai scelto dallo scheduler.

Termini chiave pt.2

Deadlock (stallo): situazioni in cui due o più processi non sono in grado di procedere perché ciascuno è in attesa che uno degli altri faccia qualcosa.

Livelock (stallo attivo): situazione in cui due o più processi cambiano continuamente il proprio stato in risposta ai cambiamenti degli altri processi, senza svolgere alcun lavoro utile.

Starvation (inedia): situazione in cui un processo pronto all'esecuzione (runnable) viene ignorato indefinitamente dallo scheduler; pur essendo in grado di procedere, non viene mai scelto.

Operating System Concerns: interazione tra processi

1. **Processi che non si conoscono tra loro**:
   - Relazione: competizione.
   - Influenza di un processo sull'altro:
     - il risultato di un processo è indipendente dall'azione degli altri;
     - la temporizzazione del processo può essere influenzata.
   - Potenziali problemi di controllo:
     - mutua esclusione;
     - deadlock (risorsa riutilizzabile);
     - starvation.
   - Soluzioni:
     - Mutual Exclusion (senza deadlock, livelock, starvation);
     - Turn Variable: solo per 2 processi concorrenti;
     - Hardware Support: interrupt disabling, TSL (XCHG).
2. **Processi che si conoscono indirettamente**:
   - Relazione: cooperazione tramite condivisione.
   - Influenza di un processo sull'altro:
     - il risultato di un processo può dipendere dalle informazioni ottenute dagli altri;
     - la temporizzazione del processo può essere influenzata.
   - Potenziali problemi di controllo:
     - mutua esclusione;
     - deadlock (risorsa utilizzabile);
     - starvation;
     - coerenza dei dati.
   - Soluzioni:
     - Mutual Exclusion (senza deadlock, livelock, starvation);
     - Sleep Wake-Up: sincronizzazione con lo scheduler.
3. **Processi che si conoscono direttamente**:
   - Relazione: cooperazione tramite comunicazione.
   - Influenza di un processo sull'altro:
     - il risultato di un processo può dipendere dalle informazioni ottenute dagli altri;
     - la temporizzazione del processo può essere influenzata.
   - Potenziali problemi di controllo:
     - deadlock (risorsa riutilizzabile);
     - starvation.
   - Soluzioni:

- Sleep Wake-UP: sincronizzazione con lo scheduler.

Principi e difficoltà

Principi
Interleaving (alternanza) o **overlapping** (sovrapposizione):
- possono essere visti come esempi di elaborazione concorrente;
- entrambi presentano gli stessi problemi.

[Monoprocessore] La velocità relativa di esecuzione dei processi non può essere prevista e dipende da:
- le attività degli altri processi;
- il modo in cui il SO gestisce gli interrupt;
- le politiche di scheduling del SO.

Difficoltà
- condivisione di risorse globali;
- difficoltà per il SO di gestire l'allocazione delle risorse in modo ottimale;
- difficoltà nell'individuare gli errori di programmazione, poiché i risultato non sono deterministici e riproducibili.

Mutua Esclusione

Il controllo della competizione **coinvolge inevitabilmente il SO**, perché è il SO che alloca le risorse. Inoltre, **i processi stessi dovranno essere in grado di esprimere in qualche modo il requisito di mutua esclusione**, as esempio bloccando (locking) una risorsa prima del suo uso. Qualsiasi soluzione richiederà un certo supporto da parte del SO, come la messa a disposizione di un meccanismo di locking.

```
PROCESS 1 */

void P1
{
   while (true) {
      /* preceding code */;
      entercritical (Ra);
      /* critical section */;
      exitcritical (Ra);
      /* following code */;
   }
}
```

```
/* PROCESS 2 */

void P2
{
   while (true) {
      /* preceding code */;
      entercritical (Ra);
      /* critical section */;
      exitcritical (Ra);
      /* following code */;
   }
}
```

. . .

```
/* PROCESS n */

void Pn
{
   while (true) {
      /* preceding code */;
      entercritical (Ra);
      /* critical section */;
      exitcritical (Ra);
      /* following code */;
   }
}
```

Lo schema illustra il meccanismo di mutua esclusione in termini astratti. Ci sono n processi da eseguire concorrentemente. Ogni processo comprende:
1. una selezione critica che opera su una risorsa Ra;
2. altro codice, precedente e successivo alla sezione critica, che non comporta accesso a Ra.

Poiché tutti i processi accedono alla stessa risorsa Ra, si desidera che **un solo processo alla volta** si trovi nella propria sezione critica. Per imporre la mutua esclusione sono previste due funzioni: **entercritical** e **exicritical**. Ciascuna funzione prende come argomento il nome della risorsa oggetto della competizione. Qualunque processo tenti di entrare nella propria sezione critica mentre un altro processo si trova nella sia, per la stessa risorsa, viene **messo in attesa**.

Requisiti:
1. La mutua esclusione deve essere **imposta** (enforced);
2. Un processo che si ferma nella propria sezione non critica deve farlo **senza interferire con gli altri processi**;
3. **Niente deadlock o starvation**;
4. A un processo non deve essere negato l'accesso a una sezione critica quando **nessun altro processo la sta usando**;
5. **Non si fanno assunzioni** sulle velocità relative dei processi o sul numero di processi;

6. Un processo rimane all'interno della propria sezione critica **solo per un tempo finito**.

Mutual Exclusion with Busy Waiting

- **Busy Waiting**: i processi fuori dalla sezione critica continuano a testare la variabile fino a che lock non ridiventi 0;
- **Check asincrono**: lo scheduler testo lo stato <<Pronto>>, che non rispecchia anche la usabilità della sezione critica.

Turn Variable (variabile di blocco): stato inerente l'esecuzione di sezioni critiche. E' una variabile condivisa che definisce lo stato di uso di una risorsa (0: risorsa libera; 1: risorsa in uso) e segnala se è in uso da parte di un processo (nella sua sezione critica).
- **Strict Alternation**: l'utilizzo di una sola variabile permette di descrivere solo 2 stato (processo A in Critical Region, processo B in Critical Region). Esiste anche l'evenienza ce nessuno sia in critical region.

Interrupt Disabling (disabilitazione delle interruzioni): il processo fa un'esplicita richiesta di disabilitazione di tutte le interruzioni, durante l'esecuzione della sezione critica, in modo da non incorrere in inconsistenza di dati sulla risorsa condivisa.
- Non funziona sui multi-core, perché la disabilitazione degli interrupt ha effetto solo su una CPU alla volta.

Special HW Instruction (istruzioni speciali implementate nell'hardware): per assicurare l'atomicità nell'esecuzione assembly dell'istruzione, questa viene implementata in hardware.
- necessaria conoscenza dell'hardware sottostate;
- ridotta probabilità del codice.

TSL (Test and Set Lock): operazione in **HW**, definita nella CPU. Mette il contenuto di [lock] nel registro ed inserisce un valore non nullo (es. PID) in [lock].

```
enter_region:
        TSL REGISTER,LOCK
        CMP REGISTER,#0
        JNE enter_region
        RET


leave_region:
        MOVE LOCK,#0
        RET
```

Atomic Operation (operazione atomica): sequenza indivisibile di comandi. Nessun altro processo può vedere uno stato intermedio della sequenza o interrompere la sequenza. Le operazioni **HW** sono ovviamente, atomiche.

Turn Variable

Turn Variable (variabile di turno): definisce il turno di uso della risorsa (indica non chi ha la precedenza ma chi si mette in coda). Annovera anche la situazione in cui nessun processo è interessato alla sezione critica. Da usarsi insieme alla variabile lock.
L'algoritmo controllerà:
1. prima se i flag di entrambi i processi A e B sono 1, per vedere se sono entrambi interessati;
2. di chi è il turno;
3. se sono entrambi interessati e il turno è del processo B, allora il processo A può entrare nella region critica per primo.

```
#define FALSE  0
#define TRUE   1
#define N      2                        /* number of processes */

int turn;                               /* whose turn is it? */
int interested[N];                      /* all values initially 0 (FALSE) */

void enter_region(int process);         /* process is 0 or 1 */
{
     int other;                         /* number of the other process */

     other = 1 – process;               /* the opposite of process */
     interested[process] = TRUE;        /* show that you are interested */
     turn = process;                    /* set flag */
     while (turn == process && interested[other] == TRUE)  /* null statement */ ;
}

void leave_region(int process)          /* process: who is leaving */
{
     interested[process] = FALSE;       /* indicate departure from critical region */
}
```

Interrupt Disabling

Vantaggi
- funziona solo su sistemi monoprocessore;
- la disabilitazione degli interrupt garantisce la mutua esclusione.

Svantaggi:
- l'efficienza dell'esecuzione potrebbe risultare sensibilmente degradata;
- questo approccio non funziona in un'architettura multiprocessore;
- può essere gestito in modo sicuro solo dal kernel (pericoloso nello user-space).

Exchange instruction

L'exchange instruction:
- scambia il contenuto di un registro con quello di una locazione di memoria;
- implementa nelle architetture Pentium e Itanium (IA64).

```
void exchange (int *register, int *memory)
{
  int temp;
  temp = *memory;
  *memory = *register;
  *register = temp;
}
```

Compare&Swap Instruction

Nella compare and exchange instruction:
- viene effettuato un confronto (compare) tra un valore in memoria e un valore di test;
- se i valori sono uguali, avviene in memoria uno scambio (swap) con il nuovo valore;
- è eseguita in modo atomico;
- disponibile nelle architetture x86, IA64, sparc, IBM.

```
int compare_and_swap(int* reg, int oldval, int newval)
{
  ATOMIC();
  int old_reg_val = *reg;
  if (old_reg_val == oldval)
     *reg = newval;
  END_ATOMIC();
  return old_reg_val;
}
```

Esempio di utilizzo:

```
/* program mutualexclusion */
const int n = /* number of processes */;
int bolt;
void P(int i)
{
  while (true) {
    while (compare_and_swap(&bolt, 0, 1) == 1)
        /* do nothing */;
    /* critical section */;
    bolt = 0;
    /* remainder */;
  }
}
void main()
{
  bolt = 0;
  parbegin (P(1), P(2), . . . ,P(n));

}
```

Hardware Support

Vantaggi:
- applicabile a un qualsiasi numero di processi, su un singolo processore o su più processori che condividono la memoria principale;
- semplice e facile da verificare;
- può essere usato per supportare più sezioni critiche; ogni sezione critica può essere definita dalla propria variabile.

Svantaggi:
- viene impiegato il busy-waiting: mentre un processo attende l'accesso a una sezione critica continua a consumare tempo di processore;
- la starvation è possibile quando un processo lascia una sezione critica e più di un processo è in attesa;
- il deadlock è possibile (ad esempio a causa della priorità dei processi);
- non è disponibile in tutte le architetture: lo sviluppatore deve essere consapevole dell'architettura.

Concorrenza: Semafori

I seguenti costrutti sono detti meccanismi comuni (di sincronizzazione) e sono usati per gestire la concorrenza tra processi e thread:
- **Semaphore** (semaforo): costrutto basato su un valore intero, usato per la segnalazione tra processi. Su un semaforo possono essere eseguite solo tre operazioni, tutte atomiche: *initialize* (inizializzazione), *decrement* (decremento) e *increment* (incremento). L'operazione di decremento può comportare il blocco di un processo, mentre l'operazione di incremento può comportare lo sblocco di un processo.
- **Binary** Semaphore (semaforo binario): semaforo che assume solo i valori 0 e 1.
- **MutEx**: simile a un semaforo binario. Una differenza chiave tra i due è che il processo che blocca il mutex (imposta il valore a zero) deve essere quello che lo sblocca (imposta il valore a 1).
- **Condition Variale** (variabile di condizione): tipo di dato usato per bloccare un processo o un thread finché una determinata condizione non è vera.

Il **semaforo** è una variabile intera, usata per mettere a dormire i processi (**sleep**) o svegliarli (**wakeup**).

Operazioni: ne sono definite 3, le sole ammesse per manipolare il semaforo:
- **init()**: abilitazione del semaforo (valore non negativo);
- **down()[semWait()]**: generalizzazione di sleep(); controlla il semaforo. Se non è zero, decremento il semaforo. Se zero, il processo va a dormire;

- **up()[semSignal()]**: generalizzazione di wakeup(); controlla il semaforo. Se non è zero, incrementa il semaforo. Se zero, il processo va a dormire.

Operazioni atomiche: garantiscono che, una volta partite le operazioni sul semaforo, nessun altro processo possa accadere finché l'operazione non sia stata completata (o bloccata).

SemWait(down): fondamentale per difendere la mutua esclusione nella scrittura di X. Senza di esso, a run-time, potrebbe succedere che il valore scritto da uno venga sovrascritto dall'altro, portando a valori non previsti la corretta computazione:
- $X = N + 1$;
- $X = 2N$.

SemSignal (up): fondamentale per uscire dalla mutua esclusione consentendo anche all'altro processo di entrarvi. Senza di esso si ha la starvation.

Esempio:

```
semaphore S = 1;
    int X = N;

    [Process 1]              [Process 2]
        int Y;                   int Z;
        semWait(S);              semWait(S);
        Y = X*2;                 Z = X+1;
        X = Y;                   X = Z;
        semSignal(S);            semSignal(S);
```

Risultato corretti (con il semaforo):
- sequenza Process 1 - Process 2: $X = 2N + 1$;
- sequenza Process 2 - Process 1: $X = 2(N + 1)$.

Sui semafori vanno tenute presenti queste tre considerazioni:
- non c'è modo di sapere, prima che un processo decrementi un semaforo, se si bloccherà o meno;
- su un sistema monoprocessore non c'è modo di sapere quale processo proseguirà immediatamente dopo un semSignal, quando due processi sono in esecuzione concorrente;
- non si sa se un altro processo è in attesa, quindi il numero di processi sbloccati può essere zero o uno.

Definizione della primitiva per semafori a interi

Si assuma che le primitiva semWait e semSignal siano atomiche:

```
struct semaphore {
     int count;
     queueType queue;
};
void semWait(semaphore s)
{
     s.count--;
     if (s.count < 0) {
          /* place this process in s.queue */;
          /* block this process */;
     }
}
void semSignal(semaphore s)
{
     s.count++;
     if (s.count <= 0) {
          /* remove a process P from s.queue */;
          /* place process P on ready list */;
     }
}
```

Definizione della primitiva per semaforo binario

Un semaforo binario può assumere solo i valori 0 e 1 e può essere definito dalle seguenti tre operazioni:

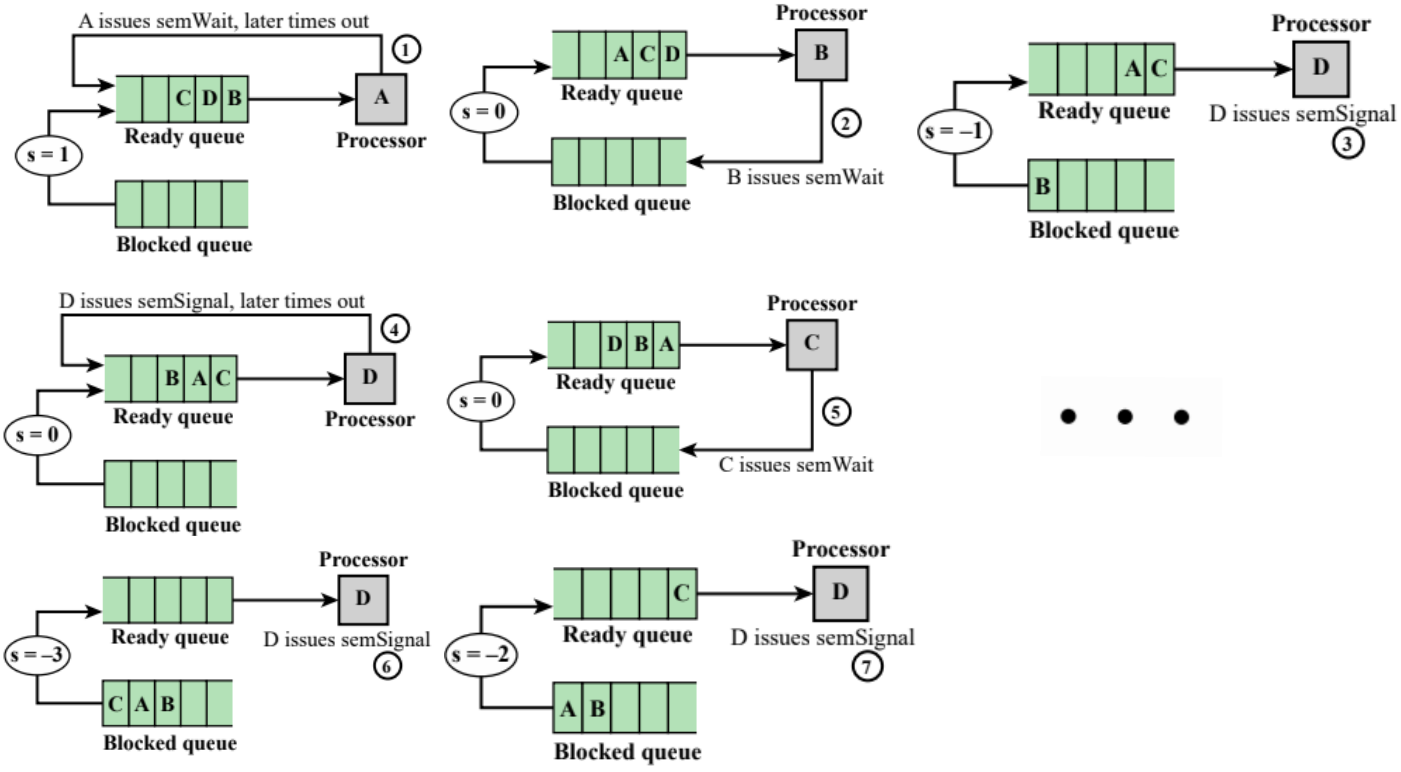
```
struct binary_semaphore {
     enum {zero, one} value;
     queueType queue;
};
void semWaitB(binary_semaphore s)
{
     if (s.value == one)
          s.value = zero;
     else {
            /* place this process in s.queue */;
            /* block this process */;
     }
}
void semSignalB(semaphore s)
{
     if (s.queue is empty())
          s.value = one;
     else {
            /* remove a process P from s.queue */;
            /* place process P on ready list */;
     }
}
```

Strong/Weak Semaphores

Una **coda** (queue) è usata per mantenere i processi in attesa sul semaforo; in base a questa caratteristica i semafori si dividono in:

- **Strong Semaphores** (semafori forti): il processo che è stato bloccato più a lungo viene rilasciato per primo dalla coda (FIFO);
- **Weak Semaphores** (semafori deboli): l'ordine in cui i processi vengono rimossi dalla coda non è specificato.

Esempio:



In questo esempio i processi A, B e C dipendono da un risultato del processo D:
1. Inizialmente (1), A è in esecuzione; B, C e D sono pronti e il conteggio del semaforo è 1, a indicare che è disponibile uno dei risultati D. Quando A eseguire una semWait sul semaforo s, il semaforo decremento a 0 e A può continuare l'esecuzione; successivamente rientra nella coda dei pronti;
2. Poi B va in esecuzione (2), prima o poi esegue una semWait e viene bloccato;
3. permettendo a D di andare in esecuzione (3). Quando D completa un nuovo risultato esegue una semSignal,
4. che permette a B di passare alla coda dei pronti(4).
5. D rientra nella coda dei pronti e C inizia l'esecuzione (5), ma viene bloccato quando esegue una semWait.
6. Analogamente, A e B vanno in esecuzione e vengono bloccati sul semaforo, permettendo a D di riprendere l'esecuzione (6).
7. Quando D ha un risultato, esegue una semSignal, che trasferisce C alla coda dei pronti (7).
8. I cicli successivi di D rilasceranno A e B dallo stato Bloccato.

MutEx

Il **MutEx** si usa quando non è necessario effettuare un conteggio degli spazi riempiti/vuoti e fundo solo 2 processi condividono la risorsa.
La **Mutual Exclusion** è realizzata tramite una variabile binaria, cioè una versione semplificata del semaforo, che può trovarsi in 2 stati:
- **Locked**;
- **Unlocked**.

Per la mutua esclusione, i semafori forti garantiscono l'assenza di **starvation**, mentre i semafori deboli non la garantiscono.
Si assumeranno **semafori forti**, perche sono più convenienti e perché è la forma di semaforo tipicamente fornita dai sistemi operativi.

```
/* program mutualexclusion */
const int n = /* number of processes  */;
semaphore s = 1;
void P(int i)
{
     while (true) {
          semWait(s);
          /* critical section   */;
          semSignal(s);
          /* remainder   */;
     }
}
void main()
{
     parbegin (P(1), P(2), . . ., P(n));
}
```

Implementazione dei semafori

E' imperativo che le operazioni **semWait()** e **semSignal()** siano implementate come **primitive atomiche**, perché manipolare un semaforo è un problema di mutua esclusione.
Possono essere implementate in **hardware** o **firmware**. In alternativa si possono usare schemi software come gli algoritmi di **Dekker** o di **Peterson**, oppure uno degli schemi di mutua esclusione supportati dall'hardware.

Si possono usare le seguenti strutture:
- Compare&Swap;
- Interrupts.

Compare&Swap

```
semWait(s)
{
   while (compare_and_swap(s.flag, 0 , 1) == 1)
      /* do nothing */;
   s.count--;
   if (s.count < 0) {
      /* place this process in s.queue*/;
      /* block this process (must also set s.flag to 0) */;
   }
   s.flag = 0;
}

semSignal(s)
{
   while (compare_and_swap(s.flag, 0 , 1) == 1)
          /* do nothing */;
      s.count++;
   if (s.count <= 0) {
      /* remove a process P from s.queue */;
      /* place process P on ready list */;
      }
      s.flag = 0;
}
```

Interrupts

```
semWait(s)
{
    inhibit interrupts;
    s.count--;
    if (s.count < 0) {
       /* place this process in s.queue */;
       /* block this process and allow interrupts */;
    }
    allow interrupts;
}

semSignal(s)
{
    inhibit interrupts;
    s.count++;
    if (s.count <= 0) {
       /* remove a process P from s.queue */;
       /* place process P on ready list */;
    }
    allow interrupts;
}
```

Le funzioni per i semafori sono:

```
int sem_init(sem_t * sem, int pshared, unsigned value); //initialization
int sem_wait(sem_t *sem); //wait
int sem_post(sem_t *sem); //signal
int sem_destroy(sem_t *sem); //destruction
```

I parametri rappresentano:
- **sem**: il semaforo;

- **pshared**: 0 se il semaforo è condiviso tra thread, 1 se è condiviso tra processi;
- **value**: il valore iniziale (quante risorse si possono condividere).

Le funzioni restituiscono -1 in caso di errore, 0 altrimenti. In caso di errore viene impostato **errno**, per consentire l'identificazione del tipo di errore.

Esempio:

```
...
#include <semaphore.h>

sem_t sem;

void* thread_fun(void* arg){
   sem_wait(&sem);
   //critical section
   sem_post(&sem);
}

int main (){
   sem_init(&sem,0,1);
   pthread_t t1,t2;
   pthread_create(&t1,NULL,thread,NULL);
   pthread_create(&t2,NULL,thread,NULL);
   pthread_join(t1,NULL);
   pthread_join(t2,NULL);
   sem_destroy(&sem);
   return 0;
}
```

Condition Variable

Soluzione al problema della mutua esclusione tra processi, gestita con una variabile:

```
int bolt = 0;
    void P(int i)
    {
        while (bolt == 1) /* do nothing */;
        bolt = 1;
        /* critical section */;
        bolt = 0;
        /* remainder */;
    }
```

Concorrenza: Altri meccanismi

Altri meccanismi comuni di sincronizzazione sono:
- **Monitor**: costrutto di un linguaggio di programmazione che incapsula variabili, procedure di accesso e codice di inizializzazione all'interno di un tipo di dato astratto. Le variabili del monitor sono accessibili solo tramite le sue procedure di accesso e un solo processo alla volta può accedere attivamente al monitor. Le procedure di accesso sono sezioni critiche. Un monitor può avere una coda di processi in attesa di accedervi.
- **Event Flags**: parola di memoria usata come meccanismo di sincronizzazione. Il codice applicativo può associare un evento diverso a ciascun bit di un flag. Un thread può attendere un singolo evento o una combinazione di eventi controllando uno o più bit del flag corrispondente. Il thread è bloccato finché tutti i bit richiesti sono impostato (AND) oppure finché almeno uno dei bit è impostato (OR).
- **Mailboxes/Messages**: mezzo con cui due processi scambiano informazioni, utilizzabile anche per la sincronizzazione.

- **Barriers** (barriere): quando un processo raggiungi la barriera, viene bloccato finché tutti i processi hanno raggiungo la barriera. Ciò permette a gruppi di processi di sincronizzarsi.
- **Spinlocks**: meccanismo di mutua esclusione in cui un processo esegue un ciclo infinito in attesa che il valore di una variabile lock indiche la disponibilità.

Monitor

Il **monitor** è un costrutto di linguaggio di programmazione che fornisce una funzionalità equivalente a quella dei semafori ed è più facile da controllare.
E' implementato in diversi linguaggi di programmazione, tra cui:
- Concurrent Pascal;
- Pascal-Plus;
- Modula-2;
- Modula-3;
- Java.

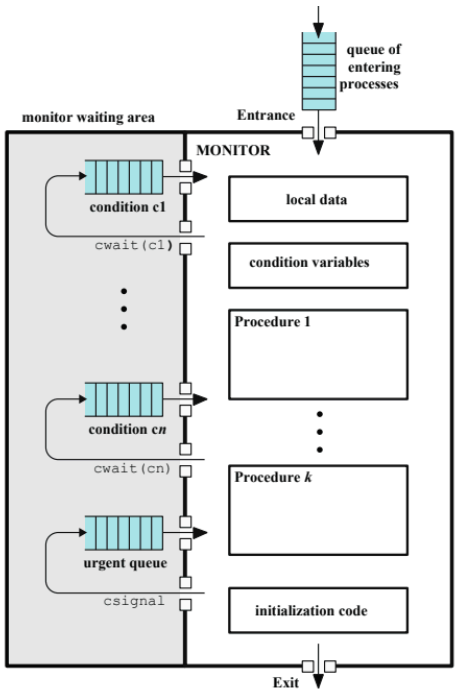
E' stato implementato anche come libreria di programma: un modulo software costruito da una o più procedure, una sequenza di inizializzazione e dati locali.

Un monitor ha tre caratteristiche:
1. le variabili di dati locali sono accessibili solo dalle procedure del monitor e non da procedure esterne;
2. un processo entra nel monitor invocando una delle sue procedure;
3. un solo processo alla volta può essere in esecuzione nel monitor.

Le caratteristiche 1 e 2 sono simili a quelle degli oggetti nei linguaggi orientati agli oggetti, dove un monitor è facilmente impresentabile come un oggetto con caratteristiche speciali.
La caratteristica 3 è necessaria per garantire la mutua esclusione. Una struttura di dati condivisa può può essere protetta inserendola in un monitor, che fornisce una funzione di esclusione reciproca per l'accesso alla risorsa.

Event Flags

La sincronizzazione si ottiene attraverso l'uso di **variabili di condizione** (condition variables), che sono contenute all'interno del monitor e accessibili solo al suo interno.
Le variabili di condizione sono manipolare da due funzioni:
- **cwait(c)**: sospende l'esecuzione del processo chiamante sulla condizione c;
- **csignal(c)**: riprende l'esecuzione di un processo bloccato da una **cwait** sulla stessa condizione.



Mailboxes/Message

Quando i processi interagiscono tra loro, devono essere soddisfatti due requisisti fondamentali:
- **sincronizzazione**: per imporre la mutua esclusione;
- **comunicazione**: per scambiare informazioni.

Il **message passing** (scambio di messaggi) è un approccio per fornire entrambe queste funzioni, e funziona sia con sistemi distribuiti sia con sistemi multiprocessore a memoria condivisa e sistemi monoprocessore.
Con i messaggi si scambino informazioni tra processi, anche su macchine differenti.

Due processi possono interagire scambiandosi messaggi, e lo fanno usando due **funzioni primitive** (2 operazioni):
- **Send** (destination, &message): un processo invia informazioni sotto forma di messaggio a un altro processo designato da una destinazione;
- **Receive** (source, &message): un processo riceve informazioni eseguendo la primitiva receive, indicando la sorgente e il messaggio.

Problematiche:
- **Ackonwledgement**: conferma di arrivo del messaggio;
- **Authentication**: identità del server.

Lo scambio di messaggi è fondamentale nel caso dei sistemi distribuiti.

Interazione fra processi:
- **Mutua esclusione**, cioè sincronizzazione;
- **Cooperazione**, cioè scambio di informazioni.

Entrambi i **requisiti** sono soddisfatti dalle **code di messaggi** (queue) che sono una risorsa consumabile (non riutilizzabile).

Lo scambio di messaggi prevede che:
- **Shared Resources**: non ci siano risorse condivise (per questo viene detta anche shared nothing, nessuna condivisione);
- **Lock**: non siano inclusi meccanismi quali l'uso dei lock o analoghi per ottenere la mutua esclusione, poiché l'uso dei soli messaggi permette ai processi che devono interagire tra loro di garantirsi la mutua esclusione e di scambiare informazioni per la cooperazione.

In questa tecnica si sfrutta un **canale di comunicazione logico** sul quale viaggia un messaggio destinato ad un indirizzo.

Nel progettare il meccanismo di scambio di messaggi vanno fatte quattro scelte:
- **Sincronizzazione**:
  - **Send**: bloccante o non bloccante;
  - **Receive**: bloccante, non bloccante o con test di messaggio d'arrivo.
- **Indirizzamento**:
  - **Diretto**: la send indica il destinatario; la receive può essere esplicita o implicita;
  - **Indiretto**: la mailbox può essere statica, dinamica o di tipo ownership.
- **Formato** di un messaggio: contenuto, con lunghezza fissa o variabile;
- **Gestione delle code** dei messaggi: FIFO o a priorità.

La comunicazione di un messaggio tra due processi implica una **sincronizzazione** tra i due: il ricevente non può ricevere un messaggio finché questo non è stato inviato da un altro processo.
Quando in un processo viene eseguita una **receive**, ci sono due possibilità:
- Se un messaggio era già stato inviato in precedenza, il messaggio viene ricevuto e l'esecuzione continua.
- Se non c'è nessun messaggio in attesa, il processo viene bloccato finché non arriva un messaggio, oppure continua l'esecuzione abbandonando il tentativo di ricevere.

I criteri normalmente usati combinano **send** e **receive** in tre modi:

1. **Send bloccante e receive bloccante**: mittente e ricevente restano bloccati finché il messaggio non è stato consegnato. questa modalità è detta a volte **rendezvous** e consente una sincronizzazione stretta tra i processi.
2. **Send non bloccante e receive bloccante**: il mittente continua l'esecuzione, mentre il ricevente resta bloccato finché non arriva il messaggio richiesto. E' la combinazione più utile, perche permette a un processo mittente di inviare uno o più messaggi a moti riceventi, il più rapidamente possibile. E' il caso di un server (un processo di servizio) che deve fornire servizi o risorse a molti altri processi.
3. **Send non bloccante e receive non bloccante**: nessuna delle due parti è obbligata ad attendere.

In ognuno dei casi ci possono essere problemi. La **send non bloccante** può consumare risorse (tempo, memoria, ecc…) e il programmatore deve controllare l'arrivo a destinazione del messaggio. Con la **receive bloccante**, il processo può rimanere bloccato se il messaggio non arriva.

Gli schemi per specificare i processi nelle primitive **send** e **receive** si dividono in due categorie:

- **Indirizzamento diretto**:
  - La primitiva **send** include un **identificatore specifico** del processo destinatario;
  - La primitiva **receive** può essere gestita in due modi:
    i. richiedono che il processo designi **esplicitamente** un processo mittente (efficace per processi concorrenti che cooperano);
    ii. con **indirizzamento implicito**: il parametro sorgente della primitiva receive assume un valore, restituito quando l'operazione di receive è stata eseguita.
- **Indirizzamento indiretto:**
  - I messaggi vengono inviati a una **struttura condivisa**, costituita da code che possono contenere temporaneamente i messaggi;
  - Le code sono dette **mailbox**;
  - Un processo invia un messaggio alla mailbox e l'altro processo preleva il messaggio dalla mailbox;
  - Consente una maggiore flessibilità nell'uso dei messaggi.
  - Sono possibili quattro tipi di relazione:
    a) **Uno a uno** (one-to-one): permette di stabilire un collegamento di comunicazione privato tra due processi. Questo isola la loro interazione da interferenze erronee da parte di altri processi.
    b) **Molti a uno** (many-to-one): è utile per l'interazione **client/server;** un processo fornisce un servizio a un certo numero di altri processi. In questo caso la mailbox viene spesso chiamata **port**.
    c) **Uno a molti** (one-to-many): permette di avere un mittente e più riceventi. E' utile per le applicazioni in cui un messaggio, o qualche informazione, deve essere trasmesso in **broadcast** a un insieme di processi.
    d) **Molti a molti** (many-to-many): permette a più processi server di fornire servizio concorrente a più clienti.

Il **formato del messaggio** dipende dagli obiettivi del sistema di messaggistica e dal fatto che questo giri su un singolo computer o su un sistema distribuito:

- In alcuni sistemi operativi i progettisti hanno preferito messaggi **brevi e di lunghezza fissa**, per ridurre al minimo l'elaborazione e l'overhead di memorizzazione.
- Se deve essere passata una grande quantità di dati, i dati possono essere messi in un **file** e il messaggio si limita a fare riferimento a quel file.
- Un approccio più flessibile è consentire messaggi di **lunghezza variabile**.

Esempio: Soluzione al problema della mutua esclusione tra processi, gestita con messaggi.

Si assume l'uso della primitiva **receive bloccante** e della primitiva **send non bloccante**. Un insieme di processi concorrenti condivide una mailbox, **box**, che può essere usata da tutti i processi per inviare e ricevere.
La mailbox viene inizializzata in modo da contenere **un solo messaggio con contenuto nullo**. Un processo che vuole entrare nella propria sezione critica prova prima a ricevere un messaggio. Se

la mailbox è vuota, il processo viene bloccato. Una volta che un processo ha acquistato il messaggio, esegue la propria sezione critica e poi rimette il messaggio nella mailbox. In questo modo il messaggio funziona come un **token** che viene passato da processo a processo.

```
/* program mutualexclusion */
const int n = /* number of processes */;

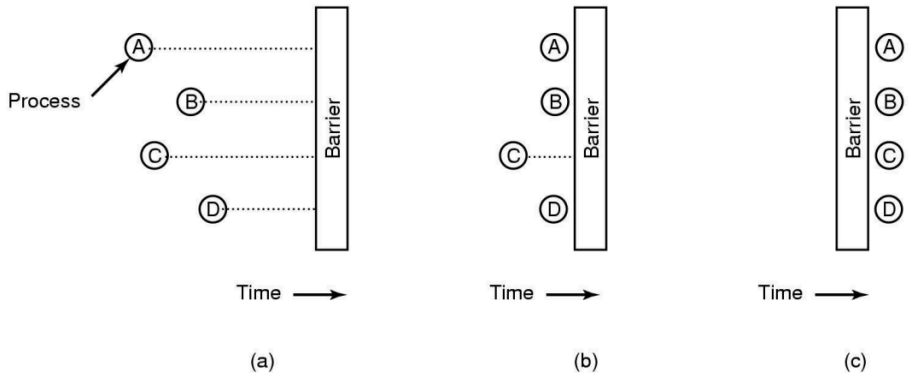
void P(int i)
{
    message msg;
    while (true) {
        receive (box, msg);
        /* critical section */;
        send (box, msg);
        /* remainder */;
    }
}

void main()
{
    create_mailbox (box);
    send (box, null);
    parbegin (P(1), P(2), . . ., P(n));
}
```

La soluzione precedente assume che, se più di un processo esegue contemporaneamente l'operazione di receive, allora:
- se c'è un messaggio, questo viene consegnato a **un solo processo** e gli altri vengono bloccati;
- se la coda dei messaggi è vuota, **tutti i processi vengono bloccati**; quando un messaggio diventa disponibile, **un solo processo bloccato** viene attivato e riceve il messaggio.

Barriers

Le barriere servono a **sincronizzare gruppi di processi**.



SpinLock

Uno **spin lock** è un modo per proteggere una risorsa condivisa dalla modifica di due o più processi contemporaneamente. E' usato nel kernel dei sistemi operativi.
Il primo processo che tenta di modificare la risorsa "acquisisce" il blocco e continua sulla sua strada, facendo ciò che era necessario con la risorsa.
Tutti gli altri processi che successivamente tentano di acquisire il blocco vengono arrestati: si dice che "ruotano in posizione" (da cui il nome spin lock) in attesa che il blocco venga rilasciato dal primo processo (**Busy Waiting**).
Quando usarlo:
- **OK** —> **sezioni critiche corte** per architetture **multiprocessore**:

  - Non ha senso un architetture **monoprocessore**: blocca l'unico processore attivo (allora forse conviene **disabilitare gli interrupts**).
  - Lo spinlock è utile negli **SMP**: evita il context switch.
- **NO** —> **sezioni critiche lunghe**: usare al posto suo **semafori** o **messaggi**, soprattutto in architetture SMP. Anche su sistemi monoprocessore, se il **context switch** è meno danno del busy waiting.

Concorrenza: producer-consumer

Resource sharing

La **condivisione di risorse** tra processi concorrenti può quindi essere implementata con 3 metodi diversi:
1. **Istruzioni hardware** (TSL,XCHG) —> busy waiting;
2. **Sincronizzazione** (semafori) —> risorsa riusabile;
3. **Message passing** (sincronizzazione + informazione) —> risorsa consumabile.

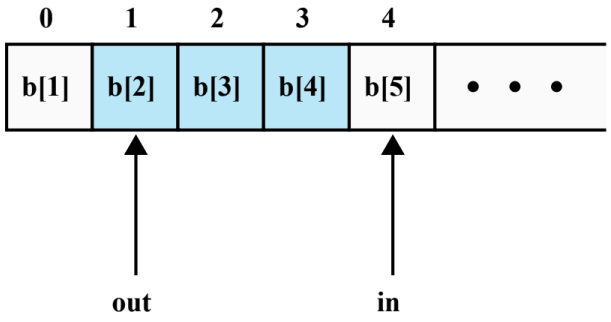
Se si può implementare un'applicazione con uno qualsiasi dei 3 metodi, allora lo si può fare anche con gli altri 2. Un particolare meccanismo potrà rivelarsi più conveniente degli altri in termini di:
- Facilità di sviluppo: considerazioni inerenti il numero di processi concorrenti;
- **Prestazioni**: considerazioni inerenti il Busy Waiting;
- **Gestione**.

Producer/Consumer Problem

Nel problema del produttore/consumatore:
- uno o più **produttori** generano dei dati e li inseriscono in un **buffer**;
- uno o più **consumatori** prelevano dal buffer gli elementi, uno alla volta;
- il buffer è condiviso, quindi solo un produttore o un consumatore alla volta può accedervi.

Il problema consiste nel garantire che:
- il produttore non possa aggiungere dati in un buffer **pieno**;
- il consumatore non possa rimuovere dati da un buffer **vuoto**.

Il buffer è un array **b[1]…b[n]** con due puntatori: **in**, la cella dove il **producer inserisce**, e **out**, la cella da cui il **consumer preleva**. Le celle colorare sono quelle **occupate**.



Producer/Consumer con semafori binari (buffer infinito)

In questa soluzione ci sono **due semafori binari** e una **variabile intera**:
- **n** è una **variabile intera** (non un semaforo) che conta gli elementi nel buffer;
- **s** (inizializzato a 1) garantisce la **mutua esclusione** sul **buffer** e su **n**;
- **delay** (inizializzato a 0) serve a far **attendere il consumer** quando il buffer è vuoto.

```
/* program producerconsumer */
int n = 0;
binary_semaphore s = 1, delay = 0;
```

```
void producer()
{
     while (true) {
          produce();
          semWaitB(s);
          append();
          n++;
          if (n==1) semSignalB(delay);
          semSignalB(s);
     }
}
```

```
void consumer()
{
     int m; /* a local variable */
     semWaitB(delay);
     while (true)  {
          semWaitB(s);
          take();
          n--;
          m = n;
          semSignalB(s);
          consume();
          if (m==0) semWaitB(delay);
     }
}
```

Producer:
1. Produce l'elemento (**produce()**).
2. Entra nella sezione critica con **semWaitB(s)**, aggiunge l'elemento (**append()**) e incrementa **n**.
3. Se **n==1**, il buffer era vuoto fino a un attimo prima, quindi il producer sveglia il consumer con **semSignalB(s)**.
4. Esce dalla sezione critica con **semSignalB(s)**.

Consumer:
1. All'inizio fa **semWaitB(delay)**; aspetta che sia stato inserito il primo elemento.
2. Nel ciclo entra nella sezione critica con **semWaitB(s)**, preleva l'elemento (**take()**), decrementa **n** e salva il valore nella variabile locale **m**.
3. Esce dalla sezione critica con **semSignalB(s)** e consuma l'elemento (**consume()**).
4. Se **m==0**, il buffer è vuoto, quindi fa **semWaitB(delay)** e si blocca finché il producer non inserisce un nuovo elemento.

Il consumer deve decidere se bloccarsi in base al valori di n dopo il decremento. Se leggesse n fuori dalla sezione critica, nel frattempo il producer potrebbe averlo modificato, e il consumer prenderebbe la decisione su un valore non più corretto (ad esempio bloccandosi anche se c'è un elemento). Copiano n in m mente è ancora dentro la sezione critica, la decisione si basa sul valore giusto.

Producer/Consumer con semafori generali (buffer infinito)

```
/* program  producerconsumer */
semaphore n = 0, s = 1;
```

```
void producer()
{
   while (true) {
        produce();
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
   }
}
```

```
void consumer()
{
   while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        consume();
   }
}
```

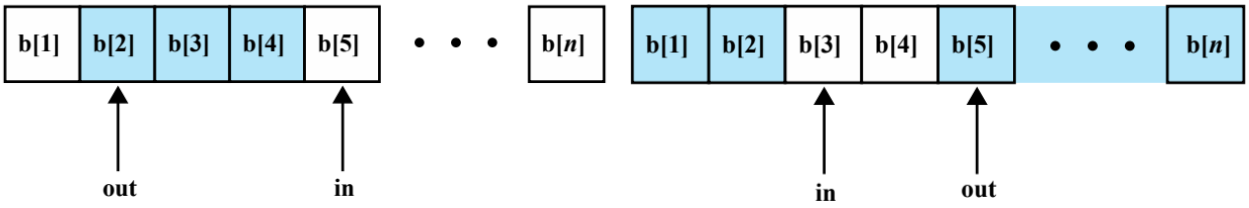
Rispetto alla soluzione con semafori binari, qui bastano due semafori:
- n (inizializzato a 0) è un semaforo generale che conta gli elementi presenti nel buffer.
- s (inizializzato a 1) garantisce la mutua esclusione sull'accesso al buffer.

Producer:
1. Produce l'elemento (produce()).
2. Entra nella sezione critica con semWait(s), aggiunge l'elemento (append()) ed esce con semSignal(s).
3. Fa semSignal(n), incrementando il numero di elementi disponibili. Il producer non attende mai, perché il buffer è infinito e non può essere pieno.

Consumer:
1. **semWait(n)**; se non ci sono elementi (n=0) il consumer si blocca finché il producer non ne inserisce uno.
2. Poi entra nella sezione critica con **semWait(s**), preleva l'elemento (**take()**) ed esce con **semSignal(s)**.
3. Consuma l'elemento fuori dalla sezione critica (**consume()**).

L'ordine delle due **semWait** nel consumer è importante. Se facesse **semWait(s)** prima di **semWait(n)**, potrebbe bloccarsi sul buffer vuoto tenendo **s**, e il producer non potrebbe più entrare nella sezione critica per inserire elementi: **deadlock**.
Il semaforo **n** sostituisce sia la variabile intera **n** sia il semaforo **delay** della versione binaria, quindi non serve più la copia locale **m**.

Buffer Circolare



Nel problema del produttore/consumatore con buffer finito il buffer condiviso viene realizzato come **buffer circolare**: un array di **n** celle usato "a giro", in cui, arrivati all'ultima cella, si ricomincia dalla prima. In questo modo il producer può riutilizzare le celle già svuotate dal consumer, e la zona occupata può anche "girare intorno" alla fine dell'array.
A differenza del buffer infinito, qui servono anche due condizioni di attesa: il producer deve fermarsi se il buffer è **pieno**, e il consumer se è **vuoto**. Come visto per il buffer infinito, la mutua esclusione sull'accesso al buffer è garantita dai semafori.

Bounded-buffer Producer/Consumer

Nel caso del produttore/consumatore con **buffer finito**, il problema si risolve usando tre semafori:
- **s** (inizializzato a 1): garantisce la mutua esclusione sul buffer;
- **n** (inizializzato a 0): conta gli elementi presenti;
- **e** (inizializzato a **sizeofbuffer**) conta le celle libere.

```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n= 0, e= sizeofbuffer;
```

```
void producer()
{
     while (true) {
          produce();
          semWait(e);
          semWait(s);
          append();
          semSignal(s);
          semSignal(n);
     }
}
```

```
void consumer()
{
     while (true) {
          semWait(n);
          semWait(s);
          take();
          semSignal(s);
          semSignal(e);
          consume();
     }
}
```

Producer:
- Dopo aver prodotto l'elemento, fa **semWait(e)**: se non ci sono celle libere (buffer pieno) si blocca finché il consumer non ne libera una.
- Entra nella sezione critica con **semWait(s)**, aggiunge l'elemento (**append()**) ed esce con **semSignal(s)**.
- Fa **semSignal(n)**, per segnalare che c'è un elemento in più.

Consumer:
- **semWait(n)** lo blocca se il buffer è vuoto.
- Entra nella sezione critica con **semWait(s)**, preleva l'elemento (**take()**) ed esce con **semSignal(s)**.
- Fa **semSignal(e)**, per segnalare che si è liberata una cella.
- Consuma l'elemento (**consume()**) fuori dalla sezione critica.

Rispetto al buffer infinito è stato aggiunto il semaforo **e**, poiché ora anche il producer deve poter aspettare.

Cosa succede scambiando **due semWait adiacenti**?
Si rischia il **deadlock**. Se il producer facesse **semWait(s)** prima di **semWait(e)**, a buffer pieno resterebbe bloccato su **e** tenendo **s**, e il consumer, cha ha bisogno di **s** per prelevare e liberare una cella, non potrebbe mai entrare: nessuno dei due andrebbe avanti. Lo stesso vale per il consumer: scambiando **semWait(n)** e **semWait(s)**, a buffer vuoto si bloccherebbe su **n** tenendo **s**, e il producer non potrebbe inserire nulla. Quindi prima si aspetta la risorsa (**e** o **n**), poi si prende la mutua esclusione.

Cosa succede scambiando **due semSignal adiacenti**?
Non causa problemi di correttezza: un **signal** non blocca mai, quindi non può creare deadlock. Cambia solo l'ordine in cui vengono rilasciati o svegliati i processi, ad esempio un consumer svegliato da **n** potrebbe trovare **s** ancora occupato e attendere un attimo.

Cosa succede se c'è **solo un producer o solo un consumer**?
Il programma si blocca prima o poi. Con solo il producer, **e** viene decrementato a ogni inserimento: dopo **sizeofbuffer** elementi vale 0 e il producer si ferma su **semWait(s)**, perche nessuno libera celle. Con solo il consumer, **n** vale 0 fin dall'inizio, quindi si blocca subito su **semWait(n)**, perche nessuno produce elementi.

<u>Bounded-buffer P/C using Monitors</u>

```
void producer()
{
     char x;
     while (true) {
     produce(x);
     append(x);
     }
}
void consumer()
{
     char x;
     while (true) {
       take(x);
       consume(x);
     }
}
```

Un **monitor** è un costruttore che racchiude dati e procedure in un unico blocco:
- i dati locali sono accessibili solo dalle procedure del monitor;
- un solo processo alla volta può essere in esecuzione nel monitor;

- il programmatore può decidere quando una procedura in esecuzione nel monitor deve fermarsi o riprendere, in base a un predicato di condizione.

Per questo **producer()** e **consumer()** si limitano a chiamare **append(x)** e **take(x)**, senza sapere come sono implementate all'interno del monitor. Al loro interno non ci sono semafori: la sincronizzazione sta tutta nel monitor.

```
/* program producerconsumer */
monitor boundedbuffer;
char buffer [N];                                    /* space for N items */
int nextin, nextout;                                  /* buffer pointers */
int count;                                  /* number of items in buffer */
cond notfull, notempty;  /* condition variables for synchronization */
```

Init: `nextin = nextout = count = 0;`

```
void append (char x)
{
     if (count == N) cwait(notfull);
     buffer[nextin] = x;
     nextin = (nextin + 1) % N;
     count++;
     /* one more item in buffer */
     csignal(notempty);
}
```

```
void take (char x)
{
     if (count == 0) cwait(notempty);
     x = buffer[nextout];
     nextout = (nextout + 1) % N;
     count--;
     csignal(notfull);
}
```

append(x):
- Se **count==N** (buffer pieno) fa **cwait(notfull)** e aspetta.
- Inserisce **x** in **buffer[nextin]** e aggiorna **nextin = (nextin +1) % N**, quindi **count++**.
- Fa **csignal(notempty)**, per avvisare un eventuale consumer in attesa che c'è un elemento in più.

take(x):
- Se **count==0** (buffer vuoto) fa **cwait(notempty)** e aspetta.
- Preleva **x** da **buffer[nextout]** e aggiorna **nextout = (nextout + 1)%N**, quindi **count- -**.
- Fa **csignal(notfull)**, per avvisare un eventuale producer in attesa che si è liberata una cella.

Bounded-buffer Producer/Consumer using messages

```
const int
    capacity = /* buffering capacity */ ;
    null =/* empty message */ ;
int i;
void producer()
{   message pmsg;
    while (true) {
     receive (mayproduce, pmsg);
     pmsg = produce();
     send (mayconsume, pmsg);
    }
}
void consumer()
{   message cmsg;
    while (true) {
     receive (mayconsume, cmsg);
     consume (cmsg);
     send (mayproduce, null);
    }
}

void main()
{
    create_mailbox (mayproduce);
    create_mailbox (mayconsume);
    for (int i = 1; i <= capacity; i++) send (mayproduce,
null);
    parbegin (producer, consumer);
}
```

Questa soluzione al problema del produttore/consumatore con buffer finito usa solo lo scambio di messaggi, tramite due mailbox:
- **mayproduce**: contiene un messaggio vuoto (**null**) per ogni cella libera del buffer. Inizialmente è **piena**, con **capacity** messaggi;
- **mayconsume**: contiene i messaggi con i dati prodotti, pronti per essere consumati. Inizialmente è **vuota**.

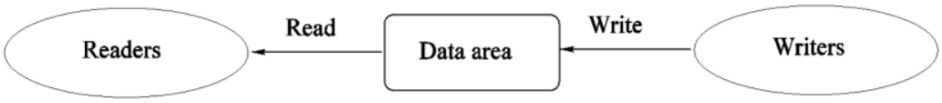
main:
- Crea le due mailbox.
- Con un ciclo for invia **capacity** messaggi **null** a **mayproduce**, che rappresentano le celle libere.
- Avvia producer e consumer in parallelo con **parbegin**.

producer:
- Fa **receive(mayproduce, pmsg)**: prende un messaggio vuoto, cioè si prende una cella libera. Se **mayproduce** è vuota (buffer pieno), si blocca finché il consumer non ne rimette uno.
- Produce il dato con **produce()** e lo invia **mayconsume** con **sen(mayconsume, pmsg)**.

consumer:
- Fa **receive(mayconsume, cmsg)**: se non ci sono dati (buffer vuoto) si blocca finché il producer non ne invia uno.
- Consuma il dato con **consume(cmsg)**.
- Fa **send(mayproduce, null)**: restituisce un messaggio vuoto, cioè libera una cella.

In pratica i messaggi in **mayproduce** fanno da permessi per produrre (come il semaforo **e** della versione con semafori), mentre quelli in **mayconsume** fanno da elementi presenti nel buffer (come il semaforo **n**). Il numero totale di messaggi in circolazione resta sempre pari a **capacity**, e questo è ciò che limita il buffer.

Reader/Writes Problem



Una base di dati è condivida fra vari processi concorrenti, che si dividono in due categorie:
- **Lettori** (readers): accedono solo in lettura e non modificano la base di dati;
- **Scrittori** (writers): possono sia leggere che scrivere.

Il problema consiste nel far convivere questi due tipi di accesso rispettando alcune regole:
- Da un lato, più lettori possono leggere contemporaneamente, perché la lettura non crea interferenze.
- Dall'altro, solo uno scrittore alla volta può accedere alla base di dati (accesso esclusivo), e finche è in atto una scrittura non è permessa alcuna lettura.

Rispettare queste regole, però, non basta a stabilire chi debba passare per primo quando lettori e scrittori sono in attesa nello stesso momento. A seconda di chi si decide di favorire si ottengono due varianti: nella prima la **priorità va ai lettori**, nella seconda **agli scrittori**. Ciascuna ha un difetti: favorire una categoria può lasciare l'altra in attesa per un tempo indefinito (**starvation**).

Variante 1 (priorità ai lettori):
- Nessun lettore deve essere tenuto in attesa, a meno che uno scrittore abbia già ottenuto l'accesso alla base di dati condivisa.
- Nessun lettore deve essere messo in attesa solo perché uno scrittore sta aspettando di scrivere.
- E' soggetta però al problema della **starvation** degli scrittori.

Variante 2 (priorità agli scrittori):
- Uno scrittore deve poter svolgere la scrittura il più presto possibile.
- Nessun nuovo lettore deve iniziare la lettura se uno scrittore è in attesa di scrivere.
- E' soggetta pero al problema della **starvation** dei lettori.

Soluzione con semafori (priorità ai lettori)

```
/* program readersandwriters */
int readcount = 0;
semaphore x = 1, wsem = 1;
```

Requisiti:
- un numero qualsiasi di lettori può leggere contemporaneamente;
- solo uno scrittore alla volta può scrivere;
- quando uno scrittore sta scrivendo, nessun lettore può leggere.

```
while (true) {
  semWait (wsem);
  WRITEUNIT();
  semSignal (wsem);
}
```

```
while (true) {
  semWait (x);
  readcount++;
  if (readcount == 1) semWait (wsem);
  semSignal (x);
  READUNIT();
  semWait (x);
  readcount--;
  if (readcount == 0) semSignal (wsem);
  semSignal (x);
}
```

writer():
- Acquisisce il lock esclusivo usando il semaforo binario **wsem**.

reader():
- il semaforo binario **x** serve ad aggiornare in sicurezza la variabile **readcount**.
- Quando c'è un solo lettore (**readcount == 1**) fa il lock di **wsem** (nessuna scrittura).
- Una volta eseguita la lettura, sblocca **wsem** se nessun lettore è attivo (cioè **readcount == 0**).

Il semaforo **x** rende atomici gli aggiornamenti di **readcount**, mentre **wsem** rappresenta l'accesso alla base di dati. Solo il primo lettore che arriva prende **wsem**, e cosi blocca gli scrittori; i lettori successivi entrano senza altre attese. L'ultimo lettore che esce (**readcount == 0**) rilascia **wsem**. Finché c'è sempre almeno un lettore attivo, wsem non viene mai rilasciato e lo scrittore può aspettare all'infinito: è la starvation.

Soluzione con semafori (priorità agli scrittori)

Per i lettori serve un semaforo aggiuntivo. Non si deve lasciar formare una lunga cosa su **rsem**, altrimenti gli scrittori non potrebbero saltare la coda. Perciò solo un lettore può mettersi in coda su **rsem**, mentre gli altri lettori si mettono in coda sul semaforo **z**, subito prima di aspettare su **rsem**.

Situazione	Stato
Solo lettori nel sistema	`wsem` impostato; nessuna coda
Solo scrittori nel sistema	`wsem` e `rsem` impostati; gli scrittori in coda su `wsem`
Lettori e scrittori, con lettura per prima	`wsem` impostato dal lettore; `rsem` impostato dallo scrittore; tutti gli scrittori in coda su `wsem`; un lettore in coda su `rsem`; gli altri lettori in coda su `z`
Lettori e scrittori, con scrittura per prima	`wsem` impostato dallo scrittore; `rsem` impostato dallo scrittore; gli scrittori in coda su `wsem`; un lettore in coda su `rsem`; gli altri lettori in coda su `z`

```
void reader(int i)
{
   message rmsg;
   while (true) {
      rmsg = i;
      send(readrequest, rmsg);
      receive(mbox[i], rmsg);
      READUNIT();
      rmsg = i;
      send(finished, rmsg);
   }
}
void writer(int j)
{
   message rmsg;
   while (true) {
      rmsg = j;
      send(writerequest, rmsg);
      receive(mbox[j], rmsg);
      WRITEUNIT();
      rmsg = j;
      send(finished, rmsg);
   }
}
void controller()
{
   while (true) {
      if (count > 0) {
         if (!empty(finished)) {
            receive(finished, msg);
            count++;
         }
         else if (!empty(writerequest)) {
            receive(writerequest, msg);
            writer id = msg.id;
            count = count - 100;
         }
         else if (!empty(readrequest)) {
            receive(readrequest, msg);
            count--;
            send(msg.id, "OK");
         }
      }
      if (count == 0) {
         send(writer id, "OK");
         receive(finished, msg);
         count = 100;
      }
      while (count < 0) {
         receive(finished, msg);
         count++;
      }
   }
}
```

```
/* program readersandwriters */
int  readcount, writecount;
semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;
void reader(int i)
{
   while (true) {
      semWait (z);
         semWait (rsem);
            semWait (x);
               readcount++;
               if (readcount == 1) semWait (wsem);
            semSignal (x);
         semSignal (rsem);
      semSignal (z);
      READUNIT();
      semWait (x);
         readcount--;
         if (readcount == 0) semSignal (wsem);
      semSignal (x);
   }
}
void writer ()
{
   while (true) {
      semWait (y);
         writecount++;
         if (writecount == 1) semWait (rsem);
      semSignal (y);
      semWait (wsem);
      WRITEUNIT();
      semSignal (wsem);
      semWait (y);
         writecount--;
         if (writecount == 0) semSignal (rsem);
      semSignal (y);
   }
}
void main()
{
   readcount = writecount = 0;
   parbegin (reader(1), ... reader(m), writer(1), ... writer(n));
}
```

Si aggiungono **y** e **writecount**, che per gli scrittori fanno ciò che x e readcount fanno per i lettori.

Il primo scrittore che arriva prende **rsem** e blocca l'ingresso di nuovi lettori; solo quando l'ultimo scrittore in attesa ha finito (**writecount == 0**) **rsem** viene rilasciato. In questo modo i lettori che arrivano dopo uno scrittore in attesa restano fermi. Il semaforo **z** fa si che solo un lettore alla volta possa mettersi in coda su **rsem**, e cosi uno scrittore può sempre passare davanti ai lettori in attesa.

Soluzione con scambio di messaggi

Un processo **controller** decide chi può accedere. Lettori e scrittori gli inviano una richiesta (**readrequest** o **writerequest**) e aspettano il permesso sulla propria mailbox **mbox[i]**; a fine operazione mandano un messaggio su **finished**. Il valore di **count** indica lo stato (il valore iniziale lo assumiamo uguale a 100):

- **count > 0**: nessuno scritto in attesa, ogni lettore ammesso fa **count- -**, ogni lettore che finisce fa **count++**;
- una richiesta di scrittura sottrae 100 a **count**; quando **count == 0** non ci sono lettori attivi e lo scrittore riceve OK, poi il controller aspetta il suo **finished** e riporta **count** a 100.
- **count < 0**: uno scrittore è in attesa che i lettori ancora attivi finiscano, e il controller aspetta i loro **finished**.

Poiché il controller controlla **writerequest** prima di **readrequest**, questa soluzione favorisce gli scrittori.

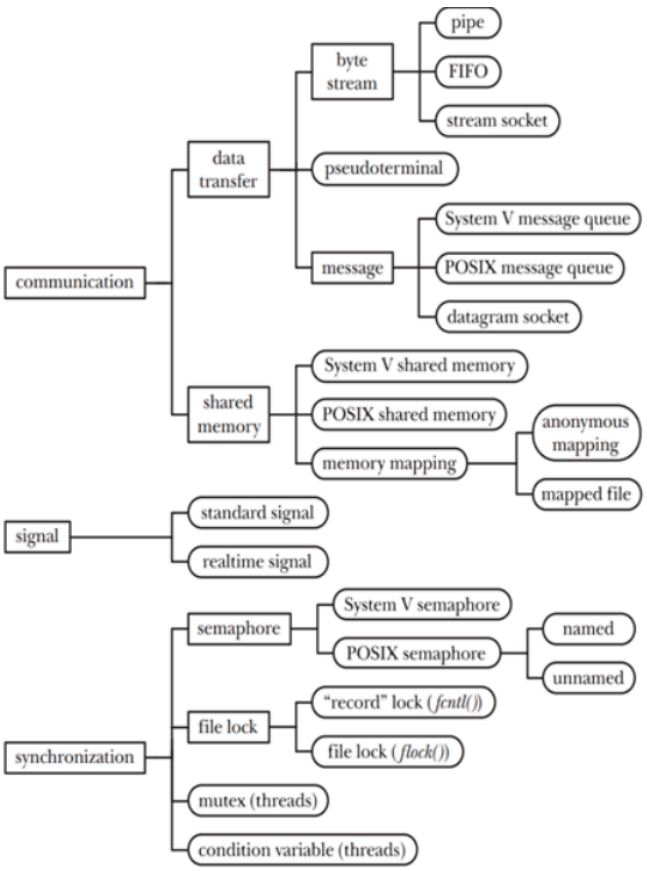
IPC (Inter Process Communication)

Ogni **processo** ha il proprio spazio di indirizzamento, e un processo generico (PID > 0) può accedere soltanto a quello. Il kernel (PID == 0), invece, può accedere a tutto.
Un processo **indipendente** non può influenzare o essere influenzato dagli altri processi in esecuzione nel sistema (non condivide risorse con altri processi).
Un processo **cooperante** può influenzare o essere influenzato da altri processi in esecuzione nel sistema (condivide risorse con altri processi):
- scopo applicativo comune;
- condivisione di informazioni (**comunicazione**);
- influenza da e verso altri processi (**sincronizzazione**).

Cooperazione = lavoro congiunto di processi per raggiungere scopi applicativi comuni con condivisione e scambio di informazioni.

Vantaggi del processo di cooperazione:
- **Condivisione delle informazioni** (es. un file o una directory condivisa): accesso concorrente;
- **Velocizzazione della computazione** (possibile in verità solo con più CPU o canali di I/O): esecuzione di sotto-attività in parallelo;
- **Modularità**: dividendo le funzioni del sistema in processi o thread separati (es. sistemi operativi modulari);
- **Convenienza** (un singolo utente può lavorare su molte attività): multi tasking.

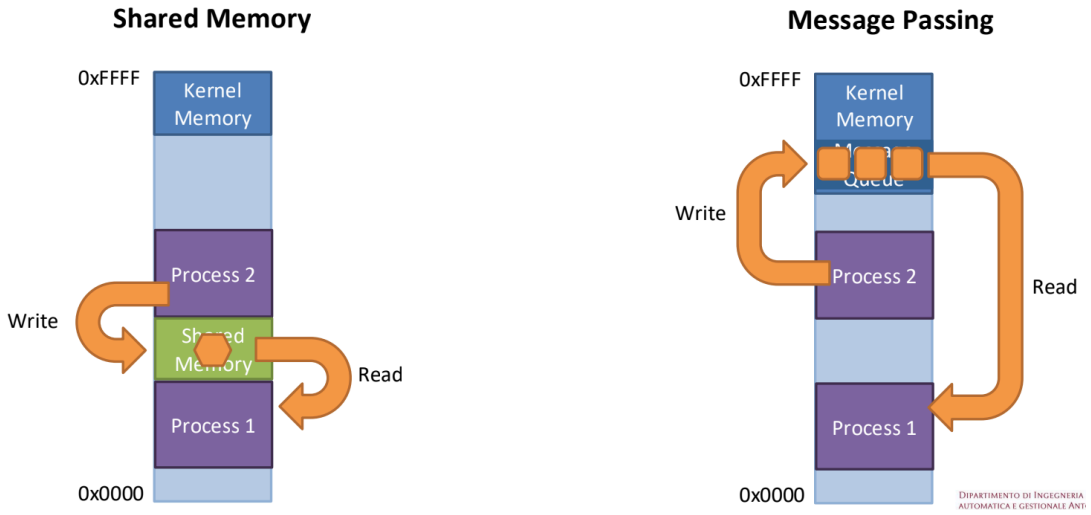
I processi cooperanti, quindi, lavorano insieme per uno scopo applicativo comune: condividono informazioni (comunicazione) e si influenzano a vicenda (sincronizzazione) . Per questo hanno bisogno di un meccanismo di comunicazione tra processi, l'**IPC**.
Ecco alcuni meccanismi per:
- comunicare;
- inviare segnali;
- sincronizzare le proprie azioni.



IPC: Meccanismi

Esistono due modelli base di IPC forniti dalla maggior parte dei sistemi:

1. **Modello a memoria condivisa** (*shared memory model):* i processi usano una memoria condivisa per scambiarsi i dati.
2. **Modello a scambio di messaggi** (*message passing model*): i processi si inviano messaggi tra loro attraverso il kernel.

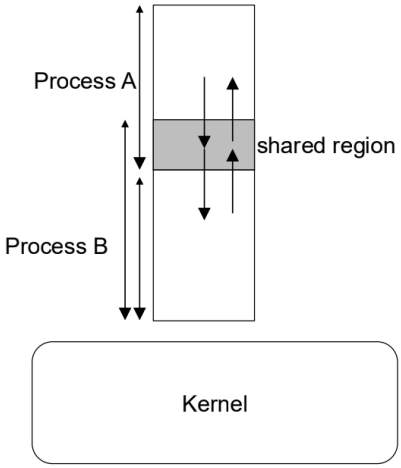


IPC: Modello di comunicazione a memoria condivisa

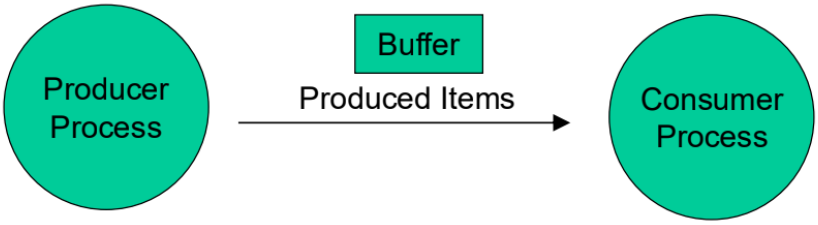
Nel modello a memoria condivisa i processi scambiano i dati usando una regione di memoria che condividono, invece di passare messaggi attraverso il kernel. Questa scelta ha conseguenze precise su come si comporta la comunicazione; ecco alcune caratteristiche abilitate:

- quantità di informazioni da trasmettere;
- velocità di esecuzione;
- semplicità di uso nelle applicazioni;
- integrazione nel linguaggio di programmazione.

Una **regione di memoria condivisa** viene stabilita tra due o più processi con l'aiuto del kernel del sistema operativo (cioè tramite system call).
I processi possono **leggere** e **scrivere** nella regione (**segmento**) di memoria condivisa direttamente, come un normale accesso alla memoria (accesso tramite puntatore). Durante questo tempo il kernel non è coinvolto, quindi l'accesso è più veloce.

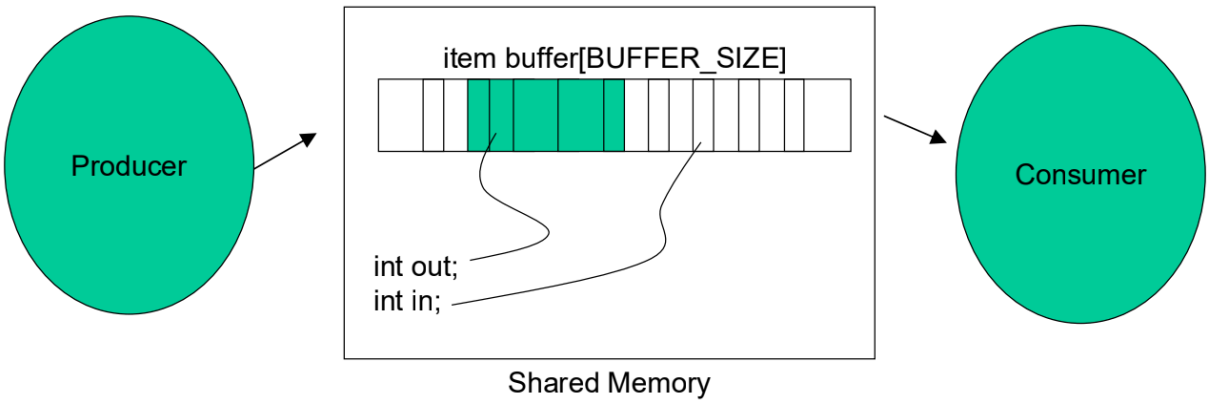


Per illustrare l'uso di un meccanismo IPC, un problema modello generale, si usa il **problema del produttore/consumatore** con **buffer limitato**.



Per risolvere il problema produttore/consumatore con buffer limitato tramite memoria condivisa, nella regione condivisa si mettono il buffer (un array **item buffer[BUFFER_SIZE]**) e due variabili intere, **in** e **out**:

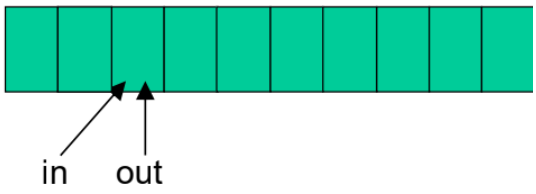
- **in** indica la posizione in cui il producer scriverà il prossimo elemento;
- **out** indica la posizione da cui il consumer leggerà il prossimo elemento.



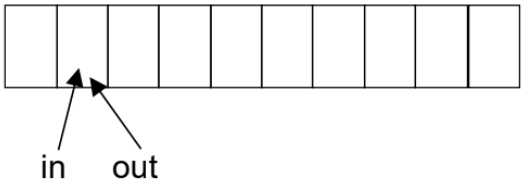
Il **buffer è circolare**: dopo l'ultima posizione si torna alla prima, ed è per questo che nel codice gli indici avanzano con **% BUFFER_SIZE**.

Buffer pieno e buffer vuoto: in entrambi i casi vale **in == out**, quindi guardare i soli indichi non basta per distinguere le due situazioni. A distinguerle sono i valori dei semafori:

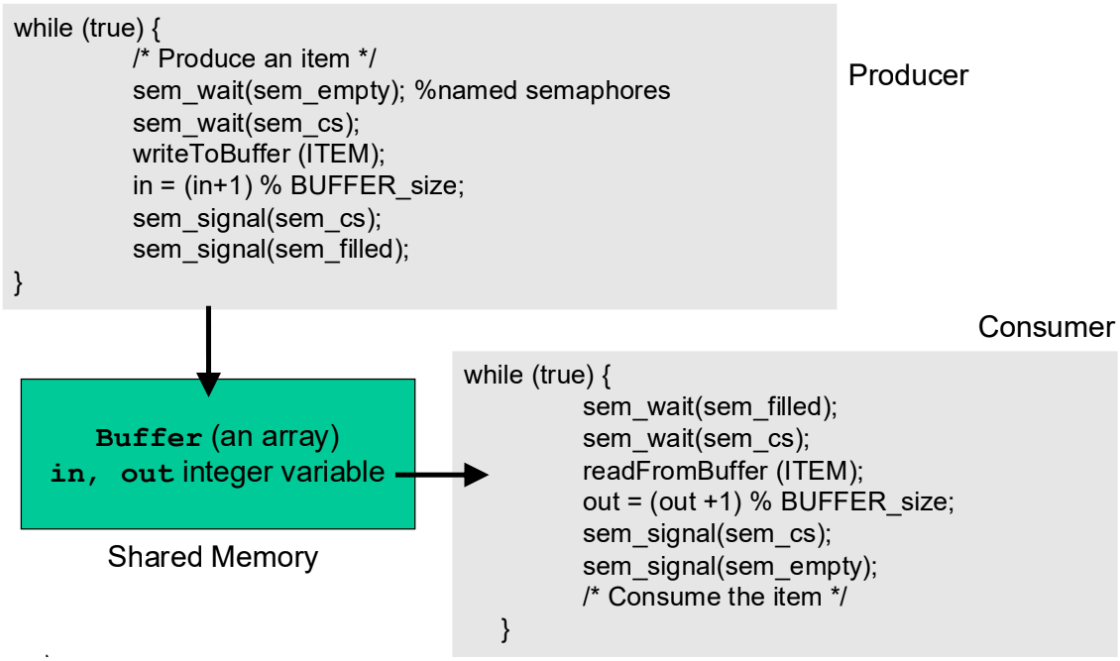
- **Buffer pieno**: si imposta **sem_empty.val == 0** e **sem_filled.val == BUFFER_SIZE**;



- **Buffer vuoto**: si imposta **sem_empty.val = BUFFER_SIZE** e **sem_filled.cal == 0**.



sem_empty conta le celle libere, **sem_filled** quelle occupate e **sem_cs** garantisce la mutua esclusione sul buffer.



Il **producer** aspetta una cella libera (**sem_empty**), entra nella sezione critica (**sem_cs**), scrive l'elemento, avanza **in**, esce dalla sezione critica e segnala che c'è una cella piena in più (**sem_filled**).
Il **consumer** fa il percorso speculare: aspetta una cella piena, entra nella sezione critica, legge l'elemento, avanza **out**, esce e segnala che c'è una cella libera in più (**sem_empty**). Consuma l'elemento fuori dalla sezione critica.

Per usare la memoria condivisa POSIX bisogna includere **#include sys/man.h**; poi le funzioni si usano in sequenza, ciascuno con un ruolo preciso nel ciclo di vita della memoria condivisa:

- **shm_open()**: crea e/o apre una pagina di memoria condivisa e restituisce un file descritto che la rappresenta.
- ltrunc() o **ftruncate()**: limita la dimensione della pagina di memoria condivisa.
- **mmap()**: mappa la pagina di memoria nello spazio di indirizzamento del processo (ora si può leggere/scrivere la pagina usano un puntatore).
- **shm_unlink()**: rimuove una pagina condiva (i processi con riferimenti aperti possono ancora accedere alla pagina).

Esempio. Shared Memory per scrivere dati

```
/* Program to write some data in shared memory */
int main() {
    const int SIZE = 4096;

    const char * name = "MY_PAGE";
    const char * msg = "Hello World!";
    int shm_fd;
    char * ptr;

    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, SIZE);
    ptr = (char *) mmap(0, SIZE, PROT_WRITE,
                        MAP_SHARED, shm_fd, 0);
    sprintf(ptr, "%s", msg);
    close(shm_fd);
    return 0;
}
```

Esempio: Shared Memory per leggere dati

```
/* Program to read some data from shared memory */
int main() {
    const int SIZE = 4096;

    const char * name = "MY_PAGE";
    int shm_fd;
    char * ptr;

    shm_fd = shm_open(name, O_RDONLY, 0666);
    ptr = (char *) mmap(0, SIZE, PROT_READ,
                        MAP_SHARED, shm_fd, 0); // if(ptr != MAP_FAILED)
    printf("%s\n", ptr);
    close(shm_fd);
    shm_unlink(shm_fd);
    return 0;
}
```

IPC: Modello di comunicazione a scambio di messaggi

Nel modello a scambio di messaggi (Message Passing) i processi non condividono una ragione di memoria, ma si inviano messaggi attraverso il kernel: un processo scrive il messaggio e l'altro lo legge. Poiché lo scambio passa dal kernel, questo modello abilita caratteristiche diverse da quelle della memoria condivisa. Ecco alcune caratteristiche abilitate:
- scalabilità;
- omogeneità delle comunicazioni;
- affidabilità;
- sicurezza;
- protezione.

Nei sistemi Unix/Linux esistono due meccanismi di comunicazione tra processi a canale:
- **pipe**: permette la comunicazione unidirezionale tra un padre e un figlio; è facile da usare e viene rimossa automaticamente quando il processo termina;
- **named-pipe** (detta anche **FIFO**): ha un nome, non viene rimossa automaticamente alla fine dei processi, non richiede una relazione padre-figlio ed è bidirezionale, quindi due processi qualsiasi possono crearla e usarla.

Pipe

La **pipe** è un canale di comunicazione tra due processi, e il suo funzionamento segue la modalità del produttore e consumatore:
- il produttore scrive da un'estremità (write-end);
- il consumatore legge dall'altra estremità (read-end);
- comunicazione unidirezionale.



Il termine "pipe" significa *tubo* in inglese, e la comunicazione avviene in modo **monodirezionale**, analogamente al normale scorrimento dei liquidi nei tubi. L'obiettivo delle pipe è permettere a più processi di comunicare come se stessere accedendo a dei file sequenziali.

Per quanto riguarda il funzionamento, una volta lette le informazioni spariscono dalla PIPE e non possono più ripresentarsi, a meno che non vengano esplicitamente riscritte dal processo che ha aperto la pipe in scrittura, all'altra estremità del tubo. Dal punto di vista dell'implementazione, la pipe è un **buffer** di dimensione più o meno grande (solitamente 4096 byte), e un processo dovrebbe essere bloccato qualora tenti di scrivere su una PIPE piena.
L'accesso è **sequenziale**, sia in scrittura sia in lettura, e nei sistemi UNIX l'uso delle PIPE avviene attraverso la nozione di descrittore. Infine, per i permessi, deve esistere una relazione padre-figlio tra i due processi, "relazionati" come conseguenza di operazioni **fork()**; le named-PIPE (FIFO) permettono invece la comunicazione anche tra processi non relazionati.

Creazione e referenziazione:

int pipe(int fd[2])	
Descrizione	invoca la creazione di una PIPE
Argomenti	fd: puntatore ad un buffer di due interi - fd[0] : descrittore di lettura dalla PIPE - fd[1]: descrittore di scrittura sulla PIPE
Restituzione	-1 in caso di fallimento, 0 altrimenti

- **fd[0]** è un canale aperto in **lettura** che consente a un processo di **leggere** dati da una PIPE.
- **fd[1]** è un canale aperto in **scrittura** che consente a un processo di **immettere** dati sulla PIPE.

fd[0] e **fd[1]** possono essere usati come normali **descrittori di file** tramite le chiamate **read()** e **write()**.

Lettura e scrittura:
- Un processo che provi a leggere da una pipe vuota rimane bloccato finché non ci sono dati disponibili.
- Un processo che provi a scrivere su una pipe piena rimane bloccato finché un altro processo non ha letto (e rimosso) abbastanza dati da permette la scrittura.
- Una pipe ha una dimensione massima equivalente a 16 pagine di memoria in Linux (65.536 byte in un sistema con una page size di 4096 byte).
- E' possibile rendere la lettura e la scrittura non bloccanti.
- E' possibile cambiare la dimensione della pipe.
- Una scrittura di **n <= PIPE_BUF** byte è atomica:
  - **PIPE_BUF** = 4096 byte;
  - si blocca se non ci sono **n** byte disponibili.
- Una scrittura di **n > PIPE_BUF** byte si potrebbe inframezzare con altre **write()**:
  - il processo rimane bloccato finché non sono stati scritti tutti gli **n** byte.
- Nel caso di write non bloccanti:
  - se **n <= PIPE_BUF** ma non ci sono n byte disponibili, **write()** fallisce e setta **errno**;
  - se **n > PIPE_BUF** potrebbe risultare in una scrittura parziale (ma non in un errore).

Le **PIPE** non sono **dispositivi** fisici, ma **logici**. La gestione dell'**EOF** (fine del file) di una PIPE è curata dal S.O. per convenzione: ciò avviene quando tutti i processi scrittori che condividevano il **descrittore fd[1]** lo hanno **chiuso**. In questo caso la chiamata **read()** effettuata da un lettore **restituisce zero** come notifica dell'evento che tutti gli scrittori hanno terminato il loro lavoro. Analogamente, un processo scrittore che tenti di **scrivere** sul descrittore **fd[1]** quando **tutte le spie del descrittore fd[0]** siano state chiuse (non ci sono lettori sulla PIPE) riceve il segnale **SIGPIPE**, altrimenti detto "Broken pipe".

Deadlock:
Per fare in modo che tutto funzioni correttamente e non si verifichino situazioni di **deadlock**, è necessario che tutti i **processi chiudano i descrittori di PIPE** che **non** gli **servono**, usando una normale **close()**.
Si noti che ogni processo lettore che **eredita** la coppia (**fd[0]**, **fd[1]**) deve chiudere la propria **copia di fd[1]** prima di mettersi a leggere da **fd[0]**, dichiarando cosi di non essere uno scrittore. Altrimenti, l'evento "tutti gli scrittori hanno terminato" non potrebbe mai avvenire se il lettore è impegnato a leggere, e si potrebbe avere un **deadlock**.

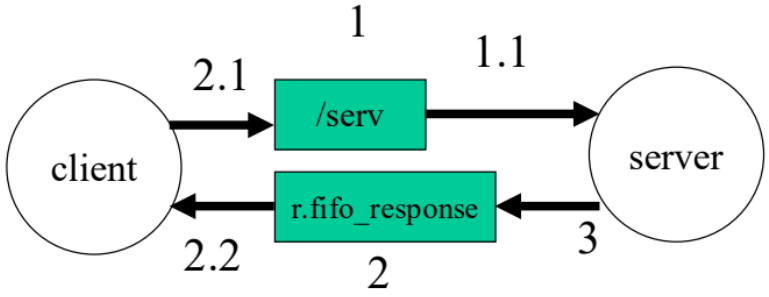
FIFO

int mkfifo(char *name, int mode)	
Descrizione	invoca la creazione di una FIFO
Argomenti	1) *name: nome della FIFO da creare 2) mode: specifica i permessi di accesso alla FIFO
Restituzione	-1 in caso di fallimento, 0 altrimenti

La rimozione di una **FIFO** dal file system avviene mediante la chiamata di sistema **unlink()**:
- normalmente, l'apertura di una FIFO è bloccante, nel senso che il processo che tenti di aprirla in lettura (scrittura) viene bloccato fino a quando un altro processo non la apre in scrittura (lettura);
- se si vuole inibire questo comportamento è possibile aggiungere il flag **O_NONBLOCK** al valore del parametro **mode** passato alla system call **open()** su di una FIFO;
- ogni FIFO deve avere sia un lettore che uno scrittore: se un processo tenta di scrivere su una FIFO che non ha un lettore esso riceve il segnale **SIGPIPE** da parte del sistema operativo (analogamente alle PIPE).

	pipe	**FIFO**
Apertura	ottenere descrittore per la lettura Ottenere descrittore per la scrittura Chiudere i 2 descrittori che non saranno utilizzati	Apertura in un solo senso (lettura o scrittura)
Scrittura	`write(fd[1], message, length)`	`write(fdc, message, length)`
Lettura	`read(fd[0], message, length)`	`read(fdc, message, length)`
Chiusura	Chiudere i 2 descrittori che sono stati utilizzati	

Esempio. Server-Client FIFO



L'esempio realizza un'architettura **client-server** con due FIFO: la FIFO pubblica **/serv**, su cui i client mandano le richieste, e una FIFO di risposta creata da ogni client (**r.fifo_response**).

1. (N^ 1)**Il server crea /serv con mkfifo e la apre in sola lettura** (**O_RDONLY**). L'apertura è bloccante: il server resta fermo finché un client non apre la FIFO in scrittura.
2. (N^ 2)**Il client prepara la risposta**. Chiede all'utente un carattere minuscolo, controllato con il confronto con **'a'** e **'z'**, e lo usa come nome della propria FIFO di risposta (**fifo_response[1] = '\0'** chiude la stringa dopo il primo carattere). Poi crea quella FIFO con **mkfifo**: se la creazione fallisce (ad esempio perche il nome è già in uso) stampa "servente sovraccarico - riprovare".
3. (N^ 2.1)**Il client invia la richiesta**. Apre **/serv** in scrittura (se non esiste stampa "servizio non disponibile" e rimuove la propria FIFO con **unlink**) e scrive nella FIFO la struttura **request**, che contiene il nome della FIFO su cui vuole la risposta. Poi chiude **/serv**.
4. (N^ 1.1)**Il server riceve la richiesta** con **read** e stampa il nome della FIFO di restituzione. Con **sleep(10)** simula il tempo di esecuzione del servizio.
5. (N^ 3)**Il server risponde**. Apre la FIFO indicata dal client in scrittura (**O_WRONLY**), scrive la stringa **"fatto"** e la chiude.
6. (N^ 2.2)**Il client legge la risposta**. Apre la sua FIFO in lettura (**O_RDONLY**), legge con **read**, stampa "risposta = fatto", poi chiude la FIFO e la rimuovo con **unlink**.

Server:

```
#include <stdio.h>
#include <fcntl.h>

typedef struct {
    long type;
    char fifo_response[20];
} request;

int main(int argc, char *argv[]) {
    char *response = "fatto";
    int pid, fd, fdc, ret;
    request r;

    ret = mkfifo("/serv", 0666);                    /* 1 */
    if (ret == -1) {
        printf("Errore nella chiamata mkfifo\n");
        exit(1);
    }

    fd = open("/serv", O_RDONLY);

    while (1) {
        ret = read(fd, &r, sizeof(request));        /* 1.1 */
        if (ret != 0) {
            printf("Richiesto un servizio (fifo di restituzione = %s)\n",
                r.fifo_response);
            /* switch sul tipo di servizio */
            sleep(10); /* emulazione di ritardo per il servizio */
            fdc = open(r.fifo_response, O_WRONLY);
            write(fdc, response, 20);               /* 3 */
            close(fdc);
            exit(0);
        } /* end if (ret != 0) */
    }
```

Client:

```
#include <stdio.h>
#include <fcntl.h>

typedef struct { long type; char fifo_response[20]; } request;

int main(int argc, char *argv[]) {
    int pid, fd, fdc, ret;
    request r;
    char response[20];

    printf("Selezionare un carattere alfabetico minuscolo: ");
    scanf("%s", r.fifo_response);
    if (r.fifo_response[0] > 'z' || r.fifo_response[0] < 'a') {
        printf("carattere selezionato non valido, ricominciare operazione\n");
        exit(1);
    }
    r.fifo_response[1] = '\0';
    ret = mkfifo(r.fifo_response, 0666);                /* 2 */
    if (ret == -1) {
        printf("\n servente sovraccarico - riprovare \n");
        exit(1);
    }

    fd = open("/serv", O_WRONLY);
    if (fd == -1) {
        printf("\n servizio non disponibile \n");
        ret = unlink(r.fifo_response);
        exit(1);
    }

    write(fd, &r, sizeof(request));                     /* 2.1 */
    close(fd);

    fdc = open(r.fifo_response, O_RDONLY);
    read(fdc, response, 20);                            /* 2.2 */
    printf("risposta = %s\n", response);

    close(fdc);
    unlink(r.fifo_response);
}
```

FIFO: Unix vs Windows

Le named-pipe (FIFO) non sono implementate allo stesso modo in tutti i sistemi: Unix e Windows (Win32) le gestiscono in modo diverso, e questo si riflette sulle loro caratteristiche.

Unix:
- Dette FIFO e create con **mkfifo()**, ma sono normali file del file system.
- Half-duplex (i dati viaggiano in un'unica direzione alla volta).
- I programmi comunicanti devono risiedere nella stessa macchina (in alternativa, occorrono i socket).
- Dati byte-oriented.

Win32:
- Meccanismo più ricco: **createNamedPipe()** (nuova pipe) e **ConnectNamedPipe()** (pipe già esistente).

- Full-duplex (i dati viaggiano contemporaneamente in entrambe le direzioni).
- I programmi comunicanti possono risiedere in macchine diverse (ambiente client/server).
- Dati byte-oriented e message-oriented.

Segnali

Nei sistemi UNIX-like, per notificare a un processo il verificarsi di un predeterminato evento si usano i segnali, che possono essere:
- **Sincroni**: sono inviati allo stesso processo che ha causato la generazione del segnale (es. divisione per 0, accesso illegale alla memoria);
- **Asincroni**: sono inviati a un processo differente da quello che ha causato la generazione del segnale (es. intercettazione di una combinazione di tasti (ctrl+c, ctrl+alt+canc), scadenza di un timer, etc).

Permettere ai processi di comunicare fra di loro (es. **Ctrl+C** da tastiera causa un **SIGINT**, **Ctrl+Z** genera un **SIGTSTP**. Viceversa, **Ctrl+D** invia un **EOF** allo **stdin**, senza far generare segnali).
I segnali **SIGKILL** e **SIGSTOP** non possono essere catturati, bloccati o ignorati.

Signal	Value	Action	Commento
SIGHUP	1	Term	Hangup rilevato sul terminale di controllo o morte del processo di controllo
SIGINT	2	Term	Interruzione da tastiera
SIGQUIT	3	Core	Uscita da tastiera
SIGILL	4	Core	Istruzione illegale
SIGABRT	6	Core	Segnale di abort da `abort(3)`
SIGFPE	8	Core	Eccezione in virgola mobile
SIGKILL	9	Term	Segnale di kill
SIGSEGV	11	Core	Riferimento di memoria non valido
SIGPIPE	13	Term	Broken pipe: scrittura su una pipe senza lettori
SIGALRM	14	Term	Segnale del timer da `alarm(2)`
SIGTERM	15	Term	Segnale di terminazione
SIGUSR1	30,10,16	Term	Segnale 1 definito dall'utente
SIGUSR2	31,12,17	Term	Segnale 2 definito dall'utente
SIGCHLD	20,17,18	Ign	Figlio fermato o terminato
SIGCONT	19,18,25	Cont	Continua se fermato
SIGSTOP	17,19,23	Stop	Ferma il processo
SIGTSTP	18,20,24	Stop	Stop digitato al terminale
SIGTTIN	21,21,26	Stop	Input da terminale per un processo in background
SIGTTOU	22,22,27	Stop	Output su terminale per un processo in background

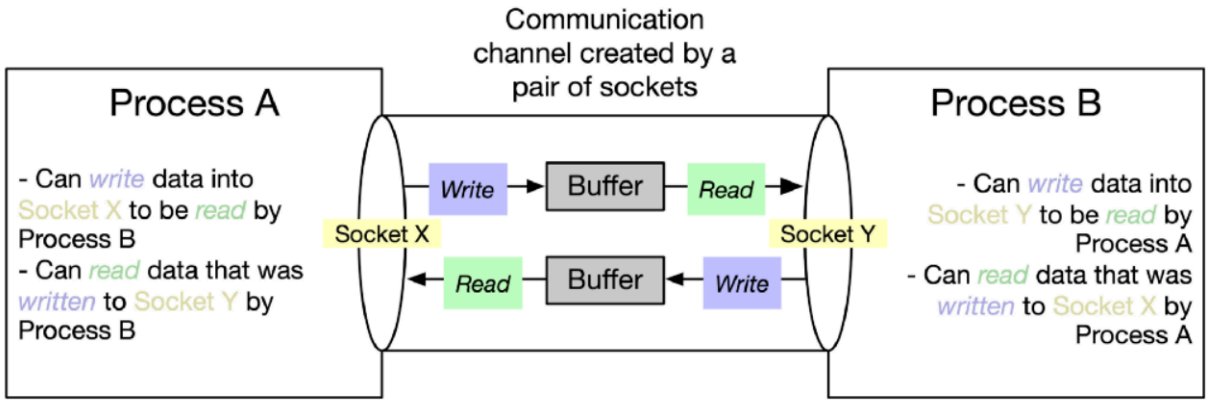
Ogni processo può gestire esplicitamente un segnale utilizzando la system call **signal()**:

sighandler_t signal(int signum, sighandler_t handler);

- **signum**: l'intero (o il nome simbolico) che individua il segnale da gestire.
- **handler**: puntatore a una funzione che indica l'azione da associare al segnale. In particolare **func** può:
  - puntare alla routine di gestione dell'interruzione (handler);
  - valere **SIG_IGN** (nel caso di segnale ignorato);
  - valere **SIG_DFL** (nel caso di azione di default).
- Ritorna un puntatore a funzione:
  - al precedente gestore del segnale;
  - **SIG_ERR** (-1), nel caso di errore.

Socket

Un **socket** è l'estremo di un canale di comunicazione: due processi comunicano attraverso una coppia di socket, uno per ciascun processo, collegati dal canale.

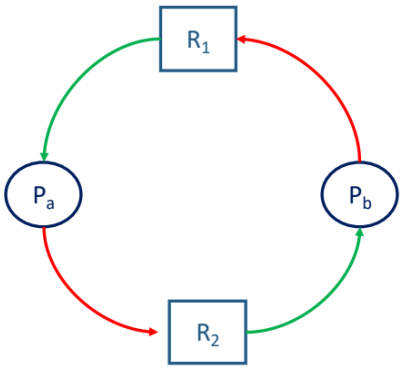


Tutte le **connessioni devono essere uniche**: se un processo vuole n connessioni verso un server, deve creare n socket, uno per ciascuna connessione.
Usare i socket ha pregi e difetti:
- **Vantaggi**: sono semplici ed efficienti;
- **Svantaggi**: essendo di basso livello, permettono solo la trasmissione di un flusso non strutturato di byte. Per questo sono Client e Server a doversi occupare di interpretare e organizzare i dati in forme più complesse.

Deadlock

Lo stallo (**deadlock**) è una situazione derivante dalla necessità di utilizzare 2 o più risorse da parte di 2 o più processi. E' il **blocco permanente** di un insieme di processi che competono per le risorse di sistema oppure comunicano tra loro.
Un gruppo di processi entra in stallo quando **ogni processo** del gruppo è **bloccato in attesa** del rilascio di una risorsa, o di un evento, che può essere liberato o generato solo da **un altro processo bloccato** dallo stesso gruppo.

Esempio. Con due processi (P_a e P_b) e due risorse (R_1 e R_2):
- il processo P_a ottiene il possesso della risorsa R_1;
- il processo P_b ottiene il possesso della risorsa R_2;
- il processo P_a richiede la risorsa R_2;
- il processo P_b richiede la risorsa R_1.



Lo stallo è **permanente** e non esiste una soluzione **efficiente**.

Condizioni necessarie

Si ha deadlock se si verificano simultaneamente le seguenti condizioni:
- **Mutua esclusione** (*Mutual Exclusion*); almeno una risorsa deve poter essere acceduta da un solo processo alla volta (gli altri vengono messi in attesa), cioè solo un processo alla volta può usare una risorsa;
- **Possesso e attesa** (*Hold-and-Wait*): un processo possiede una risorsa ed è in attesa per un'altra risorsa, cioè può mantenere le risorse allocate mentre attende l'assegnazione di altre;
- **Non-preemptive** (*No Pre-emption*): una risorsa posseduta da un processo non può essere rilasciata se non per spontanea volontà del processo stesso, cioè nessuna risorsa può essere sottratta con la forza a un processo che la detiene;
- **Attesa circolare** (*Circular Wait*): $\{P_0, P_1, \ldots, P_N\}$, P_0 attende una risorsa posseduta da P_1, P_1 attende una risorsa posseduta da P_2, …, P_N attende una risorsa posseduta da P_0; cioè esiste una catena chiusa di processi tali che ogni processo detiene almeno una risorsa necessaria al processo successivo nella catena.

Le prime tre condizioni (mutua esclusione, possesso e attesa, non-preemptive) sono **necessarie ma non sufficienti**.

Categorie di risorse (Resource Categories)

Risorsa riutilizzabile (*Reusable resource*):
- Può essere usata in sicurezza da un solo processo alla volta e non si esaurisce con quell'uso.
- Esempi: processi, canali di I/O, memoria principale e secondaria, dispositivi e strutture come file, database e semafori.

Risorsa consumabile (*Consumable resource*):
- Una risorsa che può essere create (prodotta) e distrutta (consumata).
- Esempi: interrupt, segnali, messaggi e informazioni nei buffer di I/O.

Risorse riusabili

Un processo segue un preciso schema per utilizzare una risorsa:
- richiesta di uso della risorsa;
- uso della risorsa;
- rilascio della risorsa.

La richiesta e il rilascio delle risorse avvengo attraverso chiamate di sistema (es. **acquire()**, **release()**). Il sistema operativo mantiene una tabella che memorizza lo stato delle risorse e l'eventuale processo utilizzatore.

Acquisire le risorse: si utilizza un semaforo per proteggere le risorse. Nel caso caso (a), **una risorsa**, il processo prende il semaforo con **down**, usa la risorsa e lo rilascia con **up**:

```
typedef int semaphore;
semaphore resource_1;

void process_A(void) {
    down(&resource_1);
    use_resource_1();
    up(&resource_1);
}
```

Nel caso (b), **due risorse**, il processo prende i due semafori uno dopo l'altro, usa entrambe le risorse e poi li rilascia in ordine inverso:

```
typedef int semaphore;
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources();
    up(&resource_2);
    up(&resource_1);
}
```

Quando si verifica un deadlock? Con due processi che usano le stesse due risorse, tutto dipende dall'ordine in cui l chiedono:

- (a) **Codice esente da deadlock**: i due processi richiedono le risorse nello stesso ordine.

```
typedef int semaphore;
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources();
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources();
    up(&resource_2);
    up(&resource_1);
}
```

- (b) **Codice con deadlock potenziale**: c'è un ordine invertito di richiesta/rilascio delle risorse fra i 2 processi.

```
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources();
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_2);
    down(&resource_1);
    use_both_resources();
    up(&resource_1);
    up(&resource_2);
}
```

Altri esempi con risorse riusabili

- **Risorse riusabili**: risorse D e T, processi P e Q:

Process P

Step	Action
p_0	Request (D)
p_1	Lock (D)
p_2	Request (T)
p_3	Lock (T)
p_4	Perform function
p_5	Unlock (D)
p_6	Unlock (T)

Process Q

Step	Action
q_0	Request (T)
q_1	Lock (T)
q_2	Request (D)
q_3	Lock (D)
q_4	Perform function
q_5	Unlock (T)
q_6	Unlock (D)

Il deadlock si verifica se P blocca D e Q blocca T, prima che P possa bloccare anche T.

- **Risorsa memoria**: lo spazio è disponibile per l'allocazione di 200 Kbyte, e si verifica la seguente sequenza di eventi:

```
P1
...
Request 80 Kbytes;
...
Request 60 Kbytes;
```

```
P2
...
Request 70 Kbytes;
...
Request 80 Kbytes;
```

Il deadlock si verifica se entrambi i processi arrivano alla loro seconda richiesta.

Risorse consumabili

Si consideri una coppia di processi (P1,P2), in cui ciascun processo tenta di ricevere un messaggio dall'altro processo e poi invia un messaggio all'altro processo:

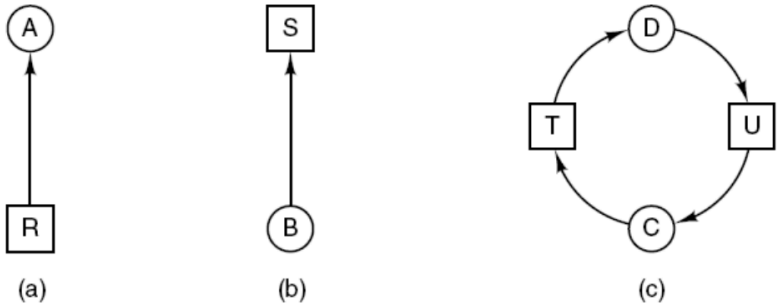
P1	P2
...	...
Receive (P2);	Receive (P1);
...	...
Send (P2, M1);	Send (P1, M2);

Il deadlock si verifica se la Receive è bloccante.

Modellizzazione del Deadlock: Resource Allocation Graph

Il **Resource Allocation Graph** è un grafo orientato che rappresenta lo stato del sistema di risorse e processi, con ogni processo e ogni risorsa rappresentati da un nodo.

Nella versione semplificata gli risorsa ha **una sola istanza**: o è libera o è posseduta da un solo processo. Una freccia dalla risorsa al processo vuol dire che il processo **possiede**, mentre una freccia dal processo alla risorsa vuol dire che la **richiede** e deve aspettare che si liberi.

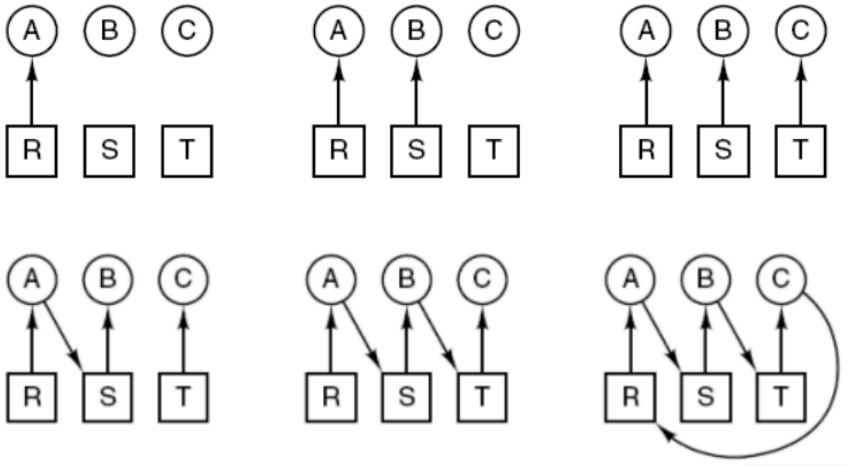


In questo esempio:
- A possiede la risorsa R
- B richiede la risorsa S
- D possiede la risorsa T e richiede la risorsa U
- C possiede la risorsa U e richiede la risorsa T

Come possiamo notare nella figura (c) abbiamo un caso di **Deadlock** (**circular wait**).

Esempio 1. Svolgimento lineare (Deadlock)

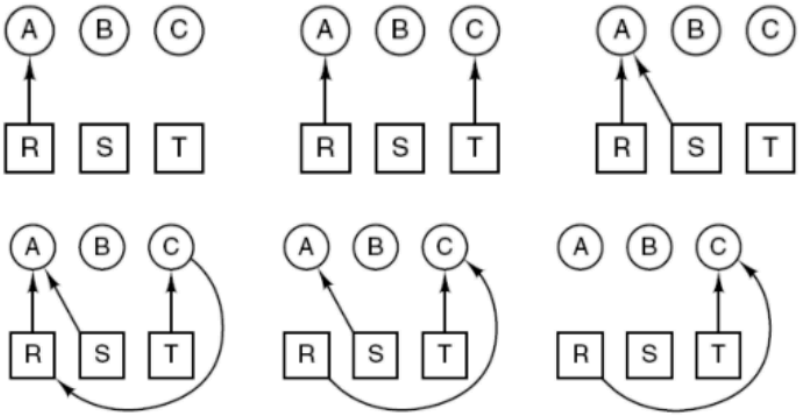
A	B	C
Request R	Request S	Request T
Request S	Request T	Request R
Release R	Release S	Release T
Release S	Release T	Release R



1. A possiede R
2. B possiede S
3. C possiede T
4. A richiede S
5. B richiede T
6. C richiede R → deadlock

Esempio 2. Processo inibito (Deadlock evitato)

A	B	C
Request R	Request S	Request T
Request S	Request T	Request R
Release R	Release S	Release T
Release S	Release T	Release R



1. A possiede R
2. C possiede T
3. A possiede S
4. C richiede R
5. A rilascia R
6. A rilascia S → no deadlock

Il **Resource Allocation Graph** completo viene descritto da:

Grafo $G(V, E)$:
- Insieme di nodi V
- Insieme di archi E

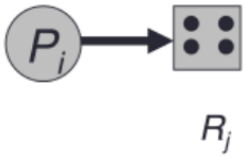
Nodi:
- **Processi** del sistema: $P = \{P_1, P_2, \ldots, P_n\}$ (in figura la rappresentazione di un singolo processo)



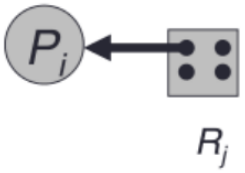
- **Risorse** del sistema: $R = \{R_1, R_2, \ldots, R_m\}$ eventualmente con più istanze identiche (es. in figura abbiamo un tipo di risorsa con 4 istanza)



Archi:
- **Arco di richiesta**: da processo a risorsa, $P_i \rightarrow R_j$ (si noti che raggiunge la risorsa)

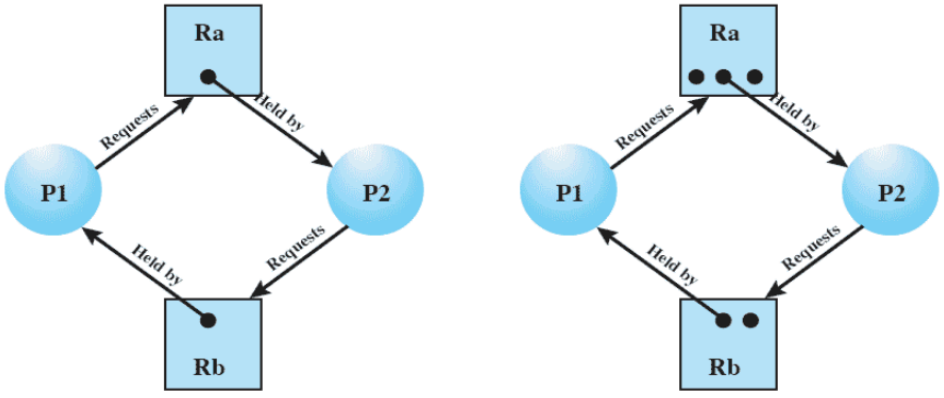


- **Arco di assegnazione**: da risorsa a processo, $R_j \leftarrow P_i$

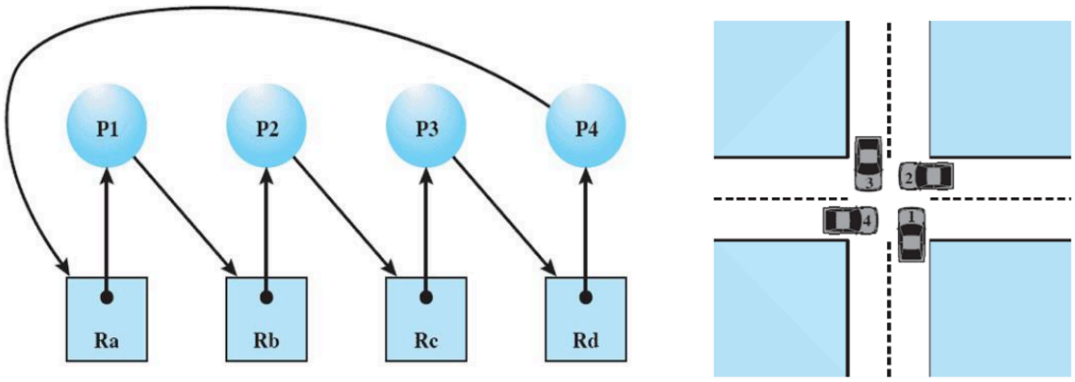


Significato degli archi:
- Un arco diretto da un processo a una risorsa indica una risorsa **richiesta** dal processo ma non ancora concessa.
- All'interno di un nodo risorsa, un **punto** indica ciascuna **istanza** di quella risorsa.
- Un arco diretto dal punto di un nodo risorsa **riusabile** a un processo indica una richiesta che è stata **concessa**: al processo è stata assegnata un'unità di quella risorsa.
- Un arco diretto dal punto di un nodo di risorsa **consumabile** a un processo indica che il processo è il **produttore** di quella risorsa.

Esempio.



Nella figura (c) c'è una sola unità di ciascuna risorsa Ra e Rb. Il processo P1 possiede Rb e richiede Ra, mentre P2 possiede Ra ma richiede Rb.
Nella figura (d), invece, non c'è deadlock, perché sono disponibili più unità di ciascuna risorsa.



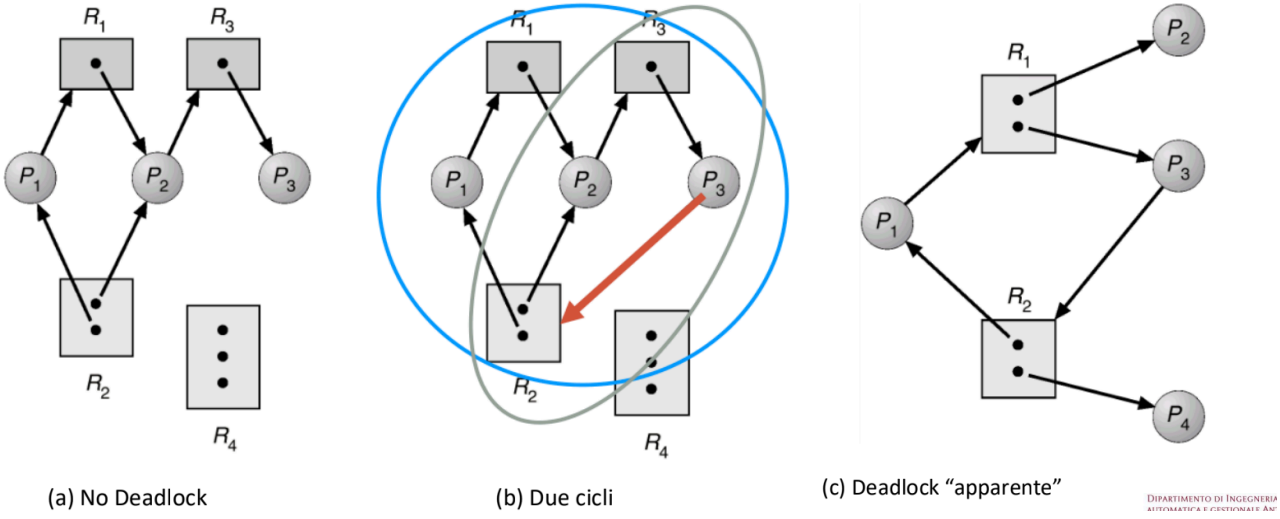
Nelle seguenti figure abbiamo un caso con quattro processi. Il grafo di allocazione delle risorse corrisponde alla situazione di deadlock “incrocio con 4 auto”. SI noti che in questo caso non abbiamo la situazione semplice in cui due processi hanno ciascuno una risorsa di cui l’altro ha bisogno. C’è invece una **catena circolare di processi e risorse** che genera il deadlock.

Cicli nel grado e deadlock

Se il grafo non contiene cicli ⇒ nessun deadlock.

Se il grafo contiene cicli ⇒
- (a) se ogni tipo di risorsa ha **più di un’istanza**, allora si ha una **possibilità** di **deadlock** (**condizione necessaria ma non sufficiente**);
- (b) se ogni tipo di risorsa **inclusa nel ciclo** ha **una sola istanza**, allora si ha un **deadlock** (**condizione necessaria e sufficiente**.

Esempio.



Nella figura (c):
- P_4 prima o poi cederà il possesso di R_2;
- R_2 potrà quindi essere acquisita da P_3;
- non siamo un una condizione di deadlock.

Deadlock: metodi di gestione

- **Prevent** (prevenire il deadlock - **deadlock prevention**): evitare che si verifichino tutte e quattro le condizioni necessarie. Descrivono come le risorse devono essere richieste.
- **Avoid** (evitare il deadlock - **deadlock avoidance**): il sistema operativo conosce in anticipo quali risorse un processo utilizzerà. In base a queste informazioni decide se tutte le richieste possono essere accettate senza causare stalli.
- **Detect 6 Recovery**: rilevare e recuperare il deadlock ed eseguire le azioni necessarie per il ripristino.
- **Ignore** (ignorare il deadlock): giustificato dalla scarsa occorrenza di questa evenienza. Richiede intervento manuale dell’utente (es. riavvio del sistema).

Nessun metodo è ottimale, meglio una loro combinazione a seconda della “classe di risorsa”.

Prevent

Prevenire il deadlock (deadlock prevention): evitare che si verificano tutte e e quattro le condizioni necessarie.

Condition	Approach
Mutual exclusion	Spool everything
Hold and wait	Request all resources initially
No preemption	Take resources away
Circular wait	Order resources numerically

Le condizioni:
- **Mutua esclusione**: almeno una risorsa deve poter essere acceduta da un solo processo alla volta (gli altri vengono messi in attesa).
- **Possesso e attesa**: un processo possiede una risorsa ed è in attesa per un'altra risorsa.
- **Non-preemptive**: una risorsa posseduta da un processo non può essere rilasciata se non per spontanea volontà del processo stesso.
- **Attesa circolare**: $\{P_0, P_1, \ldots, P_n\}$, P_0 attende una risorsa posseduta da P_1, P_1 attende una risorsa posseduta da P_2, …, P_n attende una risorsa posseduta da P_0.

Le prime tre sono condizioni **indirette**, l'attesa circolare è **diretta**.

L'obiettivo è quello di progettare un sistema in modo tale che la possibilità di deadlock sia esclusa.

Mutuale exclusion (mutua esclusione)
In generale, la prima delle quattro condizioni elencate non può essere negato. Se l'accesso a una risorsa richiede la mutua esecuzione, allora questa deve essere supportata dal sistema operativo. Alcune risorse, come i file, possono consente accessi multipli in lettura ma solo acceso esclusivo in scrittura. Anche in questo caso il deadlock può verificarsi se più di un processo richiede il permesso di scrittura.

Hold and wait (possesso e attesa)
La condizione di hold-and-wait può essere prevenuta richiedendo che un processo richieda tutte le risorse di cui ha bisogno in una sola volta e bloccando il processo finché tutte le richieste non possono essere soddisfatte contemporaneamente.

Questo approccio è **inefficiente in due modi**:
1. un processo può restare bloccato a lungo in attesa anche tutte le sue richieste siano soddisfatte, quando in realtà avrebbe potuto procedere con solo alcune delle risorse;
2. le risorse allocate a un processo possono restare inutilizzate per un periodo considerevole, durante il quale vengono negate agli altri processi.

Inoltre:
3. un processo può non conoscere in anticipo tutte le risorse di cui avrà bisogno.

Non-preemptive (no preemption)
Questa condizione può essere prevenuta in diversi modi:
- se un processo che possiede alcune risorse viene negata un'ulteriore richiesta, quel processo deve rilasciare le sue risorse originali e, se necessario, richiederle di nuovo insieme alla risorsa aggiuntiva;
- in alternativa, se un processo richiede una risorsa attualmente posseduta da un altro processo, il sistema operativo può sottrarre la risorsa (preempt) al secondo processo e imporgli di rilasciare le sue risorse. Questo secondo schema previene il deadlock solo se non esistono due processi con la stessa priorità.

Questo approccio è praticabile solo se applicato a risorse il cui stato può essere facilmente salvato e ripristinato in seguito, come nel caso di un processore.
- **OK**: per risorse il cui stato può essere facilmente salvato e ricaricato (es. registri CPU e memoria centrale).
- **NOK**: non va bene, ad esempio, per nastri o stampanti, burning (masterizzazione).

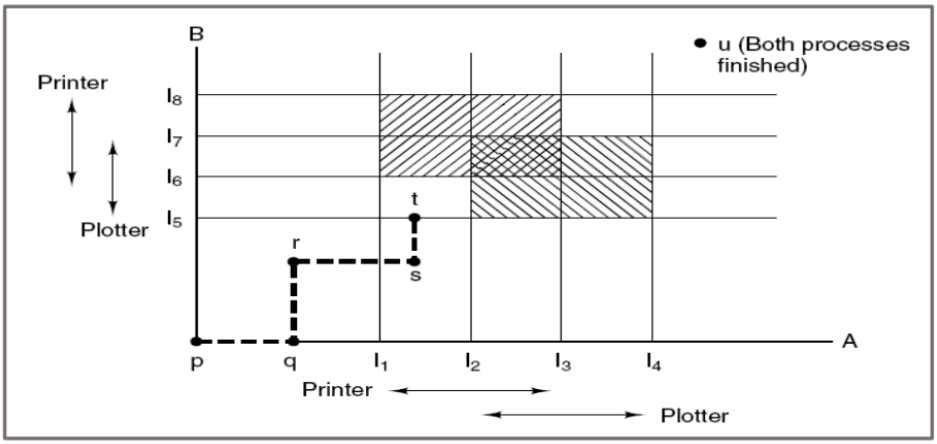
Circular wait (attesa circolare)
- Definire un ordinamento lineare dei tipi i risorsa.
- Se una risorsa di tipo R è stata assegnata a un processo, quest'ultimo può richiedere solo risorse i cui tipi seguono R nell'ordine.
- Stessi problemi di hold and wait.

Un ordinamento globale univoco viene imposto su tutti i tipo di risorsa R_i:
- si implementa tramite una funzione $f(R_i) = n$;
- se un processo chiede k istanze della risorsa R_j e detiene solo risorse R_i con $i < j$:
  - se e k istanze della risorsa R_j sono disponibili, gli vengono assegnate;
  - altrimenti il processo deve attendere.
- un processo non potrà mai chiedere istanze della risorsa R_j se detiene R_i con $i \geq j$.

Avoid

- **Consente le tre condizioni necessarie** (mutua esclusione, hold & wait, no preemption), ma compie **scelte oculate** per assicurare che il punto di deadlock non venga mai raggiunto.
- Viene presa **dinamicamente** una decisione: se la richiesta corrente di allocazione di risorse, qualora concessa, potrebbe portare a un deadlock.
- **Richiede la conoscenza delle richieste futuri** dei processi.
- **Consente più concorrenza** della deadlock prevention.

Esempio.



Modello per gestire due processi e due risorse, ad esempio una stampante e un plotter:
- L'asse orizzontale rappresenta il numero di istruzioni eseguite dal processo A.
- L'asse verticale rappresenta il numero di istruzioni eseguite dal processo B.
- In I, A richiede una stampate; a I_2 ha bisogno di un plotter.
- La stampante e il plotter vengono rilasciati rispettivamente in I_3 e I_4.
- Il processo B necessita del plotter da I_5 a I_7 e della stampante da I_6 a I_8.

Ogni punto del **grafo di allocazione** rappresenta uno **stato congiunto** dei due processi:
- **p**: nessun processo eseguito
- **q**: A eseguito, B comincia
- **r**: A ricomincia, in I_1chiede il plotter…
- …

Esistono due approcci alla deadlock avoidance:
- **Process Initiation Denial**: non avviare un processo se le sue richieste potrebbero portare a deadlock → **Prevent**.

- **Resource Allocation Denial**: non concedere a un processo una richiesta incrementale di risorse se questa allocazione potrebbe portare a deadlock → **Avoid**.

Process Initiation Denial
Evitare il deadlock (deadlock avoidance): verificare a priori se la sequenza di richieste e rilasci effettuate da n processo porta al deadlock, tenendo conto delle **risorse già assegnate** ai **processi già accettati** nel sistema.

Obiettivi:
- alto sfruttamento delle risorse;
- alta efficienza del sistema;
- semplicità di gestione.

Necessità di **informazioni a priori** sullo stato di allocazione delle risorse e sul comportamento dei processi:
- **E** (Existing): numero massimo di risorse per ogni processo;
- **A** (Available): risorse disponibili;
- **P** (Possessed): risorse assegnate;
- richieste e rilasci futuri di risorse (Claim).

E, A, P formano lo **stato di allocazione** (E, A, P) per ogni processo.

Esempio.

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

R1	R2	R3
9	3	6

Resource vector **R**

Il sistema operativo consente l'esecuzione di un nuovo processo solo se la somma delle risorse dichiarate (claim) dei processi in esecuzione più quelle del nuovo processo è minore delle risorse disponibili.

Se P1 e P2 sono in esecuzione: $A[0\ 0\ 1]$
- → P3 non può essere seguito: $A[-3\ -1\ -3]$
- → P4 non può essere seguito: $A[-4\ -2\ -1]$

Algoritmi
1. Un algoritmo generico per evitare i deadlock esamina dinamicamente lo stato di allocazione delle risorse per accertarsi che la condizione di attesa circolare non possa mai verificarsi:
   - → richiedere a ogni processo di dichiarare il numero massimo di risorse che userà per ogni tipo;
   - → e iniziare ad assegnare le risorse solo se la richiesta complessiva non porterà allo stallo.
2. **Algoritmo del banchiere** per singolo tipo di risorsa [Dijkstra, 1965]
3. **Algoritmo del banchiere** per risorse multiple [Habermann, 1969]

Questi algoritmi devono garantire che il sistema passi da uno stato sicuro a un altro stato sicuro quando un processo chiede una nuova risorsa.

Stato sicuro (sai state): se esiste una **sequenza di allocazione** delle risorse che **non** porta a un **deadlock**.

Pianificazione della esecuzione
Stato sicuro: uno stato si dice **sicuro** se esiste un **ordine di pianificazione** in cui **ogni processo** può essere **eseguito** fino al completamento anche se tutti i processi in esecuzione richiedessero immediatamente il numero massimo di risorse.

Esempio. 3 processi (A, B, C) 10 risorse

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3
(a)

	Has	Max
A	3	9
B	4	4
C	2	7

Free: 1
(b)

	Has	Max
A	3	9
B	0	–
C	2	7

Free: 5
(c)

	Has	Max
A	3	9
B	0	–
C	7	7

Free: 0
(d)

	Has	Max
A	3	9
B	0	–
C	0	–

Free: 7
(e)

Lo stato è sicuro perché esiste una sequenza possibile di allocazioni che consente il completamento di tutti i processi (B, C, A):

- B → (b), (c);
- C → (d), (e);
- A → (e++) può ora ottenere tutte le numerose risorse di cui necessita.

Avoid: Algoritmo del banchiere

Algoritmo del Banchiere [Dijkstra, 1965]: estensione dell'algoritmo di deadlock detestino.

Il banchiere (sistema operativo) sa che non tutti i clienti (processi) avranno bisogno del loro credito (risorse) massimo immediatamente, quindi riserva un numero di risorse inferiori a quelle effettivamente presenti sul sistema.

Esempio.

	Has	Max
A	0	6
B	0	5
C	0	4
D	0	7

Free: 10
(a)

	Has	Max
A	1	6
B	1	5
C	2	4
D	4	7

Free: 2
(b)

	Has	Max
A	1	6
B	2	5
C	2	4
D	4	7

Free: 1
(c)

- (a) Safe;
- (b) Safe;
- (c) **Unsafe** (→ stato da evitare): deadlock non sicuro ma possibile. Un processo potrebbe avere bisogno dell'intero insieme di risorse (intera linea di credito disponibile): se **ciascuno dei clienti** chiedesse **improvvisamente** i loro **prestiti massimi**, il banchiere non potrebbe soddisfare nessuno di loro e ci troveremmo in una situazione di **stallo** (non si può allocare le risorse a B).

	Has	Max
A	0	6
B	0	5
C	0	4
D	0	7

Free: 10
(a)

	Has	Max
A	1	6
B	1	5
C	2	4
D	4	7

Free: 2
(b)

	Has	Max
A	1	6
B	1	5
C	4	4
D	4	7

Free: 0
(c)

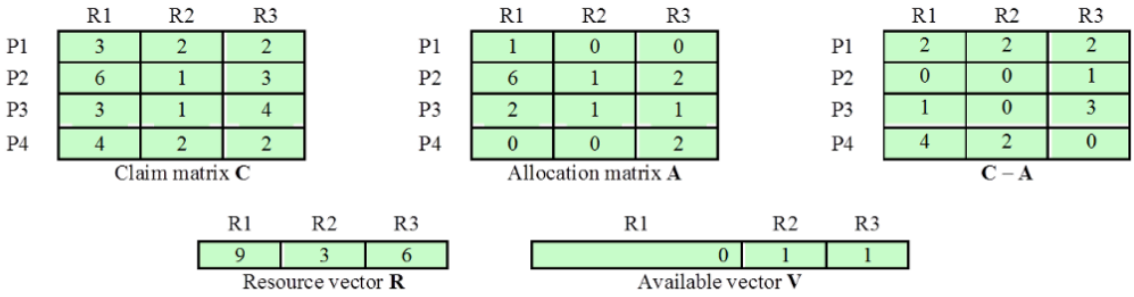
- (a) Safe;

- (b) Safe;
- (c) Safe: C satura tutte le risorse ma poi le libera.

Algoritmo del Banchiere [Habermann, 1969]: necessità di informazioni a priori sullo stato di allocazione delle risorse e sul comportamento dei processi.

Stato di allocazione:
- **tre matrici**:
  - **C** (Claim): risorse richieste per l'esecuzione;
  - **A** (Allocation): risorse correntemente assegnate;
  - **C-A**: risorse ancora necessarie per il completamento. Viene desunto come C-A.
- **due vettori**:
  - **R** (Resource): ammontare delle risorse attualmente esistenti;
  - **V** (Vector): ammontare delle risorse attualmente non allocate. Può essere desunto come $R - \Sigma(A)$.

Stato di un sistema composto da quattro processi e tre risorse (stato iniziale (a))

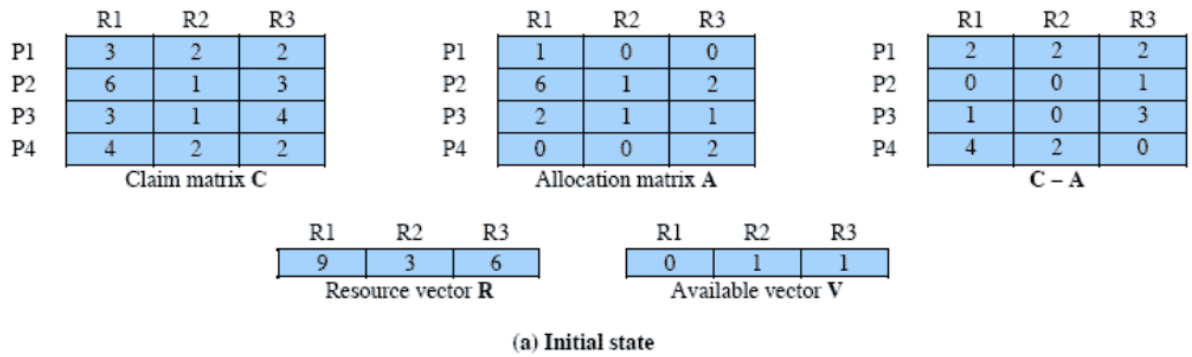


(a) Initial state

Algoritmo di verifica della sicurezza dello stato:
1. Cerca in **C-A** una riga (P1, P2, P3, P4) le cui esigenze di risorse non soddisfatte sono tutte **inferiori** o uguali a **V**. Se tale riga non esiste, il sistema alla fine andrà in stallo poiché nessun processo può essere completato (assumendo che i processi mantengano tutte le risorse fino alla loro uscita). Nel nostro caso, P2 soddisfa.
2. Supponiamo che il processo della riga scelta (P2) richieda tutte le risorse di cui ha bisogno)il che è garantito possibile) e termini. Contrassegna quel processo come terminato e aggiungi tutte le sue risorse al vettore **V**.
3. Ripetere i passaggi 1 e 2 fino a quando tutti i processi sono contrassegnati come terminati (nel qual caso lo stato iniziale era sicuro) o non è rimasto alcun processo le cui esigenze di risorse possono essere soddisfatte (nel qual caso il sistema non era sicuro).

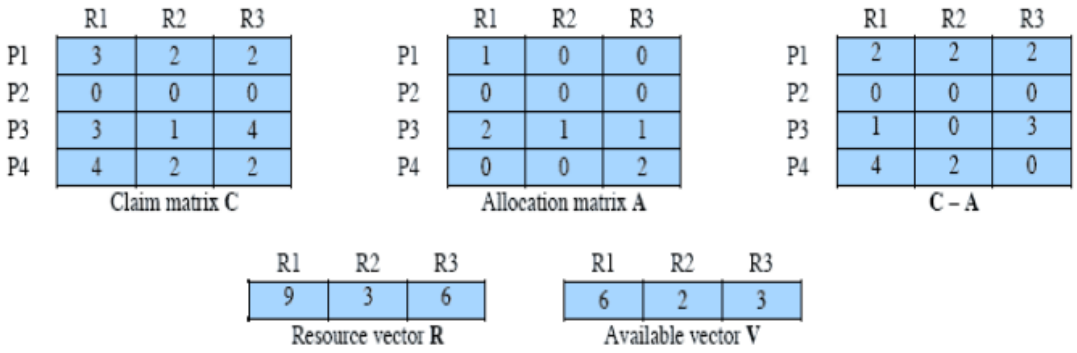
Esempio. Stato Sicuro

Sequenza di allocazione delle risorse: P2-P1-P3-P4



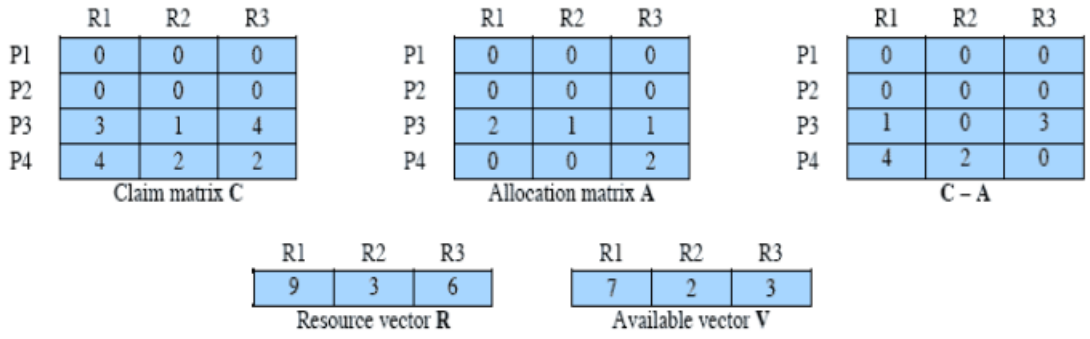
(a) Initial state

(b) P2 viene eseguito fino al completamento



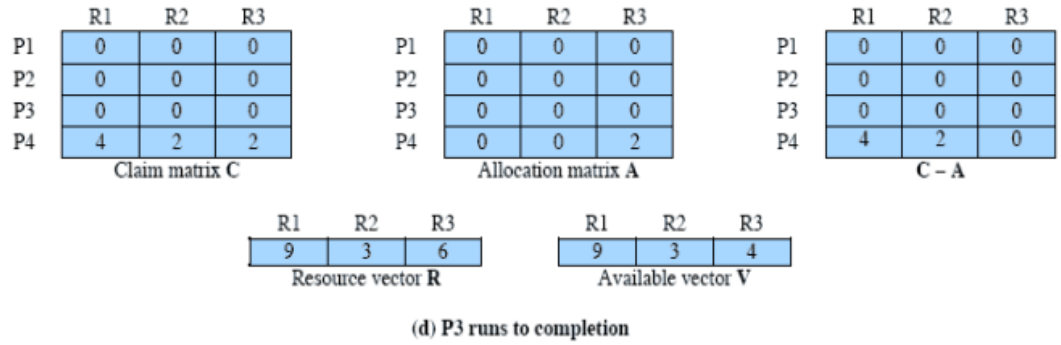
(b) P2 runs to completion

(c) P1 vene eseguito fino al completamento



(c) P1 runs to completion

(d) P3 viene eseguito fino al completamento

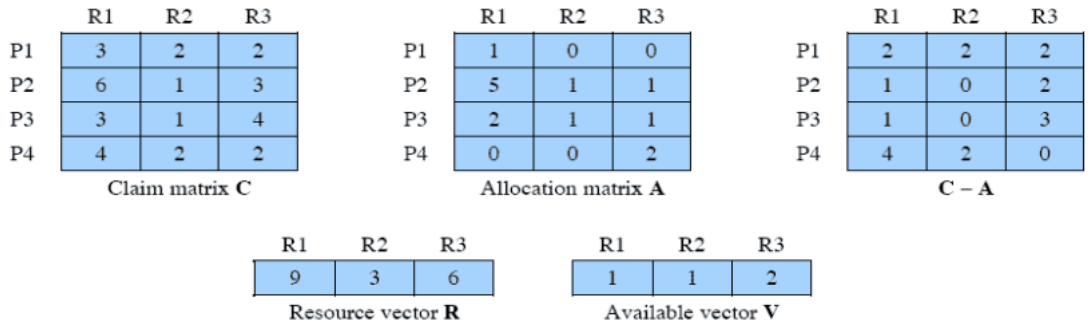


(d) P3 runs to completion

Quindi lo stato definito originariamente è uno **stato sicuro**.

Esempio. Stato non Sicuro

(a) Stato iniziale



	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	1	0	0
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	2	2	2
P2	1	0	2
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

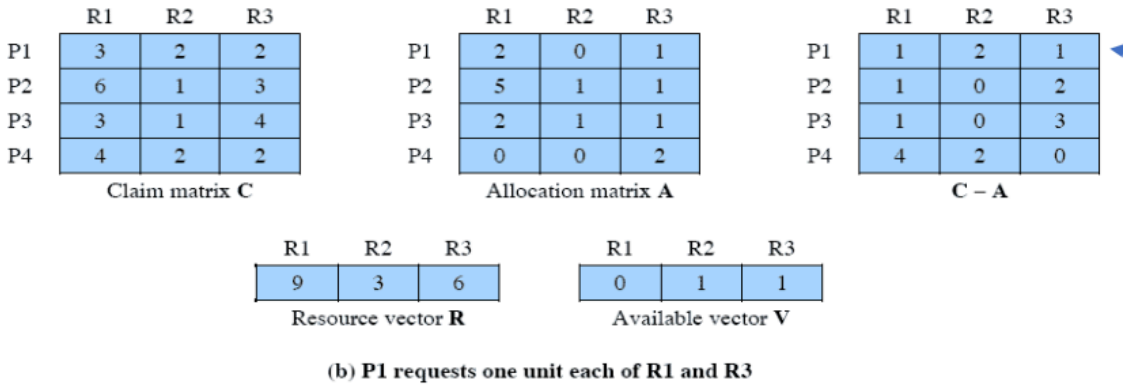
Resource vector **R**

R1	R2	R3
1	1	2

Available vector **V**

(a) Initial state

(b) P1 richiede un'unità ciascuna di R1 e R3



	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	2	0	1
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	1	2	1
P2	1	0	2
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
0	1	1

Available vector **V**

(b) P1 requests one unit each of R1 and R3

P1 è ancora in attesa di… che è maggiore di V(R1,R2) → quindi lo stato (a) è **non sicuro**.

Logica dell'algoritmo: L'algoritmo principale è mostrato nella parte (b). Con lo stato del sistema definito dalla struttura dati **state**, **request[*]** è un vettore che definisce le risorse richieste dal processi i:

1. Viene effettuato un **controllo** per assicurare che la richiesta **non superi** il claim originale del processo.
2. Si determina se è **possibile soddisfare** la richiesta (cioè se ci sono risorse sufficienti disponibili). Se non è possibile, il processo viene sospeso.
3. Se è possibile, l'ultimo passo è determinare se è sicuro soddisfare la richiesta. Per farlo, le risorse vengono **assegnate in via provvisoria** al processo i per formare newstate.

(a) Struttura dati globali

```
struct state {
    int resource[m];
    int available[m];
    int claim[n][m];
    int alloc[n][m];
}
```

(b) Algoritmo di allocazione delle risorse

```
if (alloc [i,*] + request [*] > claim [i,*])
    < error >;                              /* total request > claim*/
else if (request [*] > available [*])
    < suspend process >;
else {                                      /* simulate alloc */
    < define newstate by:
    alloc [i,*] = alloc [i,*] + request [*];
    available [*] = available [*] - request [*] >;
}
if (safe (newstate))
    < carry out allocation >;
else {
    < restore original state >;
    < suspend process >;
}
```

(c) Test di sicurezza

```
boolean safe (state S) {
    int currentavail[m];
    process rest[<number of processes>];
    currentavail = available;
    rest = {all processes};
    possible = true;
    while (possible) {
        <find a process Pk in rest such that
            claim [k,*] - alloc [k,*] <= currentavail;>
        if (found) {                        /* simulate execution of Pk */
            currentavail = currentavail + alloc [k,*];
            rest = rest - {Pk};
        }
        else possible = false;
    }
    return (rest == null);
}
```

Un test di sicurezza viene eseguito a ogni scelta del processo.

Avoid: Pro e contro

Vantaggi: Il process initiation denial è tutt'alto che ottimale… mentre il resource allocation denial ha alcuni vantaggi:

- Non è necessario sottrarre risorse (preempt) e fare rollback dei processi, come nella deadlock detestino;
- è meno restrittivo della deadlock prevention.

Restrizioni:

- Il requisito massimo di risorse per ogni processo deve essere dichiarato in anticipo,
- I processi considerati devono essere indipendenti e senza requisiti di sincronizzazione.
- Deve esserci un numero fisso di risorse da allocare.
- Nessun processo può terminare mentre detiene risorse.

Detect & Recovery

- Le strategie di **deadlock prevention** sono molto conservative: limitano l'accesso alle risorse imponendo restrizioni ai processi.
- Le strategie di **deadlock detection** fanno l'opposto: le richieste di risorse vengono concesse ogni volta che è possibile.

Definizione:
- **Detect**: rilevare la presenza di situazioni di deadlock dopo che sono avvenute:
  - Rilevazione con **istanze singole** delle risorse.
  - Rilevazione con **istanze multiple** delle risorse.
- **Recovery**: ripristinare una situazione di corretto funzionamento eliminando il deadlock:
  - **Killing**: terminazione di alcuni processi coinvolti nel deadlock.
  - **Rollback**: annullare le esecuzioni effettuate da alcuni processi coinvolti nel deadlock (utilizzo dei checkpoint).
  - **Preemption**: prelazione delle risorse.

Detection Algorithm
Algoritmo che **presuppone lo stato peggiore**: tutti i **processi** conservano **tutte le risorse acquisite** fino alla loro uscita. Effettua l'analisi dei processi e, in base alle elaborazioni, eventualmente gli assegna una marcature. Man mano che l'algoritmo progredisce, i processi verranno contrassegnati, indicando così che sono **in grado di completarsi** e quindi non sono in deadlock.

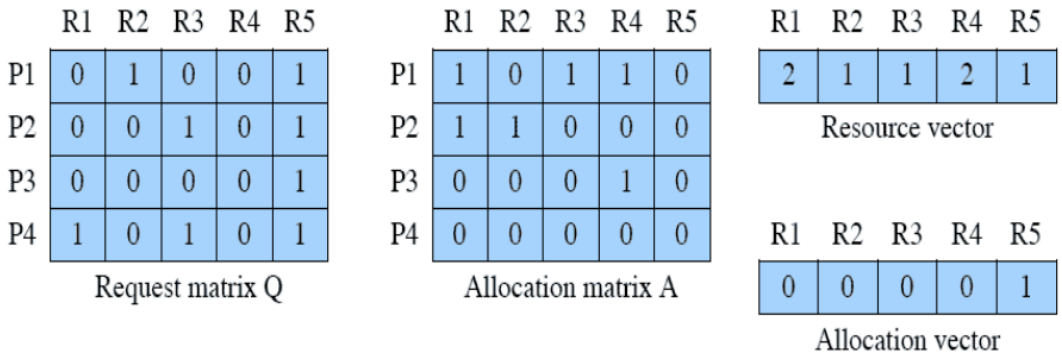
1. Cerca un processo non, P_i, per il quale l'i-esima riga di Q (Request matrix) è minore o uguale alle risorse disponibili $R - A$ (Resource vector - Allocation vector).
2. Se viene trovato un tale processo, aggiungere l'i-esima riga di Q ad A, contrassegnate il processo e tornare al passaggio 1.
3. Se non esiste tale processo, l'algoritmo termina.

Quando l'algoritmo termina, è noto che **tutti i processi non contrassegnati sono in deadlock**.

Un controllo del deadlock può essere effettuato frequentemente a ogni richiesta di risorsa oppure, meno frequentemente, a seconda di quanto sia probabile che si verifichi un deadlock. Rilevazione eseguita a ogni richiesta di risorsa:
- **Pro**: porta a una rilevazione precoce, l'algoritmo è semplice;
- **Contro**: i controlli frequenti consumano un tempo di CPU considerevole.

Algoritmo:
- marcare iterativamente i protocolli che possono essere sospesi o terminare;
- se alla fine tutti i protocolli sono marcati, non c'è deadlock.

Esempio.



	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Request matrix Q

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Allocation matrix A

R1	R2	R3	R4	R5
2	1	1	2	1

Resource vector

R1	R2	R3	R4	R5
0	0	0	0	1

Allocation vector

1. P4 viene perché non ha ancora risorse;
2. P3 viene marcato perché, se gli diamo le risorse, può terminare e rilasciare le risorse;
3. Anche se P3 rilascia le risorse, né P1 né P2 possono terminare → **Deadlock**.

Recovery Algorithms
Alcuni approcci, in ordine crescente di sofisticazione:
1. **Abortire tutti i processi in deadlock** (il più comune).
2. **Riportare ogni processo in deadlock a un checkpoint definito in precedenza** e riavviare tutti i processi (richiede un meccanismo di rollback).
3. **Abortire successivamente i processi in deadlock finché il deadlock non esiste più** (implica criteri di selezione, come il costo minimo).
4. **Sottrarre successivamente risorse (preempt) finché il deadlock non esiste più** (richiede un meccanismo di rollback).

Approach Comparison

Approach	Resource Allocation Policy	Different Schemes	Major Advantages	Major Disadvantages
Prevention	Conservative; undercommits resources	Requesting all resources at once	•Works well for processes that perform a single burst of activity •No preemption necessary	•Inefficient •Delays process initiation •Future resource requirements must be known by processes
		Preemption	•Convenient when applied to resources whose state can be saved and restored easily	•Preempts more often than necessary
		Resource ordering	•Feasible to enforce via compile-time checks •Needs no run-time computation since problem is solved in system design	•Disallows incremental resource requests
Avoidance	Midway between that of detection and prevention	Manipulate to find at least one safe path	•No preemption necessary	•Future resource requirements must be known by OS •Processes can be blocked for long periods
Detection	Very liberal; requested resources are granted where possible	Invoke periodically to test for deadlock	•Never delays process initiation •Facilitates online handling	•Inherent preemption losses

Ignore

Algoritmo dello struzzo:
- Applicato dalla maggior parte dei SO odierni.
- E' più economico dei metodi che prevedono di prevenire, evitare o individuare gli stalli.

Concretamente, il sistema operativo **non fa nulla** per prevenire, evitare o rilevare i deadlock. Se un deadlock capita, l'utente deve intervenire a mano, ad esempio riavviando il sistema.

Linux (gestione minimale ed efficiente):
- **User Space**: i processi utente possono andare in deadlock (amministrati dall'utente). Riduzione di:
  - Mutual Exclusion → Spooling;
  - Hold & Wait → massimizzare le risorse accessibili in contemporanea (no lock).
- **Kernel Space**: prevenzione dell'attesa circolare (lock acquisiti in ordine fisso e predeterminato).

Windows: I deadlock richiedono un certo grado di "bad luck", poiché richiedono che un certo numero di eventi accadano simultaneamente:
- **User Space**: i processi utente possono andare in deadlock (amministrati dall'utente).
- **Driver Verifier**: analisi statica del codice sorgente del driver, in modo da identificare eventuali deadlock potenziali.

Sleeping Barber Problem

Barber shop:
- 1 barbiere;
- 1 sedia da barbiere;
- n sedie nella sala d'attesa;
- Se le n sedie son vuote il barbiere si siede sulla sedia da barbiere e dorme;
- Se arriva un cliente lo deve svegliare;
- Ogni cliente addizionale che arriva, mentre il barbiere sta tagliando i capelli:
  - cerca una sedia in sala d'attesa;
  - se sono tutte piene, esce dal negozio.

Lo "Sleeping Barber" descrive un **problema concreto e realistico di concorrenza**:
- Un barbiere serve i clienti uno alla volta (risorsa condivisa).
- Se non ci sono clienti, si addormenta.
- Se arrivano clienti e il barbiere è occupato, alcuni aspettano in sala d'attesa (numero finito di sedie).
- Se non c'è posto, i clienti se ne vanno.

Questa situazione **simula perfettamente la gestione di risorse limitate in un sistema concorrente**: una CPU che serve processi, una stampante condiva, una coda di rete, ecc.

Lo "Sleeping Barber" **richiede contemporaneamente**:
- **mutua esclusione** (il barbiere serve un solo cliente alla volta);
- **sincronizzazione condizionale** (il barbiere dorme se non ci sono clienti, ma deve essere risvegliato quando ne arriva uno);
- **gestione di code e limiti di capacità** (numero finito di sedie nella sala d'attesa → risorsa limitata).

Questo le rende un **caso di studio completo**, più complesso di Producer-Consumer ma meno astratto di Dining Philosophers (che vedremo in seguito).

Caratteristiche:

#	Scopo Didattico	Descrizione
1	modello di sincronizzazione reale	simula perfettamente la gestione di risorse limitate in un sistema concorrente (CPU che serve processi, una stampante condivisa, una coda di rete etc.)
2	sincronizzazione e mutua esclusione bilanciate	Richiede contemporaneamente: Mutua esclusione Sincronizzazione condizionale Gestione di Code e limiti di capacità
3	Variante del Producer-Consumer	sincronizzazione condizionale bidirezionale (consumer-producer)
4	"attesa passiva" e "risveglio" (wait/signal)	evitare l'uso inefficiente della CPU e coordinare processi in modo elegante
5	Applicazioni pratiche	Thread pool server Job queue Sospensione dei processi in attesa di eventi

Lo "Sleeping Barber" può essere visto come una **variante del Producer-Consumer con buffer limitato**:
- il barbiere è il **consumer**;
- i clienti sono i **producers**;
- le sedie nella sala d'attesa sono il **buffer**.

La differenza è che qui il consumer (il barbiere) può dormire (sospendersi in attesa di lavoro) e deve essere **svegliato da un producer** quando arriva una nuova unità di lavoro. Questo introduce

un **aspetto di sincronizzazione condizionale bidirezionale** (il consumer aspetta i producers e viceversa). Inoltre, oltre al buffer, il consumer agisce da 2^ risorsa.

Elemento	Ruolo	Spiegazione
Cliente	**Producer**	"Produce" una richiesta di servizio (un taglio di capelli). Arrivando, **inserisce** questa richiesta nel sistema (occupando una sedia se disponibile).
Barbiere	**Consumer**	"Consuma" la richiesta del cliente, cioè **fornisce il servizio**. Ogni volta che un cliente è pronto, il barbiere **preleva** la richiesta e la soddisfa.
Sedie della sala d'attesa	**Buffer**	Memorizzano le richieste in attesa, come un **buffer a capacità limitata**. Se è pieno, la richiesta (il cliente) viene scartata.

Lo "Sleeping Barber" illustra bene come usare:
- **semafori o variabili di condizione** per far dormire un thread senza busy waiting;
- **signal o notify** per risvegliarlo solo quando serve.

In altre parole, insegna come **evitare l'uso inefficiente della CPU e coordinare processi in modo elegante**.

Possibile soluzione: sono definiti 3 semafori.
- **customers**: clienti in attesa. il cui valore viene copiato nell'intero **waiting** (in modo da poterlo leggere agevolmente);
- **barber**: se il barbiere è in attesa di clienti;
- **mutex**: usata per mutua esclusione.

```
#define CHAIRS 5               /* # chairs for waiting customers */

typedef int semaphore;         /* use your imagination */

semaphore customers = 0;       /* # of customers waiting for service */
semaphore barbers = 0;         /* # of barbers waiting for customers */
semaphore mutex = 1;           /* for mutual exclusion */
int waiting = 0;               /* customers are waiting (not being cut) */
```

barber: procedura eseguita dal barbiere.

```
void barber(void)
{
   while (TRUE) {
      down(customers);       /* go to sleep if # of customers is 0 */
      down(mutex);           /* acquire access to 'waiting' */
      waiting = waiting - 1;  /* decrement count of waiting customers */
      up(barbers);           /* one barber is now ready to cut hair */
      up(mutex);             /* release 'waiting' */
      cut_hair();            /* cut hair (outside critical region) */
   }
}
```

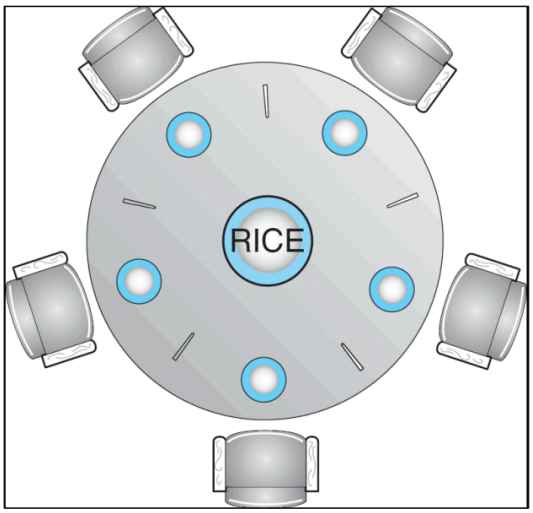
1. Quando il barbiere arriva la mattina, si blocca con un **down()** sul semaforo customers (poiché == 0).
2. Va a dormire.
3. Resta a dormire fino a che non viene svegliato da un customer.

customer: procedura eseguita dal cliente.

```
void customer(void)
{
    down (&mutex);                  /* enter critical region */
    if (waiting < CHAIRS) {         /* if no free chair, leave */
        waiting += 1;               /* waiting customers ++ */
        up(&customers);             /* wake up barber if needed*/
        up(&mutex);                 /* il barber riprende l'haircut*/
        down(&barbers);             /* a dormire se barber occupato*/
        get_haircut();
    } else {
        up(&mutex);                 // shop full, leave
    }
}
```

1. Acquisisce mutex, entrando nella critical region. Un secondo client che dovesse entrare non potrebbe fare niente finché il primo non avesse rilasciato mutex;
2. Verifica il numero di waiting. Se maggiore del numero di sedie va via senza haircut;
3. Altrimenti incrementa waiting;
4. Si mette in fila (sveglia il barbiere, se necessario);
5. Resta a dormire fino a che non viene chiamato dal barber.

Dining Philosopher Problem



I 5 filosofi a cena:
- 5 filosofi su 5 sedie;
- 5 bacchette per prendere il riso posto in una zuppiera condivisa;
- Un filosofo può prendere una bacchetta alla volta, solo se è libera e se è posta tra lui e un vicino:
- Quando il filosofo non mangia pensa e non interagisce con gli altri;
- Quando il filosofo ha due bacchette mangia, dopo di che lascia le bacchette e torna a pensare.
- Deve essere possibile che 2 filosofi possano mangiare contemporaneamente.
- Processi in competizione per l'accesso esclusivo a risorse in numero limitato.

Illustra problemi di **base** di **deadlock** e **starvation**; **caso di test** standard per valutare gli approcci alla **sincronizzazione**.

Caratteristiche:

#	Scopo Didattico	Descrizione
1	Storico e formativo	proposto da Edsger W. **Dijkstra** nel **1965** per illustrare la concorrenza e le risorse condivise.
2	combina concetti fondamentali di sincronizzazione	incapsula **tre** tipici "**nemici**" della **concorrenza**: *mutua esclusione*, *deadlock*, *starvation*.
3	modello astratto ma rappresentativo	**processi** che devono accedere a **risorse condivise**
4	esplorare strategie di soluzione	**palestra** per capire come progettare algoritmi concorrenti robusti.
5	Evidenzia limiti di concorrenza e scalabilità	facile ridurre a "serializzazione completa"

Il problema coinvolge più processi (i "filosofi") che condividono più risorse (le "forchette"/ "bacchette") e la sua soluzione deve affrontare e risolvere 3 nemici della concorrenza:

Mutua Esclusione	**Ogni risorsa** può essere usata da un **solo processo alla volta**.
Dealock	Un processo deve acquisire *due* **risorse simultaneamente** (o concorrentemente).
Starvation	**Evitare** che un **processo** non venga **mai servito**.
Efficiency	**Massimizzare** il numero di **processi** che fanno uso delle **risorse condivise**.

Il problema mappa perfettamente scenari reali nei sistemi di elaborazione: processi che devono accedere a risorse condivise.

Applicazione	Esempio
Allocazione di risorse hardware	diversi processi devono accedere a periferiche o blocchi di memoria condivisi — se ogni processo aspetta su più risorse, si può generare deadlock.
Transazioni nei DB	più **transazioni** possono richiedere **più lock** (es. tabelle, righe) - se ognuna acquisisce un lock e aspetta un altro, può verificarsi la stessa situazione dei filosofi.
Multitasking e Multithreading	**thread diversi** possono aver bisogno di accedere a **risorse condivise**. Il modello "prendi lock A poi lock B" è comune — se tutti seguono lo stesso schema si può incorrere negli stessi meccanismi relativi all'accesso alle forchette del problema ➔ ordinamento delle risorse (Circular Wait) ➔ impedire accesso contemporaneo (Hold&Wait).
Sistemi distribuiti e risorse condivise	L'**attesa** è più **complessa**, occorre ottimizzare Circular Wait e Hold & Wait
Ambienti embedded / realtime	"**paper-case**" per testare protocolli leggeri e robusti per **non bloccare** le risorse e la risposta dei **servizi**.

Dipartimento di Ingegneria Informat
Automatica e Gestionale Antonio Rub

Il problema non si risolve semplicemente con "metti un lock globale", perché ciò ridurrebbe la concorrenza. Le soluzioni efficaci richiedono di applicare:

Tecnica	Descrizione
Limite al numero di Processi interessati	Meno processi in fase rischiosa, meno Hold&Wait, meno Deadlok
Ordinamento delle Risorse	Permette di evitare l'Attesa circolare
Protocolli più complessi	Superare lo schema a 2 semafori del Produttore-Consumatore

Soluzioni

N semafori:

```
#define N 5                     /*number of philosophers*/
semaphore fork [N]
void philosopher(int i)         /*i:philosopher number, from 0 to 4*/
{
while(TRUE){
      think();                  /*philosopher is thinking*/
      wait (fork[i]);           /*take left fork*/
      wait (fork[(i+1)%N]);     /*take right for;% is modulo operator*/
      eat():                    /*self-explanatory*/
      signal (fork[(i+1)%N]);   /*put right fork back on table*/
      signal (fork[i]);         /*put left fork back on table*/
      }
}
```

- Ogni forchetta è un semaforo (**fork[i]**). Il filosofo **i** fa **wait** sulla forchetta sinistra (**fork[i]**), poi sulla destra (**fork[(i+1)%N]**), mangia e fa **signal** su entrambe.
- Il problema è che le due forchette vengono prese **una alla volta**. Se tutti e 5 prendono la sinistra nello stesso momento, ognuno aspetta la destra, che ha in mano il vicino: **deadlock** (e nessuno mangia).
- Lezione: prendere le risorse una per volta, tenendo quella già presa, porta all'attesa circolare (hold & wait + circular wait).

Soluzione errata:
- **starvation**: se tutti i filosofi prendono la forchetta sinistra, nessuno riesce a prendere la destra, nessuno mangia.
- **deadlock**: tutti i filosofi sono in attesa di una azione da parte degli altri.

Mutex:

```
#define N 5                     /*number of philosophers*/
typedef int semaphore;
semaphore mutex = 1;
void philosopher(int i)         /*i:philosopher number, from 0 to 4*/
{
while(TRUE){
      think();                  /*philosopher is thinking*/
      down (&mutex) ;           /* entering critical section */
      take_fork(i);             /*take left fork*/
      take_fork(i+1)%N;         /*take right for;% is modulo operator*/
      eat():                    /*self-explanatory*/
      put_fork(i);              /*put left fork back on table*/
      put_fork(i+1)%N;          /*put right fork back on table*/
      up (&mutex)               /* exiting critical section */
      }
}
```

- Tutto il blocco "prendi forchette, mangia, rimetti" è dentro un unico **down(&mutex)** / **up(&mutex)**.
- Niente deadlock, perché mentre un filosofo mangia gli altri non possono nemmeno tentare di prendere le forchette.
- Mangia **Un solo filosofo per volta**, anche se due non adiacenti potrebbero mangiare insieme.
- Lezione: il mutex protegge troppo, e tiene bloccato tutto anche durante **eat()**.

Room semaphore

```
#define N 5                     /*number of philosophers*/
typedef int semaphore;
semaphore fork[N] = {1};
Semaphore room = {N-1}
void philosopher(int i)         /*i:philosopher number, from 0 to 4*/
{
while(TRUE){
      think();                  /*philosopher is thinking*/
      wait (room) ;             /* setting interested status */
      wait (fork[i]);           /*take left fork*/
      wait (fork[(i+1)%N]);     /*take right for;% is modulo operator*/
      eat():                    /*self-explanatory*/
      signal (fork[(i+1)%N]);   /*put right fork back on table*/
      signal (fork[i]);         /*put right fork back on table*/
      signal (room)             /* exiting interested status*/
      }
}
```

- Si aggiunge un semaforo **room** inizializzato a **N-1** (=4): al tavolo possono “entrare” al massimo 4 filosofi per volta. Il filosofo fa **wait(room)** prima di prendere le forchette e **signal(room)** alla fine.
- Con 5 forchette e al massimo 4 filosofi interessati, **almeno uno** riesce sempre a prendere entrambe le forchette, quindi il deadlock non può più formasi. La soluzione è libera da deadlock e starvation.
- Le forchette sono ancora prese una alla volta, ma con questo limite il ciclo non si chiude.
- Lezione: si evita il deadlock **limitando quanti competono**, senza bloccare tutto.

Mutex + status

Invece di prendere le forchette una alla volta (rischio di deadlock), ogni filosofo prova a prenderle **entrambe insieme**, e ci riesce solo se i due vicini non stanno mangiando. Le forchette non sono rappresentate esplicitamente: “avere le forchette” significa “nessuno dei due vicini sta mangiando”.

```
#define N 5                     /* number of philosophers */
#define LEFT (i+N-1)%N          /* number of i's left neighbor */
#define RIGHT (i+1)%N           /* number of i's right neighbor */
#define THINKING 0              /* philosopher is thinking */
#define HUNGRY 1                /* philosopher is trying to get forks */
#define EATING 2                /* philosopher is eating */
typedef int semaphore;          /* semaphores are a special kind of int */
int state[N];                   /* array to keep track of everyone's state */
semaphore mutex = 1;            /* mutual exclusion for critical regions */
semaphore s[N];                 /* one semaphore per philosopher */

void philosopher(int i)         /* i: philosopher number, from 0 to N-1 */
{
   while (TRUE) {               /* repeat forever */
      think();                  /* philosopher is thinking */
      take_forks(i);            /* acquire two forks or block */
      eat();                    /* yum-yum, spaghetti */
      put_forks(i);             /* put both forks back on table */
   }
}
```

```
void take_forks(int i)          /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);               /* enter critical region */
    state[i] = HUNGRY;          /* record fact that philosopher i is hungry */
    test(i);                    /* try to acquire 2 forks */
    up(&mutex);                 /* exit critical region */
    down(&s[i]);                /* block if forks were not acquired */
}

void put_forks(i)               /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);               /* enter critical region */
    state[i] = THINKING;        /* philosopher has finished eating */
    test(LEFT);                 /* see if left neighbor can now eat */
    test(RIGHT);                /* see if right neighbor can now eat */
    up(&mutex);                 /* exit critical region */
}

void test(i)                    /* i: philosopher number, from 0 to N-1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

Soluzione efficiente: sono definiti N+1 semafori.
- **S[N]**: per tracciare la richiesta di risorsa condivisa di ogni filosofo;
- **mutex**: usato per mutua esclusione. Garantisce che solo un processo alla volta possa leggere/scrivere sul buffer.

Altre variabili:
- **state[N]**: array di stato/vettore per tener traccia dello stato di ciascun filosofo: **THINKING**, **HUNGRY**, **EATING**.
- **LEFT** e **RIGHT**: gli indici dei vicini di **i**, calcolati con il modulo **N**.

philosopher(i): ciclo infinito: pensa, **take_forks(i)** (prende le forchette o si blocca), mangia, **put_forks(i)** (le rimette a posto).

test(i):
- Se **i** ha fame e né il vicino sinistro né il destro stanno mangiando, allora **state[i] = EATING** e **up(&s[i])**: questo sblocca **i** (se stava già aspettando) oppure lo lascia passare subito al **down** di **take_forks**.

put_forks(i):
1. **down(&mutex)**, poi **state[i] = THINKING**: ha finito di mangiare.
2. **test(LEFT)** e **test(RIGHT)**: controlla se i vicini, che magari aspettavano affamati, ora possono mangiare. Se si, li sveglia con **up(&s[…])**.
3. **up(&mutex)**.

Perche funziona:
- Niente deadlock sulle forchette: la decisione “posso prenderle entrambe?” è presa dentro la ragione critica, quindi non esiste la situazione in cui ognuno ha una forchetta e aspetta l’altre.
- E’ **efficiente**: il **mutex** protegge solo il controllo e l’aggiornamento di **state[]** (pochi istanti) e **non** viene tenuto mentre si mangia, quindi i due filosofi non adiacenti possono mangiare

contemporaneamente (a differenza della soluzione con un solo mutex, dove mangia un filosofo per volta).

Approcci software alla mutua esclusione

La mutua esclusione può essere implementata anche a livello software. Cioè solo a livello software, senza l'aiuto dell'hardware (niente TSL, niente semafori).
Scenario: processi che comunicano tramite una memoria centrale su una macchina single/multi-processore.
Assunzione: mutua esclusione a livello di accesso alla memoria.
- Le operazioni di lettura/scrittura sulla stessa locazione sono serializzate da una sorta di arbitro della memoria (l'ordine non è noto).
- Nessun supporto da sistema operativo, hardware o linguaggio di programmazione.

Dijkstra iniziò a studiare il problema della concorrenza, producendo un algoritmo soggetto a deadlock. Successivamente Dijkstra riportò un algoritmo ideato dal matematico olandese Dekker:
- mutua esclusione;
- 2 processi.

Oltre 10 anni dopo, Peterson propose la soluzione più efficiente.

Algorithm	Year	Contribution	Relation to previous work
Dijkstra	1965–68	Formally defines the problem and introduces general algorithms. Deadlock.	Theoretical foundation
Dekker	~1968	First correct solution for 2 processes using busy waiting	Applies Dijkstra's criteria
Peterson	1981	Simpler and verifiable solution for the 2-process case	Simplifies Dekker's approach

Algoritmo di Dekker

Premessa: Ci sono due processi, P0 e P1. In ogni processo **me** è il proprio numero e **other** quello dell'altro. NCS è la sezione non critica (il processo fa altro), CS è la sezione critica (l'accesso esclusivo). Il **while(…) ;** vuoto è il busy waiting: il processo continua a controllare la condizione finche non cambia.

Algoritmo di Dijkstra - 1^ tentativo

```
/* global */
int turn = 0;

// assignments valid for P0 (flip for P1)
int me = 0, other = 1;
while (true) {
    /*NCS*/
    while (turn != me)
        /* busy wait */ ;
    /* CS */
    turn = other;
}
```

- **turn** dice di chi è il turno. Un processo aspetta finché **turn == me**, entra in CS e poi passa il turno all'altro (**turn = other**).
- La mutua esclusione è garantita, perche il turno è di uno solo alla volta.
- Il problema è l'**alternanza forzata**: se P1 non vuole entrare (o si blocca), P0 può entrare una volta, passa il turno a P1 e poi aspetta per sempre. Per questo c'è **starvation**, e il processo più lento detta il ritmo.

Algoritmo di Dijkstra - 2^ tentativo

```
/* global */
boolean flag[2] = {false, false};

// assignments valid for P0 (flip for P1)
int me = 0, other = 1;

while (true) {
    /*NCS*/
    while (flag[other])
        /* busy wait */ ;
    flag[me] = true;
    /* CS */
    flag[me] = false;
}
```

- Ogni processo ha un flag che dice “sono in CS / voglio entrare”. Si aspetta finche **flag[other]** è true, poi si alza il proprio flag ed si entra.
- Il difetto è che **controllare** e **alzare il flag** sono due istruzioni separate. P0 e P1 controllano entrambi (flag falsi), poi alzano entrambi il flag ed entrano insieme in CS: **nessuna mutua esclusione**.

Algoritmo di Dijkstra - 3^ tentativo

```
/* global */
boolean flag[2] = {false, false};

// assignments valid for P0 (flip for P1)
int me = 0, other = 1;

while (true) {
    /*NCS*/
    flag[me] = true;
    while (flag[other])
        /* busy wait */ ;
    /* CS */
    flag[me] = false;
}
```

- Si scambiano le prime due istruzioni: prima si alza il proprio flag (“voglio entrare”), poi si controlla l’altro.
- Ora la mutua esclusione c’è, ma se entrambi alzano il flag prima di controllare, vedono ognuno il flag dell’altro a true e restano bloccati nel **while** per sempre: **deadlock**.

Algoritmo di Dijkstra - 4^ tentativo

- Se l’altro vuole entrare, il processo abbassa il proprio flag, aspetta un pò (**delay**) e lo rialza, per lasciar passare l’altro.
- Il deadlock sparisce, ma se i due procedono in perfetta sincronia alzano e abbassano i flag negli stessi momenti (sequenza della slide) e nessuna entra mai: **livelock**. I processi non sono bloccati (sono attivi), ma non fanno progressi, e la cortesia reciproca non basta.

```
/* global */
boolean flag[2] = {false, false};

// assignments valid for P0 (flip for P1)
int me = 0, other = 1;

while (true) {
    /*NCS*/
    flag[me] = true;
    while (flag[other]) {
        flag[me] = false;
        /* delay */
        flag[me] = true;
    }
    /* CS */
    flag[me] = false;
}
```

Algoritmo di Dekker - 5^ tentativo

```
/* global */
turn = {0,1}
boolean flag[2] = {false, false};

// assignments valid for P0 (flip for P1)
int me = 0, other = 1; // P0 (flip for P1)

while (true) {
    /*NCS*/
    flag[me] = true;              // Voglio entrare
    while (flag[other]) {         // Se tu vuoi entrare
          if (turn == other) {       // e se è il tuo turno
                flag[me] = false;      // non voglio più entrare
                while (turn == other);  // se è il tuo turno aspetto
                flag[me] = true;       // voglio entrare
    }
}
/* CS */
turn = other;                 // Puoi entrare dopo di me
flag[me] = false;              // Non voglio più entrare
}
```

- Dobbiamo poter osservare lo stadio di entrambi i processi (**flag[2]**).
- Dobbiamo imporre un ordine delle attività (usando **turn**) per evitare il problema della “cortesia reciproca” (mutual courtesy) del tentativo precedente.
- Si aggiunge **turn** come spareggio: decide chi insiste e chi cede, evitando che cedano entrambi.
- Passo: **flag[me] = true** (voglio entrare). Se anche l’altro vuole entrare (**while (flag[other])**) e il tutto è suo (**turn == other**), cedo: abbasso il flag e aspetto che il turno cambi, poi rialzo il flag. Se invece il turno è mio, non cedo e continuo a controllare, finché l’altro abbassa il suo flag e io entro.
- All’uscita passo il turno (**turn = other**) e abbasso il flag.

Algoritmo di Peterson

- **flag[i] = true** → voglio entrare;

- **turn = j** → se c'è contesa, cedi il passo all'altro;
- **while (flag[j] && turn == j) { /* wait */ }** → aspetta finché l'altro vuole entrare e il turno è dell'altro.

```
/* global */
turn = {0,1}
boolean flag[2] = {false, false};

// assignments valid for P0 (flip for P1)
int me = 0, other = 1; // P0 (flip for P1)

/*NCS*/
flag[me] = true;
turn = 1;
while (flag[other] && (turn == other)) {
    /* busy wait */ ;
}
/* CS */
flag[me] = false;
```

- Stessa idea, scritta in modo molto più semplice, senza il **while** annidato.
- **flag[me] = true** (voglio entrare), poi **turn = 1** (per P0; in generale **turn = other**): cedo subito il turno all'altro.
- Aspetto finché l'altro vuole entrare e il turno è dell'altro (**flag[other] && turn == other**).
- Se vogliamo entrare entrambi, entrambi scriviamo **turn**, ma resta il valore scritto per ultimo: chi scrive per ultimo cede e aspetta, l'altro entra. Non c'è deadlock, perché uno dei due trova sempre la condizione falsa.
- All'uscita basta **flag[me] = false**.

Algoritmo di Dijkstra: generalizzazione a N processi

```
/* global storage */
boolean interested[N] = {false, ..., false}
boolean passed[N]     = {false, ..., false}
int k = <any>                 // k ∈ {0, 1, ..., N-1}

/* local info */
int i = <entity ID>           // i ∈ {0, 1, ..., N-1}

while (true) {
   /*NCS*/
1.  interested[i] = true
2.  while (k != i) {
3.      passed[i] = false
4.      if (!interested[k]) then k = i
    }
5.  passed[i] = true
6.  for j in 1 ... N except i do
7.      if (passed[j]) then goto 2
8.  <critical section>
9.  passed[i] = false; interested[i] = false
}
```

La complessità di questo algoritmo sta nella doppia scansione:

- **Upward** (**j < i**): il processo **i** deve assicurarsi che tutti i processi "più vecchi" (con indice più basso) che vogliono entrare abbiano la precedenza o abbiano finito.

- **Downward** (**j > i**): il processo **i** deve poi controllare che nessun processo “più giovane” (con indice più alto) sia riuscito a sorpassarlo e sia pronto a entrare.

L’algoritmo funziona in due fasi: nella prima si sceglie un candidato tra chi vuole entrare, nella seconda si verifica che il candidato sia uno solo.

Riga 1: il processo **i** mostra interessa a entrare in **CS**.
- **interested[i] = true**: il processo dichiara “voglio entrare”. Finché non esce dalla **CS** resta interessato.

Riga 2-4: **k** “sceglie” tra i processi che hanno mostrato interesse (fase uno).
- **k** è una variabile condivisa che indica il processo “scelto” al momento.
- Se **k != 1**, il processo **i** non è il prescelto e resta nel **while**. Ogni volta si mette **passed[i] = false** (**riga 3**: “non ho ancora superato la fase uno”) e guarda il prescelto **k**.
- Se **k** non è interessato (**!interested[k]**: ha finito o non vuole entrare), **i** si prende il posto: **k = i** (**riga 4**). Se invece **k** è interessato, **i** aspetta.
- Quando **k == i**, il processo esce dal **while**: è diventato il prescelto.

Riga 5: fase uno superata dal processo **i**.
- **passed[i] = true**: **i** segnala agli altri che ha superato la prima fase. Passare la fase uno non basta per entrare, perché più processi potrebbero averla superata quasi insieme.

Riga 6-7: riparti se più di un processo ha superato la fase uno (fase due)
- Il **for** controlla tutti gli altri processi **j**: se qualcuno ha **passed[j]** true, **i** non è il solo, quindi **goto 2** e riparte dalla fase uno.
- Questo capita quando due processi vedono **k** non interessato quasi contemporaneamente e scrivono entrambi **k = i**: entrambi superano la fase uno. Ripartendo, resta prescelto solo quello indicato da **k**, e gli altri tornano ad aspettare nel **while**.
- Se nessun altro ha **passed** true, **i** è l’unico e può entrare.

Riga 8: sezione critica.

Riga 9: uscita.
- **passed[i] = false; interested[i] = false**. Mettendo **interested[i]** a false, **i** non è più interessato: così gli altri, vedendo **!interested[k]**, possono prendere il posto di prescelto (**riga 4**).

Esempio. $N \geq 4$, inizialmente k = 4

First scan (prima scansione)

P1	P2	P3
intr[1] = true		
while (k!=1)		
pass[1] = false		
	intr[2] = true	
	while (k!=2)	
	pass[2] = false	
		intr[3] = true
		while (k!=3)
		pass[3] = false
if (!intr[4])		
	if (!intr[4])	
		if (!intr[4])
k = 1		
	k = 2	
		k = 3

(**intr** e **pass** sono le abbreviazioni usate per **interested** e **passed**).

Second scan (seconda scansione)

I processi P1, P2 e P3 sono ora pronti a eseguire le righe 5-7:
- Solo uno di loro potrà accedere alla CS.
- k = 3 non implica che P3 vada per primo: k è usato solo per risolvere i conflitti, quindi lo scheduler può far entrare per primo anche P1 o P2!
- Un conflitto sorge quando due processi impostano **pass[i] = true** prima che l'altro sia entrato e poi uscito dalla sezione critica.

Una possibile continuazione senza conflitti: lo scheduler lascia eseguire P3 con P1 e P2 ancora "fermi" alla riga 5.

	P3
5	pass[3] = true
6	for j in 1..N except 3 do
7	if pass[j] goto 2 *(skipped)*
	<critical section>
	pass[3] = false
	intr[3] = false

P3 finished (P3 ha finito)

P1	P2
while (k!=1)	
pass[1] = false	
	while (k!=2)
	pass[2] = false
if (!intr[3]) *(true: P3 done)*	
	if (!intr[3]) *(true: P3 done)*
then k = 1	
while (k!=1) *(skipped)*	
pass[1] = true	
	then k = 2
	while (k!=2) *(skipped)*
if (pass[2]) goto 2 (taken!!)	
	if (pass[1]) goto 2 (taken!!)

Quando si verifica un conflitto, l'algoritmo:
- torna alla riga 2
- azzera **pass[i]** tranne il processo che ha impostato k per ultimo! (qui P2).

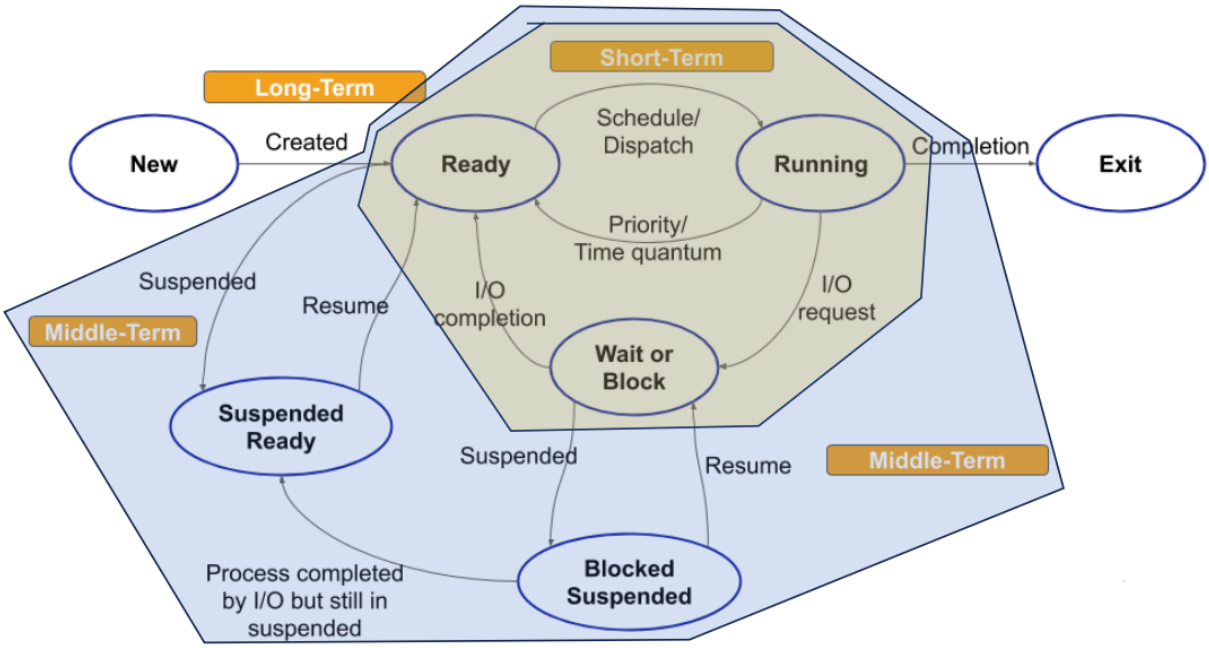
In un Algoritmo di Dijkstra a N processi generale valgono le seguenti caratteristiche:
- **Mutua esclusione**: si entra solo dopo aver visto che nessun altro ha superato la fase uno.
- **No deadlock**: **k** punta sempre a un solo processo, che prima o poi supera entrambe le fasi.
- **Starvation non garantita**: l'algoritmo no impone un ordine equo, quindi un processo sfortunato può trovarsi a ripartire più volte (e un altro può prendere **k** prima di lui).
- Richiede lettura/scrittura atomiche (su **k**, **interested**, **passed**) e memoria condivisa per **k**.

Schedulatori

Durante la sua vita un processo passa continuamente da uno stato all'altro: New, Ready, Running, Wait or Block, Suspended Ready, Blocked Suspended ed Exit. Poiché le risorse sono limitate (la CPU è una sola, la memoria è finita), il sistema operativo deve decidere quali processi ammettere,

quali tenere in memoria e quale mandare in esecuzione. Queste decisioni sono prese dagli schedulatori.

Gli schedulatori sino tre e si distinguono per l'orizzonte temporale e la frequenza con cui intervengono:
- il **Long-Term** decide quali programmi entrano nel sistema (ogni secondi/minuti) e controlla cosi il grado di multiprogrammazione;
- il **Middle-Term** decide quali processi spostare dalla memoria centrale alla memoria di massa (swapping) e viceversa;
- lo **Short-Term** decide, a ogni ms, quale processo pronto riceve la CPU.



Short-term Scheduler

Selezione quale processo (in memoria) deve essere eseguito e alloca la CPU ad esso ogni ms (jiffies). Ecco le principali caratteristiche:
- Noto come **dispatcher**;
- Si esegue più frequentemente;
- Prende la decisione a grana fine su quale processo eseguire dopo;
- Invocato quando si verifica un evento che può portare al blocco del processo corrente o che può offrire l'opportunità di sottrarre la CPU (preempt) a un processo in esecuzione a favore di un altro.

Esempi:
- interrupt di clock;
- interrupt di I/O;
- chiamate di sistema;
- segnali (es. semafori).

Criteri di short-term scheduling:
- L'obiettivo principale è allocare il tempo del processore per ottimizzare certi aspetti del comportamento del sistema.
- Serve un insieme di criteri per valutare la politica di scheduling:
  - **Criteri orientati all'utente**: Relativi al comportamento del sistema percepito dal singolo utente o processo (come il tempo di risposta in un sistema interattivo). Importanti su praticamente tutti i sistemi.

- **Criteri orientati al sistema**: Si concentrano sull'utilizzo efficace ed efficiente del processore (frequenza con cui i processi vengono completati). In genere di minore importanza nei sistemi single-user.

I criteri, inoltre, possono essere classificati in:
- **Criteri legati alle prestazioni** (performance-related):
  - quantitativi;
  - facilmente misurabili:
  - esempi: tempo di risposta (response time), throughput.
- **Criteri non legati alle prestazioni** (non-performance related):
  - quantitativi;
  - difficili da misurare;
  - esempio: prevedibilità.

Medium-term Scheduler

Esegue lo swapping:
- rimuovere un processo dalla memoria centrale e lo pone in memoria di massa;
- supportata solo da alcuni SO come i sistemi time-sharing;
- le decisioni di swapping-in si basano sulla necessità di gestire il grado di multiprogrammazione:
  - considera i requisiti di memoria dei processi di cui è stato fatto lo swap-out.

Long-Term Scheduling

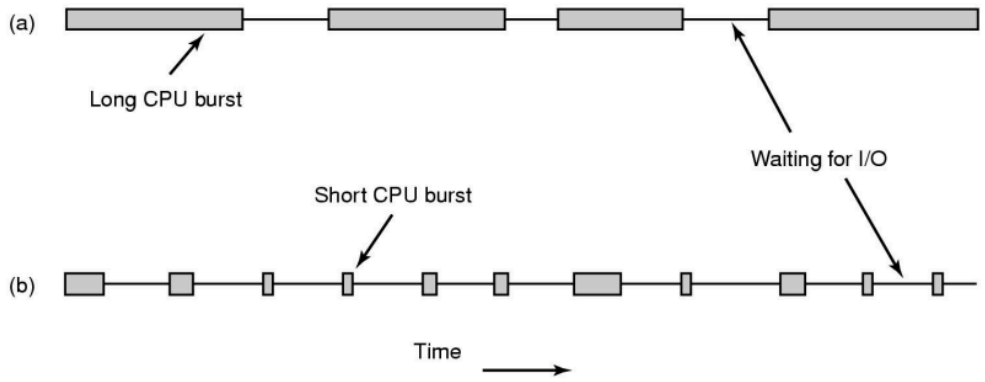
Seleziona quale processo (attualmente in memoria di massa) deve essere inserito nella coda dei processi pronti, con frequenza nell'ordine dei secondi/minuti:
- controlla il grado di multi-programmazione:
  - più processi vengono creati, minore è la percentuale di tempo in cui ciascun processo può essere eseguito;
  - può limitare il numero per fornire un servizio soddisfacente all'insieme corrente di processi.
- stabile se velocità di creazione = velocità di terminazione dei processi;
- richiamato solo quando un processo abbandona il sistema (termina);
- assente nei sistemi UNIX e Windows;
- determina quali programmi sono ammessi nel sistema per l'elaborazione;

Scheduling

Criteri di ottimizzazione:
1. **Utilizzo della CPU** (CPU usage): la CPU deve essere attiva il più possibile. In un sistema reali si va dal 40% (sistema poco carico) al 90% (utilizzo intendo).
2. **Frequenza di completamento** (throughput): numero di processi completati per unità di tempo.
3. **Tempo di completamento** (turnaround time): intervallo che va dal momento dell'immissione del processo nel sistema al momento del completamento (comprende tempi di esecuzione, tempi di attesa nella faire code).
4. **Tempo di attesa** (time-sharing): somma dei tempi spesi in attesa nella coda dei processi pronti (l'algoritmo di scheduling influisce solo sul tempo di attesa, non sul tempo di esecuzione).
5. **Tempo di risposta** (response time): tempo che intercorre dalla formulazione della richiesta fino alla produzione delle prima risposta (si conta il tempo necessario per iniziare la risposta, non per emetterla completamente).

In generale si ottimizza:
- il valore medio e/o
- il valore minimo/massimo e/o
- la varianza (per sistemi time-sharing, si preferisce minimizzare la varianza nel tempo di risposta: meglio un sistema predicibile che uno più veloce ma maggiormente variabile).

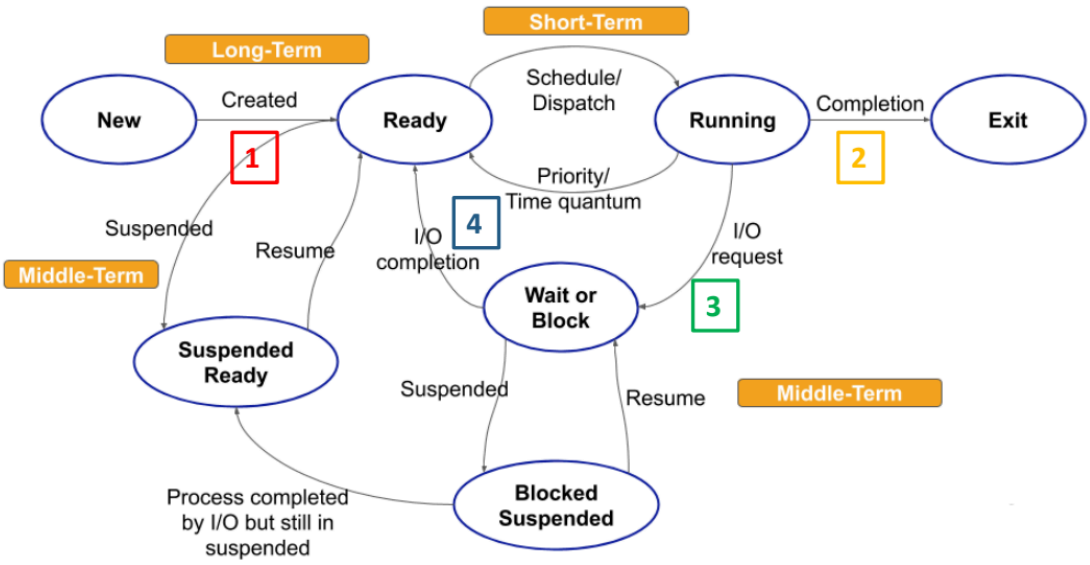
Obiettivi:
Assegnare a ogni processore i processi da eseguire, man mano che i processi stessi vengono creati e distrutti. Realizzare la turnazione dei processi sul processore in modo da garantire:
- **CPU Usage**: massimizzarne lo sfruttamento della CPU;
- **Time-Sharing**: creare l'illusione di evoluzione contemporanea dei processi;
- **Response Time**: minimizzare il tempo di risposta;
- **Throughput**: aumentare il numero di job terminati per unità di tempo (ora, generalmente).
- **Turnaround**: diminuire il tempo statistico medio necessario per il completamento di un lavoro.



Il seguente grafico mostra come due diversi tipi di processo usano il tempo (asse orizzontale), alternando fasi di CPU e attese di I/O.
- I blocchi grigi sono i CPU burst: i periodi in cui il processo sta usando la CPU per calcolare.
- Le linee sottili tra un blocco e l'altro sono le attese di I/O (waiting for I/O): il processo è fermo ad aspettare, ad esempio, il disco o la rete.

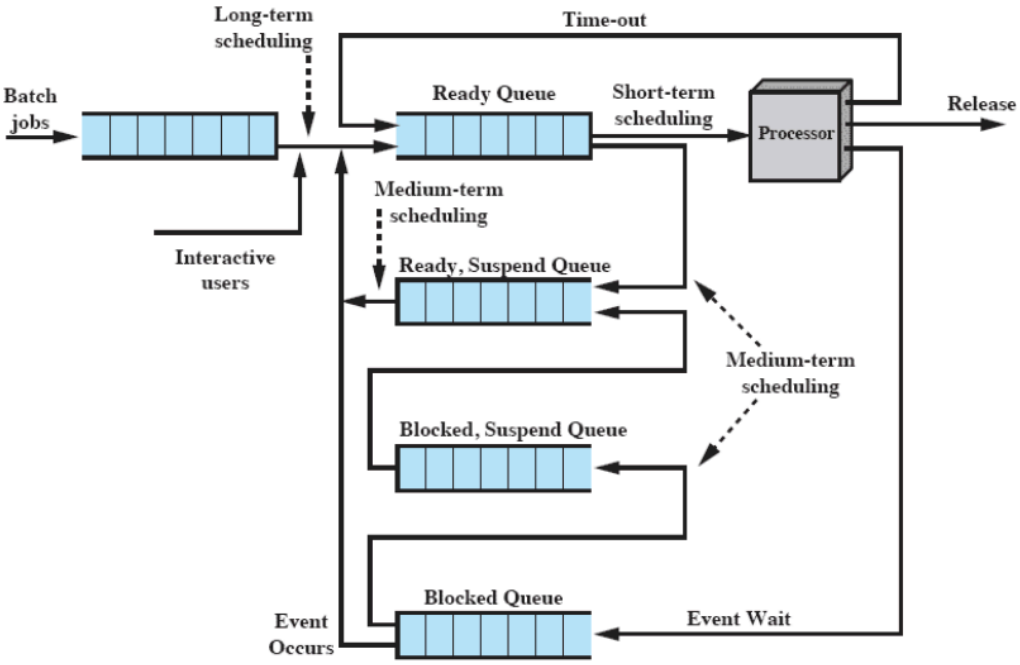
(a) **Processo CPU-bound**: pochi CPU burst, lunghi, con brevi attese di I/O in messo. Passa la maggior parte del tempo a calcolare.
(b) **Processo I/O-bound**: tanti CPU burst corti, separati da attese di I/O frequenti. Usa la CPU solo per poco tempo e poi si blocca di nuovo in attesa di I/O.

Eventi che attivano lo scheduling:
1. **Created**: decide se eseguire il processo padre o il figlio.
2. **Completion**: decide quale processo sostituisce quello uscito.
3. **I/O Request/Semaphore**: il processo si è messo in attesa, qualcun altro lo deve sostituire.
4. **I/O Completion**: una operazione di I/O necessaria è stata eseguita, rendendo pronto il processo che l'ha chiesta.



- non Pre-Emptive: senza sospensione della esecuzione → 2. Completion, 3. I/O Request/ Semaphore;
- Pre-Emptive: sospensione della esecuzione (saved & restored) → 1. Created, 4. I/O Completion.

Queuing diagram



Il seguente diagramma mostra il percorso dei processi tra le code e il processore, e dove intervengono i tre schedulatori:
- **Ingresso** (long-term scheduling): i batch jobs e gli interactive users vengono ammessi nel sistema e finiscono nella **Ready Queue**.
- **Short-term scheduling**: dalla Ready Queue lo schedulatore a breve termine sceglie il processo che va sul **Processor**.
- Tre modi di lasciare il processore:
  - **Release**: il processo termina e esce dal sistema;
  - **Time-out**: scade il quanto di tempo e il processo torna in Ready Queue;
  - **Event Wait**: il processo aspetta un evento (es. I/O) e va nella Blocked Queue.
- **Event Occurs**: quando l'evento si verifica, il processo torna dalla Blocked Queue alla Ready Queue.
- **Medium-term scheduling** (swapping): se la memoria è piena, processi pronti o bloccati vengono sospesi e spostati nella Ready, Suspend Queue o nella Blocked, Suspend Queue (in memoria di massa). Quando serve, tornano in memoria e rientrano in Ready Queue.

Caricamento del processo

Il **dispatcher** (caricatore sulla CPU) è il modulo del SO che da il controllo della CPU a un processo selezionato dallo schedulatore a breve termine. Lo schedulatore decide quale processo eseguire, il dispatcher lo manda realmente in esecuzione. Questa funzione comprende:
1. cambio di contesto (context switch);
2. passaggio alla modalità utente;
3. salto alla corretta locazione nel programma utente per ricominciarne l'esecuzione.

Latenza del dispatcher: tempo necessario al dispatcher per ferma un processo e cominciarne un altro. Durante questo tempo la CPU non fa lavoro utile, quindi il dispatcher deve esse il più veloce possibile.

Context switch
Il context switch è la somma di due operazioni:
- **sospensione** del processo in esecuzione;
- **attivazione** del processo da metter in esecuzione.

In pratica il sistema salva “a che punto era” il processo che lascia la CPU e ricarica “a che punto era” quello che la riceve, cos+ che ciascuno riprende come se non fosse mai stato interrotto.

Sospensione del processo in esecuzione
Può avvenire attraverso una chiamata:
- **sincrona rispetto alla computazione**, in **stato supervisore** (in procedure di I/O, creazione processi);
- **sincrona rispetto alla computazione**, in **stato utente** (in rilascio volontario);
- **asincrona rispetto alla computazione** (allo scadere del time slide nel time sharing).

Sincrona significa che è il processo stesso a causarla, in un punto preciso del suo codice.
Asincrona significa che arriva dall’esterno, senza che il processo se ne accorga (ad esempio l’interrupt del timer).

Salvataggio del contesto di esecuzione
- salvare tutti i registri del processore sullo **stack**;
- salvare lo stack pointer nel **Process Contro Block** (PCB).

Il **PCB** è la struttura dati in cui il SO conserva le informazioni di ogni processo. Salvare lo stack pointer basta a ritrovare poi tutti i registri, perché stanno sullo stack.

Attivazione del processo da mettere in esecuzione
Ripristina del contesto di esecuzione:
- ripristinare il valore del registro che punta alla base dello stack, prendendolo dal PCB del processo da riattivare;
- ripristinare il valore dello stack pointer, prendendolo dal PCB del processo da riattivare;
- ripristinare tutti i registri del processore, prendendoli dallo stack.

E’ l’operazione inversa del salvataggio: l’ultimo passo ricarica anche il program counter, e il processo riprende dall’istruzione in cui era stato fermato.

Selection Function
La funzione di selezione:
- determina quale processo, tra quelli pronti, viene selezionato per l’esecuzione successiva;
- può basarsi sulla priorità, sui requisiti di risorse o sulle caratteristiche di esecuzione del processo.

Se si basa sulle caratteristiche di esecuzione, le quantità importanti sono:
- **w** = tempo trascorso finora nel sistema, in attesa;
- **e** = tempo trascorso finora in esecuzione;
- **s** = tempo di servizio totale richiesto dal processo, incluso **e**; in generale questa quantità deve essere stimata o fornita dall’utente.

Da queste si ricavano altre quantità utili: **w + e** è il tempo totale passato nel sistema finora, e **s – e** è il tempo di servizio che resta al processo. Le politiche di scheduling che vedrai (FCFS, SJF, ecc.) si distinguono proprio per quale di queste grandezze usano per scegliere.

INDICE

Sistemi operativi **1**

Tipologie di Sistemi di Elaborazione **2**

Sistemi Monoprocessore 2

Sistemi Multiprocessore 2

Caratteristiche di un sistema SMP 3

Computer multicore 4

Categorie di sistemi di calcolo (tassonomia di Flynn) 4

Sistemi Distribuiti 6

Cluster 6

Cloud Computing 6

Mainframe classico 7

Mainframe moderno (HPC) 7

Sistema dedicato (embedded) 7

Sistemi di elaborazione in tempo reale (real-time) 7

Sistemi multimediali 7

PDA/Smartphone 8

Microkernel **8**

Progettazione del microkernel 8

Vantaggi di un'organizzazione a microkernel 9

S.O. : Obiettivi **9**

Input/Output 9

Virtualizzazione 11

Astrazione 12

S.O. : Funzioni **12**

Error detection 12

Dual-mode 12

Process Management/Memory Management 13

File system 13

Shell/GUI/Transaction 14

Interrupt handling 15

IPC (Inter-Process Communication) 16

I Processi **22**

PCB 23

Context Switch 24

Unix SysCall: fork(), wait(), exit() **24**
Creazione di nuovi processi 25
Creazione di processi 25
Fork() 25
Terminazione del processo 26
Thread **27**
PCB e TCB 28
Thread concorrenti e paralleli 29
Thread in processi utente 30
Thread in processi server 30
Modello di thread: User vs Kernel Level 31
Modello multi-thread 31
Librerie per i thread 34
Pthread - User Level Thread 34
System call fork() ed exec() 35
Thread in Linux 35
Thread safety 35
Concorrenza: mutua esclusione e sincronizzazione **36**
Termini chiave pt.1 36
Critical Region 37
Termini chiave pt.2 37
Operating System Concerns: interazione tra processi 37
Principi e difficoltà 38
Mutua Esclusione **38**
Mutual Exclusion with Busy Waiting 39
Turn Variable 39
Interrupt Disabling 40
Exchange instruction 40
Compare&Swap Instruction 40
Hardware Support 41
Concorrenza: Semafori **41**
Definizione della primitiva per semafori a interi 42
Definizione della primitiva per semaforo binario 43
Strong/Weak Semaphores 43
MutEx 44
Implementazione dei semafori 44

Condition Variable 46
Concorrenza: Altri meccanismi 46
Monitor 47
Event Flags 47
Mailboxes/Message 48
Barriers 50
SpinLock 50
Concorrenza: producer-consumer 51
Resource sharing 51
Producer/Consumer Problem 51
Producer/Consumer con semafori binari (buffer infinito) 51
Producer/Consumer con semafori generali (buffer infinito) 52
Buffer Circolare 53
Bounded-buffer Producer/Consumer 53
Bounded-buffer P/C using Monitors 54
Bounded-buffer Producer/Consumer using messages 55
Soluzione con semafori (priorità ai lettori) 57
Soluzione con semafori (priorità agli scrittori) 57
Soluzione con scambio di messaggi 59
IPC (Inter Process Communication) 59
IPC: Meccanismi 60
IPC: Modello di comunicazione a memoria condivisa 61
IPC: Modello di comunicazione a scambio di messaggi 64
Pipe 64
FIFO 66
FIFO: Unix vs Windows 68
Segnali 69
Socket 70
Deadlock 70
Condizioni necessarie 71
Categorie di risorse (Resource Categories) 71
Risorse riusabili 71
Altri esempi con risorse riusabili 73
Risorse consumabili 73
Modellizzazione del Deadlock: Resource Allocation Graph 73
Cicli nel grado e deadlock 77

Deadlock: metodi di gestione 77
Prevent 77
Avoid 79
Avoid: Algoritmo del banchiere 81
Avoid: Pro e contro 85
Detect & Recovery 86
Approach Comparison 87
Ignore 87
Sleeping Barber Problem 88
Dining Philosopher Problem 90
Approcci software alla mutua esclusione **95**
Algoritmo di Dekker 95
Algoritmo di Dijkstra: generalizzazione a N processi 98
Schedulatori **100**
Short-term Scheduler 101
Medium-term Scheduler 102
Long-Term Scheduling 102
Scheduling 102
Queuing diagram 104
Caricamento del processo 104